

```

while(t--) initwsolve();
return 0;
}

```

XÂU ĐỐI XỨNG 1 chưa

ôn lại

Cho xâu ký tự str. Nhiệm vụ của bạn là tìm số phép chèn tối thiểu các ký tự vào str để str trở thành xâu đối xứng. Ví dụ: str = "ab" ta có số phép chèn tối thiểu là 1 để trở thành xâu đối xứng "aba" hoặc "bab". Với xâu str="aa" thì số phép chèn tối thiểu là 0. Với xâu str="abcd" có số phép chèn tối thiểu là 3 để trở thành xâu "dcbabcd"

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu ký tự được viết trên một dòng
- T, str thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(\text{str}) \leq 40$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
3	3
abcd	0
aba	3
geeks	

Giải:

XÂU ĐỐI XỨNG 2 chưa

Bài làm tốt nhất

Cho xâu ký tự S. Nhiệm vụ của bạn là tìm số phép loại bỏ ít nhất các ký tự trong S để S trở thành xâu đối xứng. Chú ý, phép loại bỏ phải bảo toàn tính trước sau của các ký tự trong S.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu ký tự được viết trên một dòng
- T, str thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(S) \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	2
aebcbda	8
geeksforgeeks	

CẶP SỐ

Ôn lại

Cho N cặp số, trong đó số thứ nhất bao giờ cũng nhỏ hơn số thứ 2. Ta nói, cặp số $\langle c, d \rangle$ được gọi là theo sau cặp số $\langle a, b \rangle$ nếu $b < c$. Nhiệm vụ của bạn là tìm số lớn nhất chuỗi các cặp thỏa mãn ràng buộc trên. Ví dụ với các cặp $\{\langle 5, 24 \rangle, \langle 39, 60 \rangle, \langle 15, 28 \rangle, \langle 27, 40 \rangle, \langle 50, 90 \rangle\}$ ta có kết quả là 3 tương ứng với chuỗi các cặp $\{\langle 5, 24 \rangle, \langle 27, 40 \rangle, \langle 50, 90 \rangle\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào N là số cặp $\langle a, b \rangle$; dòng tiếp theo đưa vào $2 \cdot N$ số là N cặp số $\langle a, b \rangle$; các số được viết cách nhau một vài khoảng trống.
- T, N, a, b thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, a, b \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	
5	
5 24 39 60 15 28 27 40 50 90	3
2	1
5 10 1 11	

Giải :

Code c++:

// 1: sắp xếp mảng arr theo chiều tăng dần của số a trong từng cặp số.

//2: dp[i]: số lớn nhất chuỗi các cặp thỏa mãn ràng buộc bài toán tính đến vị trí thứ i của mảng

//3 : tại vị trí i , xét các cặp trước cặp i , nếu số đầu của cặp i > số thứ 2 của cặp j thì cặp i sẽ thỏa mãn => cập nhật max dp[j] sau đó +1 (chính cặp thứ i) ta được dp[i]

#include<bits/stdc++.h>

```

using namespace std;
int n,dp[105];
struct p{
    int a,b;
};p arr[105];
bool cmp(p x,p y){return x.a<y.a;}
void initwsolve(){
    int n;cin>>n;
    for(int i=1;i<=n;i++) cin>>arr[i].a>>arr[i].b;
    sort(arr+1,arr+n+1,cmp);//1
    memset(dp,0,sizeof(dp));
    dp[1]=1;// không cần cho =1 vẫn đúng vì khi so sánh với dp[0] có thể cập
nhập kết quả
    for(int i=1;i<=n;i++){
        for(int j=0;j<i;j++){//3
            if(arr[i].a>arr[j].b)dp[i]=max(dp[i],dp[j]+1);
        }
    }
    sort(dp,dp+n+1);
    cout<<dp[n]<<endl;
}
int main(){
    int t;cin>>t;
    while(t--){initwsolve();
        return 0;
    }
}

```

DI CHUYỂN VỀ GÓC TỌA ĐỘ

Ôn lại

Giả sử bạn đang ở điểm có tọa độ nguyên dương (n,m) và cần dịch chuyển về tọa độ $(0,0)$. Mỗi bước dịch chuyển bạn chỉ được phép dịch chuyển đến tọa độ $(n-1, m)$ hoặc $(n, m-1)$; Từ ô $(0,m)$, hoặc $(n, 0)$ thì có thể di chuyển 1 bước để về gốc $(0,0)$.

Hãy đếm số cách bạn có thể dịch chuyển về tọa độ $(0,0)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ n, m được viết cách nhau một vài khoảng trống.
- T, n, m thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m \leq 25$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
3	10
3 2	84
3 6	1
3 0	

Giải :

Code c++:

```
//1 :dp[i][j]: số cách bạn có thể dịch chuyển từ tọa độ(i,j) về tọa độ (0,0)
// 2:mọi số hàng 1 và cột 1 có 1 cách về (0,0) : =>dp[0][i]=1,dp[i][0]=1;
// 3:ta thấy có 2 cách để đi đến tọa độ (i,j):(i-1,j) đi thêm 1 bước , và(i,j-1) và tiến
1 bước => suy rộng cho các cách ta có dp[i][j]=dp[i-1][j]+dp[i][j-1];
#include<bits/stdc++.h>
using namespace std;
long long dp[26][26];//note ll
void initwsolve(){
    int n,m;cin>>n>>m;
    cout<<dp[n][m]<<endl;//1
}
int main(){
    int t;cin>>t;
    for(int i=1;i<=25;i++)dp[0][i]=1,dp[i][0]=1;//2
    for(int i=1;i<=25;i++){
        for(int j=1;j<=25;j++){
            dp[i][j]=dp[i-1][j]+dp[i][j-1];//3
        }
    }
    while (t--)>0;initwsolve();
    return 0;
}
```

XÂU ĐỐI XỨNG 1

Ôn lại

Cho xâu ký tự str. Nhiệm vụ của bạn là tìm số phép chèn tối thiểu các ký tự vào str để str trở thành xâu đối xứng. Ví dụ: str = "ab" ta có số phép chèn tối thiểu là 1 để trở thành xâu đối xứng "aba" hoặc "bab". Với xâu str="aa" thì số phép chèn tối

thieu là 0. Với xâu str="abcd" có số phép chèn tối thiểu là 3 để trở thành xâu "dcbabcd"

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu ký tự được viết trên một dòng
- T, str thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(\text{str}) \leq 40$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
3	3
abcd	0
aba	3
geeks	

Giải :

//l:dp[i][j]: số cách chèn tối thiểu của 1 chuỗi tính từ ký tự i của chuỗi đến ký tự j của chuỗi

```
#include<bits/stdc++.h>
using namespace std;
void initwsolve(){
    string s; cin>>s;
    int n=s.length();
    int dp[45][45];
    memset(dp,0,sizeof(dp));
    for(int i=0;i<n;i++){
        for(int j=0,k=i;k<n;k++,j++){
            if(s[j]==s[k]) dp[j][k]=dp[j+1][k-1];
            else dp[j][k]=min(dp[j+1][k],dp[j][k-1])+1;
            //bên dưới hoặc bên trái
        }
    }
    cout<<dp[0][n-1]<<endl;//1
}
```

```
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
    return 0;
}
```

Sắp xếp - Tìm kiếm

SẮP XẾP ĐỔI CHỖ TRỰC TIẾP

Hãy thực hiện thuật toán sắp xếp đổi chỗ trực tiếp trên dãy N số nguyên. Ghi ra các bước thực hiện thuật toán. Dữ liệu vào: Dòng 1 ghi số N (không quá 100). Dòng 2 ghi N số nguyên dương (không quá 100). Kết quả: Ghi ra màn hình từng bước thực hiện thuật toán. Mỗi bước trên một dòng, các số trong dãy cách nhau đúng một khoảng trống.

Ví dụ:

Input	Output
4	Buoc 1: 2 7 5 3
5 7 3 2	Buoc 2: 2 3 7 5
	Buoc 3: 2 3 5 7

Giải :

Cách 1 : c++

```
#include<bits/stdc++.h>
using namespace std;
int n,a[101];
void out(){
    for(int i=0;i<n;i++) cout<<a[i]<<' ';
}
int main(){
    cin>>n;
    for(int i=0;i<n;i++) cin>>a[i];
    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(a[j]<a[i])swap(a[i],a[j]);
        }
        cout<<"Buoc "<<i+1<<": ";
        out();
        cout<<endl;
    }
}
```

```
    return 0;
}
```

Cách 2 : python 3

```
n=int(input())
a=[]
def out():
    for i in range(n) : print(a[i],end=' ')
a = [int(i) for i in input().split()]# nhập trên 1 dòng
for i in range(0,n-1):
    for j in range(i,n):
        if a[j] < a[i]:
            a[j],a[i] = a[i],a[j]
    print("Bước "+str(i+1)+":",end=' ')
    out()
    print()
```

cách 3 : python 3 khác :

```
n = int(input())
a = [int(i) for i in input().split()]
def out():
    for i in a:
        print(i,end = " ")
    print()
for i in range(n-1):
    for j in range(i+1,n):
        if a[j] < a[i]:
            a[j],a[i] = a[i],a[j]
    print("Bước " +str(i+1) + ":",end = " ")
    out()
```

SẮP XẾP CHỌN

Hãy thực hiện thuật toán sắp xếp chọn trên dãy N số nguyên. Ghi ra các bước thực hiện thuật toán.

Dữ liệu vào: Dòng 1 ghi số N (không quá 100). Dòng 2 ghi N số nguyên dương (không quá 100).

Kết quả: Ghi ra màn hình từng bước thực hiện thuật toán. Mỗi bước trên một dòng, các số trong dãy cách nhau đúng một khoảng trống.

Ví dụ:

Input	Output
4	Buoc 1: 2 7 3 5
5 7 3 2	Buoc 2: 2 3 7 5
	Buoc 3: 2 3 5 7

Giải :

Cách 1 : c++

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int n,a[101];
```

```
void out(){
```

```
    for(int i=0;i<n;i++) cout<<a[i]<<' ';
```

```
}
```

```
int main(){
```

```
    cin>>n;
```

```
    for(int i=0;i<n;i++) cin>>a[i];
```

```
    for(int i=0;i<n-1;i++){
```

```
        int k=i,temp=a[i];
```

```
        for(int j=i+1;j<n;j++){
```

```
            if(a[j]<temp){
```

```
                k=j;temp=a[j];
```

```
            }
```

```
        }
```

```
        swap(a[i],a[k]);
```

```
        cout<<"Buoc "<<i+1<<": ";
```

```
        out();
```

```
        cout<<endl;
```

```
    }
```

```
    return 0;
```

```
}
```

Cách 2: python 3 :

```
n=int(input())
```

```
a = [int(i) for i in input().split()]# nhập trên 1 dòng
```

```
def out():
```

```
    for i in a : print(i,end=' ')
```

```

for i in range(n-1):
    k=i;temp=a[i]
    for j in range(i+1,n):
        if a[j] < temp:
            k=j;temp=a[j]
    a[i],a[k] = a[k],a[i]
    print("Buoc "+str(i+1)+":",end=' ')
    out()
    print()

```

SẮP XẾP CHÈN

Hãy thực hiện thuật toán sắp xếp chèn trên dãy N số nguyên. Ghi ra các bước thực hiện thuật toán.

Dữ liệu vào: Dòng 1 ghi số N (không quá 100). Dòng 2 ghi N số nguyên dương (không quá 100).

Kết quả: Ghi ra màn hình từng bước thực hiện thuật toán. Mỗi bước trên một dòng, các số trong dãy cách nhau đúng một khoảng trống.

Ví dụ:

Input	Output
4	Buoc 0: 5
5 7 3 2	Buoc 1: 5 7
	Buoc 2: 3 5 7
	Buoc 3: 2 3 5 7

Giải :

Cách 1 : c++

```

#include<bits/stdc++.h>
using namespace std;
int main(){
    int n;cin>>n;
    int a[n+1];
    vector <int> res;
    for(int i=0;i<n;i++) cin>>a[i];
    for(int i=0;i<n;i++) {
        cout<<"Buoc "<<i<<": ";
        res.push_back(a[i]);
        sort(res.begin(),res.end());
    }
}

```

```

        for(int i=0;i<res.size();i++) cout<<res[i]<<' ';
        cout<<endl;
    }
    return 0;
}

```

SẮP XẾP NỔI BỌT

Hãy thực hiện thuật toán sắp xếp nổi bọt trên dãy N số nguyên. Ghi ra các bước thực hiện thuật toán.

Dữ liệu vào: Dòng 1 ghi số N (không quá 100). Dòng 2 ghi N số nguyên dương (không quá 100).

Kết quả: Ghi ra màn hình từng bước thực hiện thuật toán. Mỗi bước trên một dòng, các số trong dãy cách nhau đúng một khoảng trống.

Ví dụ:

Input	Output
4	Buoc 1: 3 2 5 7
5 3 2 7	Buoc 2: 2 3 5 7

Giải :

```

#include<bits/stdc++.h>
using namespace std;
void out(int *a,int n){
    for(int i=0;i<n;i++) cout<<a[i]<<' ';
    cout<<endl;
}
int main(){
    int n;cin>>n;
    int a[n+1];
    for(int i=0;i<n;i++) cin>>a[i];
    for(int i=0;i<n-1;i++) {
        int ok=0;
        for(int j=1;j<n;j++){
            if(a[j]<a[j-1]){
                ok=1;swap(a[j],a[j-1]);
            }
        }
        if(ok==1) cout<<"Buoc "<<i+1<<": ",out(a,n);
    }
}

```

```

        else break;
    }
    return 0;
}

```

SẮP XẾP XEN KẼ

Cho mảng $A[]$ gồm n số nguyên khác nhau. Hãy đưa ra các phần tử của mảng theo khuôn dạng lớn nhất, nhỏ nhất, lớn thứ hai, nhỏ thứ 2, ... Ví dụ với $A[] = \{9, 7, 12, 8, 6, 5\}$ ta đưa ra : 12, 5, 9, 6, 8, 7.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên là số phần tử của mảng n ; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
7	
7 1 2 3 4 5 6	7 1 6 2 5 3 4
8	9 1 8 2 7 3 6 4
1 6 9 4 3 7 8 2	

Giải :

```

#include<bits/stdc++.h>
using namespace std;
int n,a[10000];
void initwsolve(){
    cin>>n;
    for(int i=0;i<n;i++)cin>>a[i];
    sort(a,a+n,greater<int>());
    int l=0,r=n-1;
    while(l<r){
        cout<<a[l]<<' '<<a[r]<<' ';
        l++;r--;
    }
    if(l==r) cout<<a[l];
    cout<<endl;
}

```

```

}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

SẮP XẾP THEO GIÁ TRỊ TUYỆT ĐỐI

Ôn lại

Cho mảng $A[]$ gồm n phần tử và số X . Hãy đưa sắp xếp các phần tử của mảng theo trị tuyệt đối của $|X - A[i]|$. Ví dụ với $A[] = \{10, 5, 3, 9, 2\}$ và $X = 7$ ta đưa ra mảng được sắp xếp theo nguyên tắc kể trên: $A[] = \{5, 9, 10, 3, 2\}$ vì $|7-10|=3, |7-5|=2, |7-3|=4, |7-9|=2, |7-2|=5$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên là số phần tử của mảng n và X ; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- T, n, X thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq n, X, A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
5 7	
10 5 3 9 2	5 9 10 3 2
5 6	5 4 3 2 1
1 2 3 4 5	

Giải :

// chú ý là sắp xếp theo nguyên tắc trên phải bảo toàn thứ tự
// vd test kia thì 5 và 9 đều có trị tuyệt đối hiệu với 7 = 2 , nhưng trong mảng a
// số 5 đứng trước 9 => kết quả thì 5 vẫn phải trước 9 => ý tưởng là cho n lần xét
// mỗi 1 lần xét thì lại xét 1 vòng lặp kiểm tra (n-1) cặp (a[j],a[j+1]);

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int n,x,a[100005];
```

```
void initwsolve(){
```

```
    cin>>n>>x;
```

```
    for(int i=0;i<n;i++)cin>>a[i];
```

```

for(int i=0;i<n;i++){
    for(int j=0;j<n-1;j++){
        if(abs(x-a[j+1])<abs(x-a[j]))swap(a[j],a[j+1]);
    }
}
for(int i=0;i<n;i++)cout<<a[i]<<' ';
cout<<endl;

}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

HỢP VÀ GIAO CỦA HAI DÃY SỐ 1

Cho mảng $A[]$ gồm n phần tử, mảng $B[]$ gồm m phần tử khác nhau. Các phần tử của mảng $A[]$ và $B[]$ đã được sắp xếp. Hãy tìm mảng hợp và giao được sắp giữa $A[]$ và $B[]$. Ví dụ với $A[] = \{1, 3, 4, 5, 7\}$, $B[] = \{2, 3, 5, 6\}$ ta có mảng hợp Union $= \{1, 2, 3, 4, 5, 6, 7\}$, mảng giao Intersection $= \{3, 5\}$. In ra đáp án theo giá trị phần tử từ nhỏ đến lớn.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m là số phần tử của mảng $A[]$ và $B[]$; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; dòng tiếp theo là m số $B[i]$ của mảng $B[]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, m, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq n, m, A[i], B[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	
5 3	1 2 3 4 5
1 2 3 4 5	1 2 3
1 2 3	

Giải:

```

#include <bits/stdc++.h>
using namespace std;
void initwsolve(){

```

```

int n,m;
cin>>n>>m;
int a[100005],b[100005],dp1[100005],dp2[100005];
for(int i=0;i<=100000;i++) {dp1[i]= 0;dp2[i]=0; }
for(int i=0;i<n;i++){
    cin>>a[i]; dp1[a[i]]=1;
}
for(int i=0;i<m;i++){
    cin>>b[i]; dp2[b[i]]=1;
}
for(int i=0;i<=100000;i++)
    if(dp1[i]==1||dp2[i]==1) cout<<i<<' ';
cout<<endl;
for(int i=0;i<=100000;i++)
    if(dp1[i]==1&dp2[i]==1) cout<<i<<' ';
cout<<endl;
}
int main() {
    int t;
    cin>>t;
    while(t-->0) initwsolve();
return 0;
}

```

HỢP VÀ GIAO CỦA HAI DÃY SỐ 2

Cho mảng A[] gồm n phần tử, mảng B[] gồm m phần tử khác nhau. Các phần tử của mảng A[] và B[] chưa được sắp xếp. Hãy tìm mảng hợp và giao được sắp giữa A[] và B[]. Ví dụ với A[] = {7, 1, 5, 2, 3, 6}, B[]={3, 8, 6, 20, 7} ta có mảng hợp Union = {1, 2, 3, 5, 6, 7, 8, 20}, mảng giao Intersection = {3, 6}.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m là số phần tử của mảng A[] và B[]; dòng tiếp theo là n số A[i] của mảng A [];dòng tiếp theo là m số B[i] của mảng B[]; các số được viết cách nhau một vài khoảng trống.
- T, n, m, A[i], B[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m, A[i], B[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
--------	---------

1 6 5 7 1 5 2 3 6 3 8 6 20 7	1 2 3 5 6 7 8 20 2 6
---------------------------------------	-------------------------

Giải:

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n,m;
    cin>>n>>m;
    int a[100005],b[100005],dp1[100005],dp2[100005];
    for(int i=0;i<=100000;i++) {dp1[i]= 0;dp2[i]=0; }
    for(int i=0;i<n;i++){
        cin>>a[i]; dp1[a[i]]=1;
    }
    for(int i=0;i<m;i++){
        cin>>b[i]; dp2[b[i]]=1;
    }
    for(int i=0;i<=100000;i++)
        if(dp1[i]==1||dp2[i]==1) cout<<i<<' ';
    cout<<endl;
    for(int i=0;i<=100000;i++)
        if(dp1[i]==1&dp2[i]==1) cout<<i<<' ';
    cout<<endl;
}
int main() {
    int t;
    cin>>t;
    while(t-->0) initwsolve();
return 0;
}
```

SẮP XẾP [0 1 2]

Bài làm tốt nhất

Cho mảng $A[]$ gồm n phần tử. Các phần tử của mảng $A[]$ chỉ bao gồm các số 0, 1, 2. Hãy sắp xếp mảng $A[]$ theo thứ tự tăng dần. Ví dụ với $A[] = \{0, 2, 1, 2, 0\}$ ta kết quả $A[] = \{0, 0, 1, 2, 2\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n là số phần tử của mảng A[]; dòng tiếp theo là n số A[i] của mảng A [] các số được viết cách nhau một vài khoảng trống.
- T, n, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $0 \leq A[i] \leq 2$; $1 \leq n \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
5	0 0 1 2 2
0 2 1 2 0	0 1
3	
0 1 0	

Giải:

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n;/**/cin>>n;
    int a[n];
    for(int i=0;i<n;i++) /**/cin>>a[i];
    sort(a,a+n);
    for(int i=0;i<n;i++) cout<<a[i]<<' ';
    cout<<endl;
}
int main() {
    int t;/**/cin>>t;
    while(t-->0) initwsolve();
return 0;
}
```

SẮP XẾP DÂY CON LIÊN TỤC

Cho mảng A[] gồm n phần tử. Hãy tìm dãy con liên tục của mảng A[R], ..., A[L] sao cho khi sắp xếp lại dãy con ta nhận được một mảng được sắp xếp. Ví dụ với A[] = { 10, 12, 20, 30, 25, 40, 32, 31, 35, 50, 60} ta chỉ cần sắp xếp lại dãy con từ A[4],..., A[9]: {30, 25, 40, 32, 31, 35} để có mảng được sắp.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n là số phần tử của mảng A[]; dòng tiếp theo là n số A[i] của mảng A [] các số được viết cách nhau một vài khoảng trống.
- T, n, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^6$; $0 \leq A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
11	4 9
10 12 20 30 25 40 32 31 35 50 60	3 6
9	
0 1 15 25 6 7 30 40 50	

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n;
    cin>>n;
    int a[n],b[n];
    for(int i=0;i<n;i++){
        cin>>a[i]; b[i]=a[i];
    }
    sort(b,b+n);
    for(int i=0;i<n;i++){
        if(a[i]!=b[i]){
            cout<<i+1<<' ';break;
        }
    }
    for(int i=n-1;i>=0;i--){
        if(a[i]!=b[i]){
            cout<<i+1<<' ';break;
        }
    }
    cout<<endl;
}
int main() {
    int t;cin>>t;
    while(t-->0) initwsolve();
return 0;
}
```

CẶP SỐ TỔNG BẰNG K

Cho mảng $A[]$ gồm n phần tử và số k . Đếm tất cả các cặp phần tử của mảng có tổng bằng k . Ví dụ $A[] = \{1, 5, 3, 4, 2\}$, $k = 7$ ta có kết quả là 2 cặp $(3, 4)$, $(5, 2)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n là số phần tử của mảng $A[]$ và k ; dòng tiếp theo là n số $A[i]$ của mảng $A[]$ các số được viết cách nhau một vài khoảng trống.
- $T, n, k, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 100$; $0 \leq k \leq 100$, $0 \leq A[i] \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2 5 9 1 5 4 1 2 3 2 1 1 1	1 3

Giải:

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n,k;/**/cin>>n>>k;
    int a[n];
    int count=0;
    for(int i=0;i<n;i++)/**/cin>>a[i];
    for(int i=0;i<n-1;i++)
        for(int j=i+1;j<n;j++)
            if (a[j]==k-a[i]) count++;
    cout<<count<<endl;
}
int main() {
    int t;/**/cin>>t;
    while(t-- ) initwsolve();
return 0;
}
```

SẮP XẾP CHỮ SỐ

Cho mảng $A[]$ gồm n phần tử. Nhiệm vụ của bạn là đưa ra mảng đã được sắp xếp bao gồm các chữ số của mỗi phần tử trong $A[]$. Ví dụ $A[] = \{110, 111, 112, 113, 114\}$ ta có kết quả là $\{0, 1, 2, 3, 4\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n là số phần tử của mảng $A[]$; dòng tiếp theo là n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^7$; $0 \leq A[i] \leq 10^{16}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
3	
131 11 48	1 3 4 8
4	1 2 3 4 6
111 222 333 446	

Giải:

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n;cin>>n;
    string s="",temp="";
    for(int i=0;i<n;i++){
        cin>>temp;s+=temp;
    }
    sort(s.begin(),s.end());
    for(int i=0;i<s.length();i++)
        if(s[i]!=s[i+1]) cout<<s[i]<<' ';
    cout<<endl;
}
int main() {
    int t;cin>>t;
    while(t-->0) initwsolve();
    return 0;
}
```

TỔNG GẦN 0 NHẤT

Cho mảng $A[]$ gồm n phần tử, hãy tìm cặp phần tử có tổng gần nhất so với 0.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n là số phần tử của mảng $A[]$; dòng tiếp theo đưa vào n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq N \leq 10^3$, $-10^6 \leq A[i] \leq 10^6$.

Output:

- Đưa ra tổng gần nhất với 0 của cặp phần tử.

Input:	Output:
2	
3	
-8 -66 -60	-68
6	-14
-21 -67 -37 -18 4 -65	

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    int n;cin>>n;
```

```
    int a[n];
```

```
    for(int i=0;i<n;i++)cin>>a[i];
```

```
    int res=1e9+7;
```

```
    for(int i=0;i<n-1;i++){
```

```
        for(int j=i+1;j<n;j++){
```

```
            if(abs(a[j]+a[i])<abs(res))res=a[j]+a[i];
```

```
        }
```

```
    }
```

```
    cout<<res<<endl;
```

```
}
```

```
int main() {
```

```
    int t;cin>>t;
```

```
    while(t-->0) initwsolve();
```

```
    return 0;
```

```
}
```

PHẦN TỬ LỚN NHẤT

Cho mảng $A[]$ gồm n phần tử, hãy tìm k phần tử lớn nhất của mảng. Các phần tử được đưa ra theo thứ tự giảm dần.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào N và K ; dòng tiếp theo đưa vào n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, K, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq K < N \leq 10^3$, $1 \leq A[i] \leq 10^6$.

Output:

- Đưa ra k phần tử lớn nhất trên một dòng.

Input:	Output:
2	
5 3	
10 7 9 12 6	12 10 9
6 2	12 9
9 7 12 8 6 5	

```
#include<bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n,k;/**/cin>>n>>k;
    int a[n];
    for(int i=0;i<n;i++)/**/ cin>>a[i];
    sort(a,a+n,greater<int>());
    for(int i=0;i<k;i++)
        cout<<a[i]<<' ';
    cout<<endl;
}
int main(){
    int t; /**/cin>>t;
    while(t-->0) initwsolve();
    return 0;
}
```

SỐ LẦN XUẤT HIỆN

Cho mảng $A[]$ gồm n phần tử đã được sắp xếp. Hãy tìm số lần xuất hiện số X trong mảng. Nếu số lần xuất hiện số x trong mảng là 0 hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào N và X; dòng tiếp theo đưa vào n số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, N, X, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$, $1 \leq A[i]$, $X \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
7 2	4
1 1 2 2 2 2 3	-1
7 4	
1 1 2 2 2 2 3	

Giải :

```
#include<bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n,k; cin>>n>>k;
    int res=0,a[n];
    for(int i=0;i<n;i++)cin>>a[i];
    for(int i=0;i<n;i++){
        if(a[i]==k)res++;
    }
    if(res>0)cout<<res<<endl;
    else cout<<-1<<endl;
}
int main(){
    int t; cin>>t;
    while(t-->0)initwsolve();
    return 0;
}
```

TỔNG CẶP SỐ NGUYÊN TỐ

Cho số tự nhiên N. Hãy tìm cặp số nguyên tố đầu tiên có tổng là N. Nếu không tồn tại cặp số nguyên tố có tổng bằng N, hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm là một số N được ghi trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	2 2
4	2 5
8	

Giải :

```
#include<bits/stdc++.h>
using namespace std;
int const N=1e6+5;
int prime[N];
void initwsolve(){
    int n;cin>>n;
    for(int i=2;i<n;i++){
        if(prime[i]==0&&prime[n-i]==0){
            cout<<i<<' '<<n-i<<endl;return;
        }
    }
    cout<<-1<<endl;
}
int main(){
    int t; cin>>t;
    prime[0]=1;prime[1]=1;
    for(int i=2;i<=sqrt(N);i++){
        if(prime[i]==0)
            for(int j=i*i;j<N;j+=i)
                prime[j]=1;
    }
    while(t--){initwsolve();
    return 0;
}
```

MERGE SORT

Cho mảng $A[]$ gồm N phần tử chưa được sắp xếp. Nhiệm vụ của bạn là sắp xếp các phần tử của mảng $A[]$ theo thứ tự tăng dần bằng thuật toán Merge Sort.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N tương ứng với số phần tử của mảng $A[]$; phần thứ 2 là N số của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $A[i] \leq 10^6$.

Output:

- Đưa ra kết quả các test theo từng dòng.

Input	Output
2	
5	1 3 4 7 9
4 1 3 9 7	1 2 3 4 5 6 7 8 9 10
10	
10 9 8 7 6 5 4 3 2 1	

Giai:

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n;cin>>n;
    int a[n];
    for(int i=0;i<n;i++)cin>>a[i];
    sort(a,a+n);
    for(int i=0;i<n;i++)cout<<a[i]<<' ';
    cout<<endl;
}
int main() {
    int t;cin>>t;
    while(t-->0) initwsolve();
return 0;
}
```

TÍCH LỚN NHẤT - NHỎ NHẤT

Cho mảng $A[]$ gồm n phần tử và mảng $B[]$ gồm m phần tử. Nhiệm vụ của bạn là tìm tích giữa phần tử lớn nhất của mảng $A[]$ và phần tử nhỏ nhất của mảng $B[]$. Ví

dụ $A[] = \{5, 7, 112, 9, 3, 6, 2\}$, $B[] = \{1, 2, 6, -1, 0, 9\}$ ta có kết quả là $-9 = 9 * (-1)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m tương ứng với số phần tử của mảng $A[]$ và $B[]$; dòng tiếp theo là n số $A[i]$; dòng cuối cùng là m số $B[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, m, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m \leq 10^6$; $-10^8 \leq A[i] \leq 10^8$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
6 6	
5 7 9 3 6 2	-9
1 2 6 -1 0 9	20
6 6	
1 4 2 3 10 2	
4 2 6 5 2 9	

Giải:

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n,m;cin>>n>>m;
    int a[n],b[m];
    for(int i=0;i<n;i++)cin>>a[i];
    for(int i=0;i<m;i++)cin>>b[i];
    sort(a,a+n);
    sort(b,b+m);
    cout<<(long long) a[n-1]*b[0]<<endl;
}
int main() {
    int t;cin>>t;
    while(t-->0) initwsolve();
return 0;
}
```

TRỘN HAI DÃY

Cho mảng $A[]$ gồm n phần tử và mảng $B[]$ gồm m phần tử. Nhiệm vụ của bạn là hợp nhất hai mảng $A[]$ và $B[]$ để được một mảng mới đã được sắp xếp. Ví dụ $A[] = \{5, 7, 112, 9, 3, 6, 2\}$, $B[] = \{1, 2, 6, -1, 0, 9\}$ ta có kết quả là $C[] = \{-1, 1, 0, 2, 3, 5, 6, 6, 7, \dots\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m tương ứng với số phần tử của mảng $A[]$ và $B[]$; dòng tiếp theo là n số $A[i]$; dòng cuối cùng là m số $B[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, m, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m \leq 10^6$; $-10^8 \leq A[i] \leq 10^8$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1 3 3 10 5 15 20 3 2	2 3 5 10 15 20

Giai:

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n,m;cin>>n>>m;
    int a[n+m];
    for(int i=0;i<n+m;i++)cin>>a[i];
    sort(a,a+n+m);
    for(int i=0;i<n+m;i++)
        cout<<a[i]<<' ';
    cout<<endl;
}
int main() {
    int t;cin>>t;
    while(t-->0) initwsolve();
return 0;
}
```

BỔ SUNG PHẦN TỬ

Cho mảng $A[]$ gồm n số nguyên dương. Gọi L, R là max và min các phần tử của $A[]$. Nhiệm vụ của bạn là tìm số phần tử cần thiết cần thêm vào mảng để mảng có đầy đủ các số trong khoảng $[L, R]$. Ví dụ $A[] = \{5, 7, 9, 3, 6, 2\}$ ta nhận được kết quả là 2 tương ứng với các số còn thiếu là 4, 8.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n , tương ứng với số phần tử của mảng $A[]$; dòng tiếp theo là n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, A[i] \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2 5 4 5 3 8 6 3 2 1 3	1 0

Giai:

#note : count để đếm xem từ $l-r$ có bao nhiêu số khác nhau

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    int n;cin>>n;
```

```
    int a[n],count=1;
```

```
    for(int i=0;i<n;i++)cin>>a[i];
```

```
    sort(a,a+n);
```

```
    for(int i=1;i<n;i++)
```

```
        if(a[i]!=a[i-1])count++;
```

```
    cout<<a[n-1]-a[0]-count+1 <<endl;
```

```
}
```

```
int main() {
```

```
    int t;cin>>t;
```

```
    while(t--) initwsolve();
```

```
return 0;
```

```
}
```

SẮP XẾP THEO SỐ LẦN XUẤT HIỆN

Cho mảng $A[]$ gồm n số nguyên. Nhiệm vụ của bạn là sắp xếp mảng theo số lần xuất hiện các phần tử của mảng. Số xuất hiện nhiều lần nhất đứng trước. Nếu hai phần tử có số lần xuất hiện như nhau, số nhỏ hơn đứng trước. Ví dụ $A[] = \{5, 5, 4, 6, 4\}$, ta nhận được kết quả là $A[] = \{4, 4, 5, 5, 6\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n , tương ứng với số phần tử của mảng $A[]$ và số k ; dòng tiếp theo là n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^4$; $1 \leq k \leq 10^3$; $1 \leq A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
5	
5 5 4 6 4	4 4 5 5 6
5	9 9 9 2 5
9 9 9 2 5	

Giai:

```
#include <bits/stdc++.h>
using namespace std;
int f[100005];
bool cmp(int a,int b){
    if(f[a]==f[b])return a<b;
    else return f[a]>f[b];
}
void initwsolve(){
    int n;cin>>n;
    int a[n];
    memset(f,0,sizeof(f));
    for(int i=0;i<n;i++){
        cin>>a[i];f[a[i]]++;
    }
    sort(a,a+n,cmp);
    for(int i=0;i<n;i++)cout<<a[i]<<' ';
    cout<<endl;
```

```

}
int main() {
    int t;cin>>t;
    while(t--) initwsolve();
return 0;
}

```

TÌM KIẾM

Cho mảng $A[]$ gồm n phần tử đã được sắp xếp. Hãy đưa ra 1 nếu X có mặt trong mảng $A[]$, ngược lại đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n , X là số các phần tử của mảng $A[]$ và số X cần tìm; dòng tiếp theo đưa vào n số $A[i]$ ($1 \leq i \leq n$) các số được viết cách nhau một vài khoảng trống.
- T, n, A, X thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, X, A[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
5 16	
2 4 7 9 16	1
7 98	-1
1 22 37 47 54 88 96	

Giai:

```

#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n,x;cin>>n>>x;
    int a[n];
    for(int i=0;i<n;i++)cin>>a[i];
    sort(a,a+n);
    for(int i=0;i<n;i++)
        if(a[i]==x){cout<<1<<endl;return;}
    cout<<-1<<endl;
}
int main() {

```

```

    int t;cin>>t;
    while(t-->0) initwsolve();
return 0;
}

```

TÌM KIẾM TRONG DÃY SẮP XẾP VÒNG

Một mảng được sắp được chia thành hai đoạn tăng dần được gọi là mảng sắp xếp vòng. Ví dụ mảng $A[] = \{ 5, 6, 7, 8, 9, 10, 1, 2, 3, 4 \}$ là mảng sắp xếp vòng. Cho mảng $A[]$ gồm n phần tử, hãy tìm vị trí của phần tử x trong mảng $A[]$ với thời gian $\log(n)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n và x ; dòng tiếp theo đưa vào n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i], x$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq N, x, A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
10 3	
5 6 7 8 9 10 1 2 3 4	9
10 3	3
1 2 3 4 5 6 7 8 9 10	

Giai:

```

#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n,x;cin>>n>>x;
    int a[n];
    for(int i=0;i<n;i++)cin>>a[i];
    for(int i=0;i<n;i++)
        if(a[i]==x){cout<<i+1<<endl;return;}
    cout<<-1<<endl;
}
int main() {
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

```
return 0;
}
```

SỐ NHỎ NHẤT VÀ NHỎ THỨ HAI

Cho mảng $A[]$ gồm n phần tử, hãy đưa ra số nhỏ nhất và số nhỏ thứ hai của mảng. Nếu không có số nhỏ thứ hai, hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n là số phần tử của mảng $A[]$; dòng tiếp theo đưa vào n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
10	
5 6 7 8 9 10 1 2 3 4	1 2
5	-1
1 1 1 1 1	

Giai:

```
#include <bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n;cin>>n;
    int a[n],res=-1;
    for(int i=0;i<n;i++)cin>>a[i];
    sort(a,a+n);
    for(int i=1;i<n;i++)
        if(a[i]!=a[i-1]){res=a[i];break;}
    if(res==-1)cout<<-1<<endl;
    else cout<<a[0]<<' '<<res<<endl;
}
int main() {
    int t;cin>>t;
    while(t-->0) initwsolve();
return 0;
}
```

XÓA DỮ LIỆU TRONG DSLK ĐƠN

Cho danh sách liên kết đơn lưu giữ các số nguyên được quản lý bởi con trỏ First. Viết chương trình con xóa tất cả các phần tử có giá trị bằng x trong danh sách liên kết đơn; chương trình con trả về số lượng các phần tử đã xóa. Sau khi xóa xong, liệt kê các phần tử còn lại trong danh sách liên kết đơn First. Ví dụ: Ta có Input sau:

14 : là số lượng phần tử trong danh sách

1 1 1 4 5 1 1 1 7 1 8 1 9 : là 14 phần tử

1 : là số cần xóa

Output : 4 5 7 8 9

Giải:

```
#include<bits/stdc++.h>
using namespace std;
vector<string> res;
main(){
    int n;cin>>n;
    for(int i=1;i<=n;i++){
        string s;cin>>s;
        res.push_back(s);
    }
    string k;cin>>k;
    for(int i=0;i<res.size();i++)
        if(res[i]!=k) cout<<res[i]<<" ";
}
```

Cách 2:

```
#include<bits/stdc++.h>
using namespace std;
vector<int> res;
main(){
    int n;cin>>n;
    for(int i=1;i<=n;i++){
        int s;cin>>s;
        res.push_back(s);
    }
    int k; cin>>k;
    for(int i=0;i<res.size();i++)
        if(res[i]!=k) cout<<res[i]<<" ";
}
```

}

LỌC DỮ LIỆU TRÙNG TRONG DSLK ĐƠN

Cho danh sách liên kết đơn lưu giữ các số nguyên được quản lý bởi con trỏ First. Viết chương trình con lọc tất cả các phần tử có giá trị trùng nhau trong danh sách liên kết đơn First, chỉ để lại 1 phần tử đại diện cho nhóm trùng. Sau khi lọc xong, liệt kê các phần tử trong danh sách liên kết đơn First.

Ví dụ: Ta có Input :

12 : là số lượng phần tử trong danh sách

1 1 1 4 5 1 4 7 7 8 1 9 : là số phần tử

Output : 1 4 5 7 8 9

Giải:

```
#include<bits/stdc++.h>
using namespace std;
vector<int> res;
main(){
    int n;cin>>n;
    int f[100005];
    memset(f,0,sizeof(f));
    for(int i=1;i<=n;i++){
        int n;cin>>n;
        res.push_back(n);
        f[n]++;
    }
    for(int i=0;i<res.size();i++)
        if(f[res[i]]>0) {
            cout<<res[i]<<" ";
            f[res[i]]=0;
        }
}
```

GIAO CỦA BA DÃY SỐ

Cho ba dãy số A[], B[], C[] gồm N1, N2, N3 phần tử đã được sắp xếp. Hãy đưa ra các phần tử có mặt trong cả ba dãy theo thứ tự tăng dần. Nếu không có đáp án, in ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm bốn dòng: dòng thứ nhất đưa vào $N1, N2, N3$ là số phần tử của mảng $A[], B[], C[]$; các dòng tiếp theo đưa vào 3 dãy $A[], B[], C[]$.
- Ràng buộc: $1 \leq T \leq 100$; $1 \leq N1, N2, N3 \leq 10^6$, $0 \leq A[i], B[j], C[k] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input:	Output:
1	20 80
6 5 8	
1 5 10 20 40 80	
6 7 20 80 100	
3 4 15 20 30 70 80 120	

Giải :

```
#include<bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n1,n2,n3,i=0,j=0,k=0,check=0;
    cin>>n1>>n2>>n3;
    long long a[n1],b[n2],c[n3];
    for(int i=0;i<n1;i++) cin>>a[i];
    for(int i=0;i<n2;i++) cin>>b[i];
    for(int i=0;i<n3;i++) cin>>c[i];
    while(i<n1 &&j<n2 &&k<n3){
        if(a[i]==b[j]&&b[j]==c[k]){
            cout<<a[i]<<' '; check=1;
            i++;j++;k++;
        }
        else if(a[i]<b[j]) i++;
        else if(b[j]<c[k]) j++;
        else k++;
    }
    if(check==0) cout<<-1;
    cout<<endl;
}
int main (){
```

```

    int t;/**/cin>>t;
    while(t-->0) initwsolve ();
return 0;
}

```

SẮP XẾP CHẴN LẺ

Cho dãy số $a[]$ có n phần tử, đánh số từ 1 đến n . Hãy sắp xếp các phần tử ở vị trí lẻ theo thứ tự tăng dần, các phần tử ở vị trí chẵn theo thứ tự giảm dần.

Input

Dòng đầu tiên ghi số n , không quá 10^5

Dòng thứ 2 ghi n số của dãy $a[]$ ($a \leq a[i] \leq 10^9$)

Output

Ghi ra dãy số kết quả trên một dòng

Ví dụ

Input	Output
4 1 2 3 4	1 4 3 2

Giai:

```

#include<bits/stdc++.h>
using namespace std;
int a[100005];
vector<int>in;
vector<int>de;
int main(){
    int n;cin>>n;
    for(int i=0;i<n;i++){
        cin>>a[i];
        if(i%2==0)in.push_back(a[i]);
        else de.push_back(a[i]);
    }
    sort(in.begin(),in.end());
    sort(de.begin(),de.end(),greater<int>() );
    if(in.size()<de.size()){
        for(int i=0;i<in.size();i++)
            cout<<in[i]<<' '<<de[i]<<' ';
        cout<<de[de.size()-1];
    }
}

```

```

else if(in.size()>de.size()){
    for(int i=0;i<de.size();i++)
        cout<<in[i]<<' '<<de[i]<<' ';
    cout<<in[in.size()-1];
}
else {
    for(int i=0;i<de.size();i++)
        cout<<in[i]<<' '<<de[i]<<' ';
}

return 0;
}

```

ĐỔI CHỖ ÍT NHẤT

Ôn lại

Cho mảng $A[]$ gồm n phần tử. Hãy tìm số phép đổi chỗ ít nhất giữa các phần tử của mảng để mảng $A[]$ được sắp xếp. Ví dụ với $A[] = \{4, 3, 2, 1\}$ ta cần thực hiện ít nhất 2 phép đổi chỗ: $\text{Swap}(A[0], A[3])$, $\text{Swap}(A[1], A[2])$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên là số phần tử của mảng n và X ; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	
4	2
4 3 2 1	2
5	
1 5 4 3 2	

Giải:

```

#include<bits/stdc++.h>
#define MAX 10005
using namespace std;
int n;
int a[MAX],b[MAX],check[MAX];
void xuli(){
    cin>>n;
    for(int i=0;i<n;i++){

```

```

        cin>>a[i]; b[i]=a[i];
    }
    sort(b,b+n);
    int res=0;
    for(int i=0;i<n;i++)
        if(a[i]!=b[i])
            for(int j=i+1;j<n;j++)
                if(a[j]==b[i]){
                    res++;
                    swap(a[i],a[j]);
                    break;
                }
    cout<<res<<endl;;
}
int main(){
    int t;cin>>t;
    while(t-->0) xuli();
}

```

Ngăn xếp

note

luôn đưa s.size() vào trong if(...) hoặc while(...)=> vì nếu sau if có s.pop sẽ có thể ko chạy được nếu s rỗng

2 hàm in stack hay sử dụng đến (chú ý :

1:stack thật được xếp từ phải sang trái. Đây in ngược cho đúng theo mảng, chuỗi nhập vào

2: đưa stack vào mới không ảnh hưởng toàn cục khi s.pop(). Chỉ có tác dụng in khi gọi hàm)

Mảng

//đưa stack vào mới không ảnh hưởng toàn cục khi s.pop()

```

void out(stack<int> s){
    vector <int> a;
    while(!s.empty()){
        a.push_back(s.top());
        s.pop();
    }
    for(int i=a.size()-1;i>=0;i--)cout<<a[i]<<' ';
}

```

```

    cout<<endl;
}
Chuỗi:
void out(stack<char> s){
    vector <char> a;
    while(!s.empty()){
        a.push_back(s.top());
        s.pop();
    }
    for(int i=a.size()-1;i>=0;i--)cout<<a[i]<<' ';
    cout<<endl;
}

```

NGĂN XẾP 1

Bài làm tốt nhất

Cho một ngăn xếp các số nguyên. Các thao tác gồm 3 lệnh: push, pop và show. Trong đó thao tác push kèm theo một giá trị cần thêm (không quá 1000). Hãy viết chương trình ghi ra kết quả của các lệnh show.

Input: Gồm nhiều dòng, mỗi dòng chứa một lệnh push, pop hoặc show. Input đảm bảo số lượng phần tử trong stack khi nhiều nhất cũng không vượt quá 200.

Output: Ghi ra màn hình các phần tử đang có trong stack theo thứ tự lưu trữ mỗi khi gặp lệnh show. Các số viết cách nhau đúng một khoảng trống. Nếu trong stack không còn gì thì in ra dòng “empty”

Ví dụ:

Input	Output
push 3	
push 5	
show	3 5
push 7	3 5 7
show	3
pop	
pop	
show	

Giải:

Cách 1:

//tạo 1 stack để thực hiện 1 trong 3 thao tác push, pop và show.

//2: vì stack s đã khai báo toàn cục=> nếu không muốn đưa tham số vào hàm show để kết quả s không đổi thì phải tạo 1 stack khác (s1=s)

//3 : chú ý khi show thì lại in kết quả ngược lại stack => tạo 1 vector để lưu kết quả sau mỗi lần s.pop();

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
stack <int> s;
```

```
string str;
```

```
int n;
```

```
void show(){
```

```
    stack <int> s1=s;//2
```

```
    vector <int> a;//3
```

```
    while(!s1.empty()){
```

```
        a.push_back(s1.top());
```

```
        s1.pop();
```

```
    }
```

```
    for(int i=a.size()-1;i>=0;i--)cout<<a[i]<<' ';//3
```

```
    cout<<endl;
```

```
}
```

```
int main(){
```

```
    while (1){
```

```
        cin>>str;
```

```
        /*1*/if(str=="push"){ cin>>n;s.push(n); }
```

```
        else if(str=="pop")s.pop();
```

```
        else if(str=="show") show();
```

```
        if(s.size()==0){ cout<<"empty";break; }
```

```
    }
```

```
    return 0;
```

```
}
```

Cách 2 : không cần tạo stack khác => đưa s vào hàm:

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
stack <int> s;
```

```
string str;
```

```
int n;
```

```
void show(stack<int> s){//đưa stack vào mới không ảnh hưởng toàn cục khi s.pop()
```

```
    vector <int> a;
```

```

while(!s.empty()){
    a.push_back(s.top());
    s.pop();
}
for(int i=a.size()-1;i>=0;i--)cout<<a[i]<<' ';
cout<<endl;
}
int main(){
    while (1){
        cin>>str;
        if(str=="push"){cin>>n;s.push(n);}
        else if(str=="pop")s.pop();
        else if(str=="show") show(s);
        if(s.size()==0){cout<<"empty";break;}
    }
    return 0;
}

```

NGĂN XẾP 2

Yêu cầu bạn xây dựng một stack với các truy vấn sau đây:

“PUSH x”: Thêm phần tử x vào stack ($0 \leq x \leq 1000$).

“PRINT”: In ra phần tử đầu tiên của stack. Nếu stack rỗng, in ra “NONE”.

“POP”: Xóa phần tử đầu tiên của stack. Nếu stack rỗng, không làm gì cả.

Input:

Dòng đầu tiên là số lượng truy vấn Q ($Q \leq 100000$).

Mỗi truy vấn có dạng như trên.

Output:

Với mỗi truy vấn “PRINT”, hãy in ra phần tử đầu tiên của stack. Nếu stack rỗng, in ra “NONE”.

Ví dụ:

Input:	Output
9	
PUSH 1	
PUSH 2	1
POP	3
PRINT	NONE
PUSH 3	
PRINT	

POP	
POP	
PRINT	

Giải:

//1: tạo 1 stack để thực hiện 1 trong 3 thao tác push, pop và print.

//3 : chú ý khi show chỉ xét stack rỗng hay không và in phần tử đầu => không phải tạo stack mới.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
stack <int> s;//1
```

```
void show(){
```

```
    if(s.size()==0)cout<<"NONE"<<endl;
```

```
    else cout<<s.top()<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    while (t--){
```

```
        string str;cin>>str;
```

```
        /*1*/if(str=="PUSH"){int n;cin>>n;s.push(n);}
```

```
        else if(str=="POP"){if(s.size() !=0)s.pop();}
```

```
        else show();
```

```
    }
```

```
    return 0;
```

```
}
```

ĐẢO TỪ

Cho chuỗi ký tự S. Nhiệm vụ của bạn là đảo ngược các từ trong S. Ví dụ S = "I like this program very much", ta nhận được kết quả là "much very program this like I".

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test là một chuỗi ký tự S.
- T, S thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{Length}(S) \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input:

2

I like this program very much
much very program this like I

Output:

much very program this like I

I like this program very much

Giải :

```
#include <bits/stdc++.h>
using namespace std;
void show(stack<string> s){
    while(s.size()){
        cout<<s.top()<<' ';
        s.pop();
    } cout<<endl;
}
int main(){
    string str;
    int t;cin>>t;
    //cin.ignore();
    while(t--){
        stack <string> s;
        while(1){
            cin>>str;
            s.push(str);
            if(getchar()=='\n')break;
        }show(s);
    }
}
```

DÃY NGOẶC ĐÚNG DÀI NHẤT

Ôn lại

Cho một xâu chỉ gồm các kí tự '(' và ')'. Một dãy ngoặc đúng được định nghĩa như sau:

- Xâu rỗng là 1 dãy ngoặc đúng.
- Nếu A là 1 dãy ngoặc đúng thì (A) là 1 dãy ngoặc đúng.
- Nếu A và B là 2 dãy ngoặc đúng thì AB là 1 dãy ngoặc đúng.

Cho một xâu S. Nhiệm vụ của bạn là hãy tìm dãy ngoặc đúng dài nhất xuất hiện trong xâu đã cho.

Input: Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).

Mỗi test gồm một xâu S có độ dài không vượt quá 10^5 kí tự.

Output: Với mỗi test in ra một số nguyên là độ dài dãy ngoặc đúng dài nhất tìm được.

Ví dụ:

Input:	Output
3	2
((()	4
)OO)	6
O(O))))	

Giải thích test 2:)O()) (0 1 2 3 4 5)

Chuỗi:) () ())
 Chỉ số: 0 1 2 3 4 5

		1		3			^
Ngăn xếp	0	0	0	0	0	5	

K quả Res=0 Res=2 Res=4 ||

//1:nếu là '(' thì thêm vào hàng đợi để lưu trữ số các dấu '(' và vị trí của nó:

//nếu là ')' thì ta phải xem 2 th :

//2:th1 là tồn tại '(' ở phía trước => ta s.pop() và lấy i-s.top() sẽ ra kết quả(vì s.pop() chính là dấu '('

//3:th còn lại : stack rỗng (<=> không có dấu '(' chưa được sử dụng ở phía trước nữa)=> ta đưa ')' vào stack

#include<bits/stdc++.h>

using namespace std;

int main(){

int t;cin>>t;

while (t--){

string str;cin>>str;

stack <int> s;

s.push(-1);

int res=0;

for(int i=0;i<str.length();i++){

if(str[i]=='(') s.push(i);//1

else{

```

        s.pop();
        if(s.size()!=0)res=max(res,i-s.top());//2
        else s.push(i);//3
    }
    }cout<<res<<endl;
}
return 0;
}

```

KIỂM TRA BIỂU THỨC SỐ HỌC

Ôn lại

Cho biểu thức số học, hãy cho biết biểu thức số học có dư thừa các cặp ký hiệu ‘(,’ ‘)’ ‘,’ hay không?

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 20$.

Ví dụ:

Input	Output
3	Yes
((a+b))	Yes
(a + (b)/c)	No
(a + b*(c-d))	

Giải thích :

(a + (b)/c)

Chuỗi: (a + (b) / c)
 Chỉ số 0 1 2 3 4 5 6 7 8
 :

			b	
		(	(	
+	+	+	+	

Λ
||
||


```

        while(s.size()!=0){//3
            char c=s.top(); s.pop();
            if(c=='(')break;
            if (c=='+'||c=='-'||c=='*'||c=='/') check=1;
        }
        if(check==0){cout<<"Yes\n"; return;}
    }
    }
    cout<<"No\n";
}
int main(){
    int t;cin>>t;
    cin.ignore();
    while (t--)initwsolve();
    return 0;
}

```

ĐẾM SỐ DẤU NGOẶC ĐÔI CHIỀU

Ôn lại

Cho một xâu chỉ gồm các kí tự '(', ')' và có độ dài chẵn. Hãy đếm số lượng dấu ngoặc cần phải đôi chiều ít nhất, sao cho xâu mới thu được là một dãy ngoặc đúng.

Input:

Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).

Mỗi test gồm 1 xâu S có độ dài không vượt quá 100 000, chỉ gồm dấu (và).

Output:

Với mỗi test, in ra đáp án tìm được trên một dòng.

Ví dụ:

Input:	Output
4	2
))((	2
((((	1
((())	3
)(())((	

Giải:

// 1: ý tưởng : cứ tại một vị trí gặp ')' ta sẽ xóa đi 1 cặp tương ứng ().=> nếu

str[i]!=')' sẽ đưa vào chuỗi và => về lý thuyết cuối cùng lấy s.size()/2 là xong

//2: nhưng trường hợp xâu đầu là)))...=> không s.pop() được sẽ âm => chỉ s.pop() khi s.size()!=0

//3:=> khi s.size()==0 chuyển nó thành '(' . khi đó đã chuyển ký tự thì count++; => kết quả cuối cùng :count+s.size()/2

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    string str;
```

```
    cin>>str;
```

```
    stack <char> s;
```

```
    int count=0;
```

```
    for(int i=0;i<str.length();i++){
```

```
        if(str[i]!='') s.push(str[i]);//1
```

```
        else {
```

```
            if(s.size()==0) {s.push('('); count++;} //3
```

```
            else s.pop();//2
```

```
        }
```

```
    }
```

```
    cout<<count+s.size()/2<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    cin.ignore();
```

```
    while (t--)>initwsolve();
```

```
    return 0;
```

```
}
```

Ứng Dụng Stack - Biểu Thức Tiền Tố (Prefix)

Trong toán học thì các biểu thức thường được [biểu diễn dưới dạng trung tố](#) (các toán tử nằm giữa các toán hạng). Đối với máy tính thì khó sử dụng để tính toán, phải đưa biểu thức dạng trung tố về dạng tiền tố hoặc [hậu tố](#).

Infix (trung tố)	Prefix(tiền tố)	Postfix(hậu tố)
x+y	+xy	xy+
x+y-z	-+xyz	xy+z -
x+y*z	+x*yz	xyz*+
x+(y-z)	+x-yz	xyz-+

Kinh nghiệm : tiền tố sang trung tố và hậu tố : duyệt ngược , đẩy chữ và cú ghép toán tử thì nhóm.

BIẾN ĐỔI TIỀN TỐ - TRUNG TỐ

Ôn lại

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng tiền tố về dạng trung tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 10^6$.

Ví dụ:

Input	Output
2	
+AB-CD	((A+B)(C-D))
-A/BC-/AKL	((A-(B/C))((A/K)-L)

Giải thích :test 2:

*	+	A	B	-	C	D
0	1	2	3	4	5	6

Note : mỗi một dấu cách là một chuỗi trong stack . s.top ở cuối mảng

6	D
5	D C
4	(C-D)
3	(C-D) B
2	(C-D) B A
1	(C-D) (A+B)
0	((A+B)*(C-D))

Kết

quả: ((A+B)*(C-D))

Giải:

//note: string(1,str[i]) : chuyển ký tự str[i] thành 1 chuỗi có 1 ký tự

//1:đẩy in stack đúng biểu thức => duyệt ngược từ cuối về

//2: nếu str[i] không phải toán tử ta s.push() còn nếu phải toán tử thì ta lấy 2 cái string đầu trong stack và cùng với toán tử để tạo thành biểu thức.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
bool oprt(char x){//operator
```

```
    if(x=='+'||x=='-'||x=='*'||x=='/')
```

```
        return 1; return 0;
```

```
}
```

```
void initwsolve(){
```

```
    string str;cin>>str;
```

```
    stack <string> s;
```

```
    for(int i=str.length()-1;i>=0;i--){//1
```

```
        if(oprt(str[i])==0) s.push(string(1,str[i]));//2
```

```
        else{
```

```
            string str1=s.top();s.pop();
```

```
            string str2=s.top();s.pop();
```

```
            string temp="("+str1+ str[i] +str2+"");
```

```
            s.push(temp);
```

```
        }
```

```
    }
```

```
    cout<<s.top()<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    cin.ignore();
```

```
    while (t--){initwsolve();
```

```
        return 0;
```

```
}
```

BIẾN ĐỔI TIỀN TỔ - HẬU TỔ

Ôn lại

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng tiền tố về dạng hậu tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 10^6$.

Ví dụ:

Input	Output
2 *+AB-CD *-A/BC-/AKL	AB+CD-* ABC/-AK/L-*

Giải thích:

Test 2:

*	+	A	B	-	C	D
0	1	2	3	4	5	6

*+AB-CD

6 D
5 D C
4 CD-
3 CD- B
2 CD- B A
1 CD- AB+
0 AB+CD-*

kết

quả AB+CD-*

Giải:

//note: string(1,str[i]) : chuyển ký tự str[i] thành 1 chuỗi có 1 ký tự

//1:để in stack đúng biểu thức => duyệt ngược từ cuối về

//2:nếu str[i] không phải toán tử ta s.push() còn nếu phải toán tử thì ta lấy 2 cái string đầu trong stack và cùng với toán tử để tạo thành biểu thức.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
bool oprt(char x){//operator
```

```
    if(x=='+'||x=='-'||x=='*'||x=='/')
```

```
        return 1; return 0;
```

```
}
```

```
void initwsolve(){
```

```
    string str;cin>>str;
```

```
    stack <string> s;
```

```
    for(int i=str.length()-1;i>=0;i--){//1
```

```

        if(oprt(str[i])==0) s.push(string(1,str[i]));//2
        else{
            string str1=s.top();s.pop();
            string str2=s.top();s.pop();
            string temp=str1+str2+ str[i];
            s.push(temp);
        }
    }
    cout<<s.top()<<endl;
}
int main(){
    int t;cin>>t;
    cin.ignore();
    while (t--)>0)initwsolve();
    return 0;
}

```

BIẾN ĐỔI TRUNG TỔ - HẬU TỔ

Ôn lại

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng trung tố về dạng hậu tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 10$.

Ví dụ:

Input	Output
2 (A+(B+C) ((A*B)+C)	ABC++ AB*C+

Giải thích test 1:

chỉ số	chuỗi	stack	res
0	(	(	

1	A	(	A
2	+	(+	A
3	(	(+ (	A
4	B	(+ (	AB
		(+ (	
5	+	+	AB
		(+ (	
6	C	+	ABC
7	)	(+	ABC+

kết
quả ABC++

Giải :

// 1: dùng 1 stack để lưu giữ các toán tử , và kết hợp hàm oprt để lưu trữ thứ tự xét toán tử và dùng 1 chuỗi res để lưu các chữ số A,B,C

//2:=> khi gặp chữ thì đưa vào res ,khi gặp '(' thì đưa vào stack ,

//3:khi gặp toán tử thì xét thứ tự ưu tiên và cuối cùng đưa toán tử vào một vị trí thích hợp

//4:khi gặp ')' thì cập nhật toán tử từ stack sang res . Và cuối cùng xóa 1 dấu '(' trong stack.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int oprt(char x){//operator//note int
    switch(x){
        case '^':return 0;
        case '%': case '*' :case '/' :return 1;
        case '+': case '-' :return 2;
        default:return 3;//dấu '('
    }
}
```

```
void initwsolve(){
    string str;cin>>str;
    string res="";
    stack <char> s;
    for(int i=0;i<str.size();i++){
        if('A'<=str[i]&&str[i]<='Z'||'a'<=str[i]&&str[i]<='z')res+=str[i];//1
        else {
            switch(str[i]){
```

```

/*2*/case '(':s.push(str[i]);break;
/*4*/case ')':
    while(s.top()!='(' && s.size()){
        res+=s.top();s.pop();
    }s.pop();break;
/*3*/case '^':case '%': case '*' :case '/' :case '+': case '-' :
    while(s.size() && oprt(s.top())<=oprt(str[i]) ){
        res+=s.top();s.pop();
    }s.push(str[i]);break;
    }
}
while(s.size()){
    if(s.top()!='(')res+=s.top();
    s.pop();
}cout<<res<<endl;
}
int main(){
    int t;cin>>t;
    cin.ignore();
    while (t--)>0)initwsolve();
    return 0;
}

```

BIẾN ĐỔI HẬU TỔ - TIỀN TỔ

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng hậu tố về dạng tiền tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 10^6$.

Ví dụ:

Input	Output
2	*+AB-CD

AB+CD-*	*-A/BC-/AKL
ABC/-AK/L-*	

GIẢI THÍCH TEST 2:

CHỈ SỐ	CHUỖI	STACK
0	A	A
1	B	A B
2	C	A B C
3	/	A /BC
4	-	-A/BC
5	A	-A/BC A
6	K	-A/BC A K
7	/	-A/BC /AK
8	L	-A/BC /AK L
9	-	-A/BC -/AKL
10	*	*-A/BC-/AKL
KQ		*-A/BC-/AKL

Giải:

//note: string(1,str[i]) : chuyển ký tự str[i] thành 1 chuỗi có 1 ký tự

//1: để in stack đúng biểu thức (ta thấy cứ 2 chữ thì sau đó là 1 toán tử của 2 chữ đó => duyệt xuôi

//2: nếu str[i] không phải toán tử ta s.push() còn nếu phải toán tử thì ta lấy 2 cái string đầu trong stack và cùng với toán tử để tạo thành biểu thức.

//3 :sau đó gộp biểu thức đó thành 1 string đưa vào stack . khi đó string lại coi như 1 toán hạng lớn.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
bool oprt(char x){//operator
```

```
    if(x=='+'||x=='-'||x=='*'||x=='/')
```

```
        return 1; return 0;
```

```
}
```

```
void initwsolve(){
```

```
    string str;cin>>str;
```

```
    stack <string> s;
```

```
    for(int i=0;i<str.length();i++){//1
```

```
        if(oprt(str[i])==0) s.push(string(1,str[i]));//2
```

```
        else{
```

```
            string str1=s.top();s.pop();
```

```

        string str2=s.top();s.pop();
        string temp=str[i]+str2+ str1;
        s.push(temp);//3
    }
}
cout<<s.top()<<endl;
}
int main(){
    int t;cin>>t;
    cin.ignore();
    while (t--)>0)initwsolve();
    return 0;
}

```

BIẾN ĐỔI HẬU TỔ - TRUNG TỔ

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng hậu tố về dạng trung tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.
- T , exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T , exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 10^6$.

Ví dụ:

Input	Output
2	
ABC++	(A+(B+C))
AB*C+	((A*B)+C)

Giải thích test 2:

CHỈ SỐ	CHUỖI	STACK
0	A	A
1	B	A B
2	*	(A*B)

3	C	(A*B) C
4	+	((A*B)+C)
KQ		((A*B)+C)

Giải:

//note: string(1,str[i]) : chuyển ký tự str[i] thành 1 chuỗi có 1 ký tự

//1:đề in stack đúng biểu thức (ta thấy cứ 2 chữ thì sau đó là 1 toán tử của 2 chữ đó
=> duyệt xuôi

//2:nếu str[i] không phải toán tử ta s.push() còn nếu phải toán tử thì ta lấy 2 cái string đầu trong stack và cùng với toán tử để tạo thành biểu thức.

//3 :sau đó gộp biểu thức đó thành 1 string đưa vào stack . khi đó string lại coi như 1 toán hạng lớn.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
bool oprt(char x){//operator
```

```
    if(x=='+'||x=='-'||x=='*'||x=='/')
```

```
    return 1; return 0;
```

```
}
```

```
void initwsolve(){
```

```
    string str;cin>>str;
```

```
    stack <string> s;
```

```
    for(int i=0;i<str.length();i++){//1
```

```
        if(oprt(str[i])==0) s.push(string(1,str[i]));//2
```

```
        else{
```

```
            string str1=s.top();s.pop();
```

```
            string str2=s.top();s.pop();
```

```
            string temp='('+str2+ str[i]+str1+'');
```

```
            s.push(temp);//3
```

```
        }
```

```
    }
```

```
    cout<<s.top()<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    cin.ignore();
```

```
    while (t--)>initwsolve();
```

```
    return 0;
```

```
}
```

TÍNH GIÁ TRỊ BIỂU THỨC HẬU TỔ

Hãy viết chương trình chuyển tính toán giá trị của biểu thức hậu tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức hậu tố exp. Các số xuất hiện trong biểu thức là các số đơn có 1 chữ số.

Output:

- Đưa ra kết quả mỗi test theo từng dòng, chỉ lấy giá trị phần nguyên.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 20$.

Ví dụ:

Input	Output
2	
231*+9-	-4
875*+9-	34

Giải:

//1: để tính đúng biểu thức => duyệt xuôi

//2: nếu str[i] không phải toán tử ta s.push() còn nếu phải toán tử thì ta lấy 2 cái số đầu trong stack và cùng với toán tử để tạo thành biểu thức.

//3 :sau đó đưa kết quả vừa có được là 1 số đưa vào stack . khi đó s,top() lại coi như 1 toán hạng lớn .

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
bool oprt(char x){
```

```
    if(x=='+'||x=='-'||x=='*'||x=='/')
```

```
        return 1; return 0;
```

```
}
```

```
int cal(int a,int b,char x){
```

```
    if(x=='+')return a+b; if(x=='-')return a-b;
```

```
    if(x=='*')return a*b; if(x=='/')return a/b;
```

```
}
```

```
void initwsolve(){
```

```
    string str;cin>>str;
```

```
    stack<int> s;
```

```
    for(int i=0;i<str.length();i++){
```

```
        if(oprt(str[i])==0) s.push(str[i]-'0');//1
```

```
        else{
```

```
            int str1=s.top();s.pop();
```

```

        int str2=s.top();s.pop();
        /**/ int temp=cal(str2,str1,str[i]);//str2 sau str1 ới đúng thứ tự
        s.push(temp);//3
    }
    }cout<<s.top()<<endl;
}
int main(){
    int t; cin>>t;
    while(t-->0) initwsolve();
}

```

TÍNH GIÁ TRỊ BIỂU THỨC TIỀN TỐ

Hãy viết chương trình tính toán giá trị của biểu thức tiền tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp. Các số xuất hiện trong biểu thức là các số đơn có 1 chữ số.

Output:

- Đưa ra kết quả mỗi test theo từng dòng, chỉ lấy giá trị phần nguyên.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{length}(\text{exp}) \leq 20$.

Ví dụ:

Input	Output
2	
+/8/632	8
+/7*45+20	25

Giải:

//1: để tính đúng biểu thức => duyệt ngược vì tiền tố là dấu ở trước

//2: nếu str[i] không phải toán tử ta s.push() còn nếu phải toán tử thì ta lấy 2 cái số đầu trong stack và cùng với toán tử để tạo thành biểu thức.

//3 :sau đó đưa kết quả vừa có được là 1 số đưa vào stack . khi đó s,top() lại coi như 1 toán hạng lớn .

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
bool oprt(char x){
```

```
    if(x=='+'||x=='-'||x=='*'||x=='/')
```

```
        return 1; return 0;
```

```

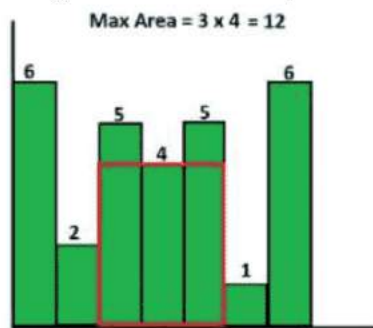
}
int cal(int a,int b,char x){
    if(x=='+')return a+b;  if(x=='-')return a-b;
    if(x=='*')return a*b;  if(x=='/')return a/b;
}
void initwsolve(){
    string str;cin>>str;
    stack<int> s;
    for(int i=str.length()-1;i>=0;i--){
        if(oprt(str[i])==0) s.push(str[i]-'0');//1
        else{
            int str1=s.top();s.pop();
            int str2=s.top();s.pop();
            /*2*/ int temp=cal(str1,str2,str[i]);//str2 sau str1 ới đúng thứ tự
            s.push(temp);//3
        }
    }cout<<s.top()<<endl;
}
int main(){
    int t; cin>>t;
    while(t-->0) initwsolve();
}

```

HÌNH CHỮ NHẬT LỚN NHẤT

Ôn lại

Cho N cột, mỗi cột có chiều cao bằng $H[i]$. Bạn hãy tìm hình chữ nhật lớn nhất bị che phủ bởi các cột?



Input:

Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).

Mỗi test bắt đầu bởi số nguyên N ($N \leq 100\,000$).

Dòng tiếp theo gồm N số nguyên $H[i]$ ($1 \leq H[i] \leq 10^9$).

Output:

Với mỗi test, in ra diện tích hình chữ nhật lớn nhất tìm được.

Ví dụ:

Input	Output
2	12
7	6
6 2 5 4 5 1 6	
3	
2 2 2	

Giải thích test 1

Vòng for thứ nhất

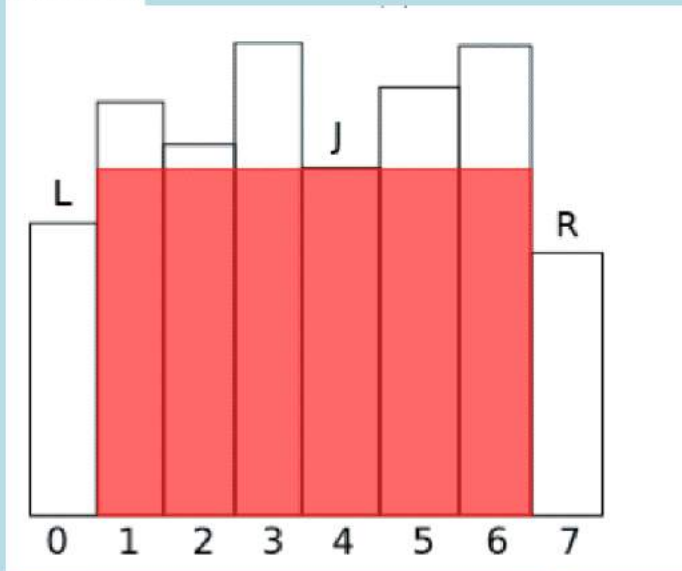
		stack	stack
vị trí	mảng	val	pos
0	6	6	0
1	2	2	1
2	5	2 5	1 2
3	4	2 4	1 3
4	5	2 4 5	1 3 4
5	1	1	5
6	6	1 6	5 6
mảng			
r	1 5 3 5 5 7 7		

Vòng for thứ 2:

		stack	stack
vị trí	mảng	val	pos
6	6	6	6
5	1	1	5
4	5	1 5	5 4
3	4	1 4	5 3
2	5	1 4 5	5 3 2
1	2	1 2	5 1
0	6	1 2 6	5 1 0
mảng	-1 -1 1 1 3 -1		
l	5		

$r-l-1 \Rightarrow$ khoảng cách $>>$

Với mỗi ô jj trên hàng ii , ta tìm $f(j)f(j)$ là số ô 1 liên tiếp trên cột jj , tính từ hàng ii trở lên. Sau đó, với mỗi cột jj , ta tiếp tục tìm ô gần nhất bên trái và ô gần nhất bên phải có ff nhỏ hơn $f(j)f(j)$, sau đó tính diện tích hình chữ nhật ở cột jj là $S=f(j) \times (r-l-1)$ với l, r là chỉ số 2 ô bên trái và bên phải nói trên.



Giải :

```
#include<bits/stdc++.h>
using namespace std;
long long n,a[1000005];
void initwsolve(){
    cin>> n;
    for(int i=0;i<n;i++) cin>>a[i];
    long long res=0;
    vector<long long> l(n),r(n);
    stack <long long> val, pos;
    //stack val và pos lưu giá trị và vị trí của số lowsnn hơn bên phải ngay sau đó
    for(int i=0;i<n;i++){
        while(val.size()&&a[i]<val.top()){
            r[pos.top()]=i;
            val.pop();pos.pop();
        }val.push(a[i]);pos.push(i);
    }
    while(val.size()){
        r[pos.top()]=n;
        val.pop();pos.pop();
    }
```

```

    }
    for(int i=n-1;i>=0;i--){
        while(val.size()&& a[i]<val.top()){
            l[pos.top()]=i;
            val.pop();pos.pop();
        } val.push(a[i]);pos.push(i);
    }
    while(val.size()){
        l[pos.top()]=-1;//note
        val.pop();pos.pop();
    }
    for(int i=0;i<n;i++){
        res=max(res,a[i]*(r[i]-l[i]-1));
    }
    cout<<res<<endl;
}
int main(){
    int t;cin>>t;
    while (t--){initwsolve();
    return 0;
}

```

Cách 2:

```

#include<bits/stdc++.h>
using namespace std;
long long n,a[1000005];
void initwsolve(){
    cin>>n;
    for(int i=0;i<n;i++) cin>>a[i];
    long long res=0;
    vector<long long> l(n),r(n);
    for(int i=0;i<n;i++) l[i]=-1;
    for(int i=0;i<n;i++) r[i]=n;
    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(a[j]<a[i]) {r[i]=j;break;}
        }
    }
    for(int i=1;i<n;i++){

```

```

        for(int j=i-1;j>=0;j--){
            if(a[j]<a[i]) {l[i]=j;break;}
        }
    }
    for(int i=0;i<n;i++){
        res=max(res,a[i]*(r[i]-l[i]-1));
    }
    cout<<res<<endl;
}
int main(){
    int t;cin>>t;
    while (t-->0)initwsolve();
    return 0;
}

```

BIỂU THỨC TĂNG GIẢM

Ôn lại

Cho dãy ký tự S chỉ bao gồm các ký tự I hoặc D. Ký tự I được hiểu là tăng (Increasing) ký tự D được hiểu là giảm (Decreasing). Sử dụng các số từ 1 đến 9, hãy đưa ra số nhỏ nhất được đoán nhận từ S. Chú ý, các số không được phép lặp lại. Dưới đây là một số ví dụ mẫu:

- A[] = "I" : số tăng nhỏ nhất là 12.
- A[] = "D" : số giảm nhỏ nhất là 21
- A[] = "DD" : số giảm nhỏ nhất là 321
- A[] = "DDIDDIID": số thỏa mãn 321654798

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test là một xâu S
- T, S thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(S) \leq 8$; .

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input:	Output:
4	12
I	21
D	321
DD	321654798
DDIDDIID	

Giải :

//1 : ta thấy chuỗi có n ký tự nhưng kq có n+1 ký tự =>for(int i=1;i<=str.length()
//2 : dùng 1 stack để lưu trữ dãy giảm => tìm được ký tự tăng ('I'hoặc /0) thì sẽ in
luôn , và in luôn cả stack

// 3: còn nếu phải ký tự giảm ('D') thì đưa nó vào ngăn xếp

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    string str;cin>>str;
```

```
    str=' '+str;
```

```
    stack<int> s;
```

```
    for(int i=1;i<=str.length();i++){//1
```

```
        if(str[i]=='D'){s.push(i); }//3
```

```
        else{//2
```

```
            cout<<i;
```

```
            while(!s.empty()){
```

```
                cout<<s.top(); s.pop();
```

```
            }
```

```
        }
```

```
    }
```

```
    cout<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    while (t--){initwsolve();
```

```
        return 0;
```

```
}
```

BIỂU THỨC TƯƠNG ĐƯƠNG

Ôn lại

Cho biểu thức đúng P chỉ bao gồm các phép toán +, -, các toán hạng cùng với các ký tự '(', ')'. Hãy bỏ tất cả các ký tự '(', ')’ trong P để nhận được biểu thức tương đương. Ví dụ với $P = a - (b + c)$ ta có kết quả $P = a - b - c$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức P được viết trên một dòng.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, P thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(P) \leq 10^3$.

Ví dụ:

Input	Output
2	
a-(b+c)	a-b-c
a-(b-c-(d+e))-f	a-b+c+d+e-f

Giải thích test 2:

chỉ số	chuỗi	a-(b+c)	lệnh
0	a		
1	-		
2	(	-	if1
3	b	-	
4	+	-	if2
5	c	-	
6	)		
kq		a-b-c	

Giải :

// xét trong một ngăn xếp lưu trữ dấu . có 3 th liên quan đến ngăn xếp:

//1: nếu gặp '(' thì ta đưa dấu + hoặc - trước '(' vào stack để xét xem các dấu trong biểu thức có bị đảo hay không

//2:ta thấy là các dấu sẽ bị đảo nếu dấu trước () là dấu '-' . => + thành - và - thành +

//3: khi gặp ')' thì ta s.pop() tức xóa dấu trước dấu ngoặc (ám chỉ là đã thực hiện xét các dấu có trong biểu thức () rồi)

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    string str;cin>>str;
```

```
    int res=0;
```

```
    stack<char> s;
```

```
    for(int i=0;i<str.length();i++){
```

```
        if(str[i]=='('){//1
```

```
            if(str[i-1]=='+'||str[i-1]=='-') s.push(str[i-1]);// note : phải có dòng if
```

```
        }
```

```
        /*2*/if(s.size()&& s.top()=='-') {//note s.size trước s.top
```

```

        if(str[i]=='+') str[i]='-';
        else if(str[i]=='-') str[i]='+';
    }
    if(str[i]=='')&& s.size())s.pop();
}
for(int i=0;i<str.length();i++){
    if(str[i]!='(' && str[i]!=')') cout<<str[i];
}cout<<endl;
}
int main(){
    int t;cin>>t;
    while (t-->0)initwsolve();
    return 0;
}

```

PHẦN TỬ BÊN PHẢI NHỎ HƠN

Ôn lại

Cho mảng $A[]$ gồm n phần tử. Hãy đưa ra các phần tử nhỏ hơn tiếp theo của phần tử lớn hơn đầu tiên phân tử hiện tại. Nếu phần tử hiện tại không có phần tử lớn hơn tiếp theo ta xem là -1. Nếu phần tử không có phần tử nhỏ hơn tiếp theo ta cũng xem là -1. Ví dụ với mảng $A[] = \{5, 1, 9, 2, 5, 1, 7\}$ ta có kết quả là $ans = \{2, 2, -1, 1, -1, -1, -1\}$ vì:

Next Greater		Right Smaller	
5	-> 9	9	-> 2
1	-> 9	9	-> 2
9	-> -1	-1	-> -1
2	-> 5	5	-> 1
5	-> 7	7	-> -1
1	-> 7	7	-> -1
7	-> -1	7	-> -1

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n là số phần tử của mảng $A[]$, dòng tiếp theo đưa vào n số $A[i]$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n$, $A[i] \leq 10^6$.

Ví dụ:

Input	Output
2	
7	
5 1 9 2 5 1 7	2 2 1 1 -1 -1 -1
8	2 5 5 5 -1 3 -1 -1
4 8 2 1 9 5 6 3	

Giải thích test 1:

Vòng for 1

chỉ số	mảng	stack val	stack pos
0	5	5	0
1	1	5 1	0 1
2	9	9	2
3	2	9 2	2 3
4	5	9 5	2 4
5	1	9 5 1	2 4 5
6	7	9 7	2 6

mảng

b:
2 2 -1 4 6 6 -
1

Vòng for 2:

chỉ số	mảng	stack val	stack pos
0	5	5	0
1	1	1	1
2	9	1 9	1 2
3	2	1 2	1 3
4	5	1 2 5	1 3 4
5	1	1 1	1 5
6	7	1 1 7	1 5 6

mảng

c
1 -1 3 5 5 -1
-1

Giải :

//1: khi gặp 1 số lớn hơn s.top() thì ta xóa toàn bộ các số và vị trí mà số <a[i] có trong stack, đưa toàn bộ dữ liệu của pos vào mảng b, cập nhật số lớn hơn và vị trí của nó vào 2 stack

//mảng a lưu trữ các số gốc, mảng b lưu trữ vị trí các số > tiếp theo, mảng c lưu trữ vị trí các số < tiếp theo

//2: kết thúc vòng for đầu ta tìm được các phần tử ko thỏa mãn lớn hơn đầu tiên phần tử hiện tại => cho kết quả=-1 từ cách lấy các vị trí từ pos

//3: stack val sẽ lưu trữ tất cả các phần tử ko thỏa mãn nhỏ hơn tiếp theo của phần tử hiện tại và stack pos sẽ lưu trữ vị trí của chúng.

//4: kết thúc vòng for thứ hai ta tìm được các phần tử ko thỏa mãn nhỏ hơn tiếp theo phần tử hiện tại => cho kết quả =-1 từ việc lấy các vị trí từ pos

// 5: như vậy nếu như b[i]!=-1 (tức tìm được số lớn hơn đầu tiên p tử hiện tại) &&c[b[i]]!=-1 (tìm được số nhỏ hơn tiếp theo của số lớn hơn đầu tiên p tử hiện tại) thì in ra số thỏa mãn, ngược lại in ra -1

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int n,a[1000005],b[1000005],c[1000005]; //mảng a lưu trữ các số gốc, mảng b
```

```
void initwsolve(){
```

```
    cin>>n;
```

```
    for(int i=0;i<n;i++) cin>>a[i];
```

```
    stack<int> val,pos;
```

```
    /*1*/ for(int i=0;i<n;i++){
```

```
        while(val.size()&&a[i]>val.top()){
```

```
            b[pos.top()]=i;
```

```
            val.pop();pos.pop();
```

```
        }
```

```
        val.push(a[i]);pos.push(i);
```

```
    }
```

```
    /*2*/ while(val.size()){ //cho những số >> <<=-1
```

```
        b[pos.top()]=-1;
```

```
        val.pop();pos.pop();
```

```
    }
```

```
    /*3*/ for(int i=0;i<n;i++){
```

```
        while(val.size()&&a[i]<val.top()){
```

```
            c[pos.top()]=i;
```

```
            val.pop();pos.pop();
```

```
        }
```

```
        val.push(a[i]);pos.push(i);
```

```

    }
/*4*/ while(val.size()){
    c[pos.top()]=-1;
    val.pop();pos.pop();
}
/*5*/ for(int i=0;i<n;i++){
    if(b[i]!=-1&& c[b[i]]!=-1) cout<<a[c[b[i]]]<<' ';
    else cout<<-1<<' ';
}
cout<<endl;
}
int main(){
    int t;cin>>t;
    while (t--)initwsolve();
    return 0;
}

```

SO SÁNH BIỂU THỨC

Gần y hệt **BIỂU THỨC TƯƠNG ĐƯƠNG**

Cho P1, P2 là hai biểu thức đúng chỉ bao gồm các ký tự mở ngoặc '(' hoặc đóng ngoặc ')' và các toán hạng in thường. Nhiệm vụ của bạn là định xem P1 và P2 có giống nhau hay không.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào P1, dòng tiếp theo đưa vào P2.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, P thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(P) \leq 100$.

Ví dụ:

Input	Output
2	
-(a+b+c)	YES
-a-b-c	NO
a-b-(c-d)	

Giải:

// xét trong một ngăn xếp lưu trữ dấu . có 3 th liên quan đến ngăn xếp:
 //1: nếu gặp '(' thì ta đưa dấu + hoặc - trước '(' vào stack để xét xem các dấu trong biểu thức có bị đảo hay không
 //2:ta thấy là các dấu sẽ bị đảo nếu dấu trước () là dấu '-' . => + thành - và - thành +
 //3: khi gặp ')' thì ta s.pop() tức xóa dấu trước dấu ngoặc (ám chỉ là đã thực hiện xét các dấu có trong biểu thức () rồi)

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
string repair(string str){
```

```
    stack<char> s;
```

```
    for(int i=0;i<str.length();i++){
```

```
        if(str[i]=='('){//1
```

```
            if(str[i-1]=='+'||str[i-1]=='-') s.push(str[i-1]);
```

```
        }
```

```
        /*2*/if(s.size()&&str[i-1]=='-'){//note s.size trước s.top
```

```
            if(str[i]=='+') str[i]='-';
```

```
            else if(str[i]=='-') str[i]='+';
```

```
        }
```

```
        if(str[i]==')'&&s.size())s.pop();
```

```
    }
```

```
    string res="";
```

```
    for(int i=0;i<str.length();i++){
```

```
        if(str[i]!='(' && str[i]!=')') res+=str[i];
```

```
    }
```

```
    return res;
```

```
}
```

```
void initwsolve(){
```

```
    string s1,s2;cin>>s1>>s2;
```

```
    if(repair(s1)==repair(s2)) cout<<"YES"<<endl;
```

```
    else cout<<"NO"<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    while (t-->0)initwsolve();
```

```
    return 0;
```

```
}
```

PHẦN TỬ CÓ SỐ LẦN XUẤT HIỆN NHIỀU HƠN BÊN PHẢI

Ôn lại

Cho mảng $A[]$ gồm n phần tử. Nhiệm vụ của bạn là tìm phần tử gần nhất phía bên phải có số lần xuất hiện lớn hơn phần tử hiện tại. Nếu không có phần tử nào có số lần xuất hiện lớn hơn phần tử hiện tại hãy đưa ra -1 .

Ví dụ với dãy $A[] = \{1, 1, 2, 3, 4, 2, 1\}$, ta nhận được kết quả $ans[] = \{-1, -1, 1, 2, 2, 1, -1\}$ vì số lần xuất hiện mỗi phần tử trong mảng là $F = \{3, 3, 2, 1, 1, 2, 3\}$ vì vậy phần tử $A[0] = 1$ có số lần xuất hiện là 3 và không có phần tử nào xuất hiện nhiều hơn 3 nên $ans[0] = -1$, tương tự như vậy với $A[2]=2$ tồn tại $A[6] = 1$ có số lần xuất hiện là 3 nên $ans[2] = 1$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng, dòng đầu tiên đưa vào số n là số các phần tử của mảng $A[]$; dòng tiếp theo đưa vào n số của mảng $A[]$; các phần tử được viết cách nhau một vài khoảng trống.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $0 \leq n, A[i] \leq 10^6$.

Ví dụ:

Input	Output
1 7 1 1 2 3 4 2 1	-1 -1 1 2 2 1 -1

chỉ số	mảng	stack val	stack pos
0	1	1	0
1	1	1 1	0 1
2	2	1 1 2	0 1 2
		1 1 2	0 1 2
3	3	3	3
4	4	1 1 2	0 1 2

		3 4	3 4
		1 1 2	0 1 2
5	2	2	5
6	1	1 1 1	0 1 6

```

mảng -1 -1 6 5 5 6
b     -1
mảng -1 -1 1 2 2 1
a     -1

```

Giải :

//mảng a lưu trữ các số gốc , mảng f lưu trữ tần suất , mảng b để lưu trữ vị trí của số có tần suất lớn hơn ở bên phải nó

// stack val và pos dùng để lưu giá trị và vị trí của các số không thể tìm được số có tần suất lớn hơn ở bên phải nó

// (cuối cùng tại các vị trí đó cho nó bằng = -1)

// in ra như đầu bài.vì mảng b là mảng đánh dấu vị trí => in ra các a[b[i]]nếu b[i] !=1 , còn lại in -1

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int n,a[1000005],b[1000005],f[1000005];
```

```
void initwsolve(){
```

```
    cin>>n;
```

```
    memset(f,0,sizeof(f));
```

```
    for(int i=0;i<n;i++) {cin>>a[i];f[a[i]]++;}
```

```
    stack<int> val,pos;
```

```
    for(int i=0;i<n;i++){
```

```
        while(val.size()&&f[a[i]]>f[val.top()]){
```

```
            b[pos.top()]=i;
```

```
            val.pop();pos.pop();
```

```
        }
```

```
        val.push(a[i]);pos.push(i);
```

```
    }
```

```
    while(val.size()){
```

```
        b[pos.top()]=-1;
```

```
        val.pop();pos.pop();
```

```
    }
```

```
    for(int i=0;i<n;i++) {
```

```
        if(b[i]!=-1) cout<<a[b[i]]<<' ';
```

```

        else cout<<-1<<' ';
    }cout<<endl;
}
int main(){
    int t;cin>>t;
    while (t--)initwsolve();
    return 0;
}

```

Hàng đợi

Một số hàm sử dụng đến :

```

void out(priority_queue<long long> p){
    while(!p.empty()){
        cout<<p.top()<<' ';
        p.pop();
    }
    cout<<endl;
}

```

```

void out(queue<string> q){
    while(!q.empty()){
        cout<<q.front()<<' ';
        q.pop();
    }
    cout<<endl;
}

```

CẤU TRÚC DỮ LIỆU HÀNG ĐỢI 1

Bạn đầu cho một queue rỗng. Bạn cần thực hiện các truy vấn sau:

1. Trả về kích thước của queue
2. Kiểm tra xem queue có rỗng không, nếu có in ra “YES”, nếu không in ra “NO”.
3. Cho một số nguyên và đẩy số nguyên này vào cuối queue.
4. Loại bỏ phần tử ở đầu queue nếu queue không rỗng, nếu rỗng không cần thực hiện.
5. Trả về phần tử ở đầu queue, nếu queue rỗng in ra -1.

6. Trả về phần tử ở cuối queue, nếu queue rỗng in ra -1.

Dữ liệu vào

Dòng đầu tiên chứa số nguyên T là số bộ dữ liệu, mỗi bộ dữ liệu theo dạng sau.

Dòng đầu tiên chứa số nguyên n - lượng truy vấn ($1 \leq n \leq 1000$)

N dòng tiếp theo, mỗi dòng sẽ ghi loại truy vấn như trên, với truy vấn loại 3 sẽ có thêm một số nguyên, không quá 10^6 .

Kết quả: In ra kết quả của các truy vấn..

Ví dụ:

Input	Output
1	
14	
3 1	
3 2	
3 3	
5	
6	1
4	3
4	5
4	2
4	
4	
3 5	
3 6	
5	
1	

Giải : rất dễ

```
#include<bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n;cin>>n;
    queue<int> q;
    while (n--){
        int ask;cin>>ask;
        switch(ask){
            case 1: cout<<q.size()<<endl;break;
            case 2: if(q.size()==0)cout<<"YES"<<endl;
                    else cout<<"NO"<<endl;break;
        }
    }
}
```

```

        case 3: int temp;cin>>temp;q.push(temp);break;
        case 4: if(q.size())q.pop();break;
        case 5:if(q.size()) cout<<q.front()<<endl;
                else cout<<-1<<endl;break;
        case 6:if(q.size()) cout<<q.back()<<endl;
                else cout<<-1<<endl;break;
    }
}
}
int main(){
    int t;cin>>t;
    while (t--)<initwsolve();
    return 0;
}

```

CẤU TRÚC DỮ LIỆU HÀNG ĐỘI 2

Yêu cầu bạn xây dựng một queue với các truy vấn sau đây:

“PUSH x”: Thêm phần tử x vào cuối của queue ($0 \leq x \leq 1000$).

“PRINTFRONT”: In ra phần tử đầu tiên của queue. Nếu queue rỗng, in ra “NONE”.

“POP”: Xóa phần tử ở đầu của queue. Nếu queue rỗng, không làm gì cả.

Dữ liệu vào:

Dòng đầu tiên là số lượng truy vấn Q ($Q \leq 100000$).

Mỗi truy vấn có dạng như trên.

Kết quả:

Với mỗi truy vấn “PRINT”, hãy in ra phần tử đầu tiên của queue. Nếu queue rỗng, in ra “NONE”.

Ví dụ:

Input	Output
9	
PUSH 1	
PUSH 2	2
POP	2
PRINTFRONT	NONE
PUSH 3	
PRINTFRONT	

POP	
POP	
PRINTFRONT	

```
#include<bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n;cin>>n;
    cin.ignore();
    queue<int> q;
    while(n--){
        string s;cin>>s;
        if(s=="PUSH"){int temp;cin>>temp;q.push(temp);}
        else if(s=="PRINTFRONT"){
            if(q.size())cout<<q.front()<<endl;
            else cout<<"NONE"<<endl;
        }
        else{
            if(q.size()) q.pop();
        }
    }
}
int main(){
    initwsolve();
    return 0;
}
```

HÀNG ĐỢI HAI ĐẦU (DEQUEUE)

Ôn lại

Yêu cầu bạn xây dựng một hàng đợi hai đầu với các truy vấn sau đây:

“PUSHFRONT x”: Thêm phần tử x vào đầu của dequeue ($0 \leq x \leq 1000$).

“PRINTFRONT”: In ra phần tử đầu tiên của dequeue. Nếu dequeue rỗng, in ra “NONE”.

“POPFRONT”: Xóa phần tử đầu của dequeue. Nếu dequeue rỗng, không làm gì cả.

“PUSHBACK x”: Thêm phần tử x vào cuối của dequeue ($0 \leq x \leq 1000$).

“PRINTBACK”: In ra phần tử cuối của dequeue. Nếu dequeue rỗng, in ra “NONE”.

“POPBACK”: Xóa phần tử cuối của dequeue. Nếu dequeue rỗng, không làm gì cả.

Dữ liệu vào:

Dòng đầu tiên là số lượng truy vấn Q ($Q \leq 100000$).

Mỗi truy vấn có dạng như trên.

Kết quả:

Với mỗi truy vấn “PRINTFRONT” và “PRINTBACK”, hãy in ra kết quả trên một dòng.

Ví dụ:

Input	Output
10	
PUSHBACK 1	
PUSHFRONT 2	
PUSHBACK 3	
PRINTFRONT	2
POPFRONT	1
PRINTFRONT	3
POPFRONT	NONE
PRINTBACK	
POPFRONT	
PRINTBACK	

Giải :

//tham khảo deque trong : <https://www.geeksforgeeks.org/deque-cpp-stl/>

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    int n;cin>>n;
```

```
    cin.ignore();
```

```
    deque<int> d;
```

```
    while(n--){
```

```
        string s;cin>>s;
```

```
        if(s=="PUSHFRONT"){int temp;cin>>temp;d.push_front(temp);}
```

```
        else if(s=="PUSHBACK"){int temp;cin>>temp;d.push_back(temp);}
```

```
        else if(s=="PRINTFRONT"){
```

```
            if(d.size())cout<<d.front()<<endl; else cout<<"NONE"<<endl;
```

```
        }
```

```
        else if(s=="PRINTBACK"){
```

```
            if(d.size())cout<<d.back()<<endl; else cout<<"NONE"<<endl;
```

```
        }
```

```
        else if(s=="POPFRONT"&& d.size()) d.pop_front();
```



```

void initwsolve(){
    int n;cin>>n;
    cin.ignore();
    string s;cin>>s;
    int f[26]; memset(f,0,sizeof(f));
    for(int i=0;i<s.length();i++) f[s[i]-'A']++;
    sort(f,f+26,greater<int>());//1
    priority_queue<long long> p;
    long long res=0;
    for(int i=0;i<26;i++) p.push(f[i]);//2
    while(p.size()){
        long long temp1=p.top();
        if(n){//3
            p.pop();
            p.push(temp1-1);n--;
        }
        else{//4
            res+= temp1*temp1;
            p.pop();
        }
    }
    cout<<res<<endl;
}
int main(){
    int t;cin>>t;
    while(t--){initwsolve();
        return 0;
    }
}

```

SỐ 0 VÀ SỐ 9

Cho số tự nhiên N . Hãy tìm số nguyên dương X nhỏ nhất được tạo bởi số 9 và số 0 chia hết cho N . Ví dụ với $N = 5$ ta sẽ tìm ra $X = 90$.

Input:

- Dòng đầu tiên ghi lại số lượng test T ($T \leq 100$).
- Những dòng kế tiếp mỗi dòng ghi lại một test. Mỗi test là một số tự nhiên N được ghi trên một dòng ($N \leq 100$).

Output:

- Đưa ra theo từng dòng số X nhỏ nhất chia hết cho N tìm được .

Ví dụ:

Input	Output
2	90
5	9009
7	

Giải thích: queue từ đầu while đến khi while kết thúc:

2
7
90 99
99 900 909
900 909 990 999
909 990 999 9000 9009
990 999 9000 9009 9090 9099
999 9000 9009 9090 9099 9900 9909
9000 9009 9090 9099 9900 9909 9990 9999
9009 9090 9099 9900 9909 9990 9999 90000 90009

Kq : 9009

Giải :

//1:ban đầu đưa 9 vào trong queue

//2:thực hiện thêm vào queue các số sau (90,99,...) bằng cách

q.push(q.front()*10);q.push(q.front()*10+9);

//3:tại đó xóa số cũ đi . thêm vào cho đến khi nào tìm được q.front() chia hết cho n thì kết thúc vòng lặp

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    long long n;cin>>n;
```

```
    queue<long long> q;
```

```
    q.push(9);//1
```

```
    while(1){
```

```
        /*3*/ if(q.front()%n==0){cout<<q.front()<<endl;break;}
```

```
        else{//2
```

```
            q.push(q.front()*10);
```

```
            q.push(q.front()*10+9);
```

```
            q.pop();
```

```
        }
```

```
    }
```

```
}
```

```
int main(){
    int t;cin>>t;
    while(t-->0)initwsolve();
    return 0;
}
```

SỐ NHỊ PHÂN TỪ 1 ĐẾN N

Ôn lại

Cho số tự nhiên n. Hãy in ra tất cả các số nhị phân từ 1 đến n.

Input:

- Dòng đầu tiên ghi lại số lượng test T ($T \leq 100$).
- Mỗi test là một số tự nhiên n được ghi trên một dòng ($n \leq 10000$).

Output:

- Đưa ra kết quả mỗi test trên một dòng.

Ví dụ:

Input	Output
2	1 10
2	1 10 11 100 101
5	

Giải thích : trạng thái queue khi n=5:

10 11

11 100 101

100 101 110 111

101 110 111 1000 1001

110 111 1000 1001 1010 1011

Giải:

// note: inn từ 1 đến n => tư duy phải chạy vòng for 1-n;

//chạy từ 1 => đẩy 1 vào queue, sau đó cứ mỗi lần lấy số đầu của queue thì đẩy vào đó 2 số temp+'0' và temp+'1'

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    int n;cin>>n;
```

```
    queue<string> q;
```

```
    q.push("1");
```

```
    for(int i=1;i<=n;i++){//1
```

```
        string temp=q.front();
```

```
        cout<<temp<<' ';
```

```
        q.pop();
```

```

        q.push(temp+'0');
        q.push(temp+'1');
    }cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t--)initwsolve();
    return 0;
}

```

Cách 2 : dùng long long

// note: inn từ 1 đến n => tư duy phải chạy vòng for 1-n;

//chạy từ 1 => đẩy 1 vào queue , sauddos cứ mỗi lần lấy số đầu của queue thì đẩy vào đó 2 số temp+'0' và temp+'1'

```

#include<bits/stdc++.h>
using namespace std;
void initwsolve(){
    int n;cin>>n;
    queue<long long> q;
    q.push(1);
    for(int i=1;i<=n;i++){//1
        long long temp=q.front();
        cout<<temp<<' ';
        q.pop();
        q.push(temp*10);
        q.push(temp*10+1);
    }cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t--)initwsolve();
    return 0;
}

```

SỐ BDN 1 (nhị phân)

Ôn lại

Ta gọi số nguyên dương K là một số BDN nếu các chữ số trong K chỉ bao gồm các 0 hoặc 1 có nghĩa. Ví dụ số K = 1, 10, 101. Cho số tự nhiên N ($N < 2^{63}$). Hãy

cho biết có bao nhiêu số BDN nhỏ hơn N. Ví dụ N=100 ta có 4 số BDN bao gồm các số: 1, 10, 11, 100.

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng Test;
- T dòng kế tiếp mỗi dòng ghi lại một bộ Test. Mỗi test là một số tự nhiên N.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
3	2
10	4
100	7
200	

Giải:

//1: đếm đến khi nào $\leq n$ thì thôi $\Rightarrow$ dùng while

//2: bước đầu phải đẩy 1 vào queue, sau đó cứ mỗi lần lấy số đầu của queue thì đẩy vào đó 2 số $temp+0$ và $temp+1$

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    long long n;cin>>n;//note :ll
```

```
    queue<long long> q;
```

```
    q.push(1);//2
```

```
    int res=0;
```

```
    while(1){//1
```

```
        long long temp=q.front();
```

```
        if (temp<=n)res++; else break;
```

```
        q.pop();
```

```
        q.push(temp*10);//2
```

```
        q.push(temp*10+1);
```

```
    }cout<<res<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    while(t-->0)initwsolve();
```

```
    return 0;
```

```
}
```

SỐ BDN 2 (chia hết cho N)

Ta gọi số nguyên dương K là một số BDN nếu các chữ số trong K chỉ bao gồm các 0 hoặc 1 có nghĩa. Ví dụ số K = 101 là số BDN, k=102 không phải là số BDN.

Số BDN của N là số $P = M * N$ sao cho P là số BDN. Cho số tự nhiên N ($N < 1000$), hãy tìm số BDN nhỏ nhất của N.

Ví dụ. Với $N=2$, ta tìm được số BDN của N là $P = 5 * 2 = 10$. $N = 17$ ta tìm được số BDN của 17 là $P = 653 * 17 = 11101$.

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng Test;
- T dòng kế tiếp mỗi dòng ghi lại một bộ Test. Mỗi test là một số tự nhiên N.

Output:

Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
3	10
2	11100
12	11101
17	

Giải:

// ý tưởng dùng queue đẩy các số chứa các cs 1 và 0 đến khi nào tại q.front() mà chia hết cho n thì in ra kết quả và dùng tìm kiếm

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    int n;cin>>n;
```

```
    queue <long long> q;
```

```
    q.push(1);
```

```
    while(1){
```

```
        long long temp=q.front();
```

```
        if(temp%n==0){cout<<temp<<endl;break;}
```

```
        q.pop();
```

```
        q.push(temp*10);
```

```
        q.push(temp*10+1);
```

```
    }
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
while(t--)>initwsolve();
return 0;
}
```

SỐ LỘC PHÁT 1 (gồm 6 và 8)

Một số được gọi là lộc phát nếu chỉ có 2 chữ số 6 và 8. Cho số tự nhiên N. Hãy liệt kê các số lộc phát có không quá N chữ số.

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng bộ test ($T < 10$);
- T dòng kế tiếp mỗi dòng ghi số N ($1 < N < 15$).

Output:

- In ra đáp án theo thứ tự giảm dần.

Ví dụ:

Input	Output
2	88 86 68 66 8 6
2	888 886 868 866 688 686 668 666 88 86 68 66 8 6
3	

Giải :

//1: ý tưởng dùng queue đẩy các số chứa các cs 6 và 8 đến khi nào độ dài q.front() == n thì thôi

//2 như vậy mỗi 1 lần xóa q.front() thì phải đưa chúng vào 1 vector .

//3 cuối cùng các kết quả dư thừa trong queue cũng phải đưa vào vector nốt => 2 lần

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    int n;cin>>n;
```

```
    queue <string> q;
```

```
    q.push("6");q.push("8");
```

```
    vector<string> res;
```

```
    while(1){
```

```
        string temp=q.front();
```

```
        res.push_back(temp);//2
```

```
        q.pop();
```

```
        /*1*/ if(temp.length()<=n-1){
```

```
            q.push(temp+'6');
```

```
            q.push(temp+'8');
```

```
        }
```

```

        else break;
    }
    /*3*/while(q.size()){
        res.push_back(q.front());
        q.pop();
    }
    for(int i=res.size()-1;i>=0;i--) cout<<res[i]<<' ';
    cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t--){initwsolve();
    return 0;
}

```

SỐ LỘC PHÁT 2

Một số được gọi là lộc phát nếu chỉ có 2 chữ số 6 và 8. Cho số tự nhiên N. Hãy liệt kê các số lộc phát có không quá N chữ số.

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng bộ test ($T < 10$);
- T dòng kế tiếp mỗi dòng ghi số N ($1 < N < 15$).

Output:

- Dòng đầu tiên là số lượng số lộc phát tìm được. Dòng thứ hai in đáp án theo thứ tự tăng dần.

Ví dụ:

Input	Output
2	6
2	6 8 66 68 86 88
3	14 6 8 66 68 86 88 666 668 686 688 866 868 886 888

Giải :

//1: ý tưởng dùng queue đẩy các số chứa các cs 6 và 8 đến khi nào độ dài $q.front() > n$ thì thôi

//2 như vậy mỗi 1 lần xóa $q.front()$ thì phải đưa chúng vào 1 vector .

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```

void initwsolve(){
    int n;cin>>n;
    queue<string> q;
    q.push("6"); q.push("8");//1
    vector <string> res;
    while( q.front().size()<=n ){//1
        string temp=q.front();
        res.push_back(temp);//2
        q.pop();//2
        q.push(temp+"6");
        q.push(temp+"8");
    }
    cout<<res.size()<<endl;//in biến đếm
    for(int i=0;i<res.size();i++)
        cout<<res[i]<<' ';
    cout<<endl;
}

int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
    return 0;
}

```

SỐ LỘC PHÁT 3

Một số được gọi là lộc phát nếu chỉ có 2 chữ số 6 và 8. Cho số tự nhiên N. Hãy liệt kê các số lộc phát có không quá N chữ số.

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng bộ test ($T < 10$);
- T dòng kế tiếp mỗi dòng ghi số N ($1 < N < 15$).

Output:

- Dòng đầu tiên là số lượng số lộc phát tìm được. Dòng thứ hai in ra đáp án theo thứ tự giảm dần.

Ví dụ:

Input	Output
2	6
2	88 86 68 66 8 6

Giải:

//1: ý tưởng dùng queue đẩy các số chứa các cs 6 và 8 đến khi nào độ dài q.front() > n thì thôi

//2 như vậy mỗi 1 lần xóa q.front() thì phải đưa chúng vào 1 vector .

//3 : giống y lược phát 2 nhưng chỉ là in kết quả theo thứ tự ngược lại

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    int n;cin>>n;
```

```
    queue<string> q;
```

```
    q.push("6"); q.push("8");//1
```

```
    vector <string> res;
```

```
    while( q.front().size()<=n ){//1
```

```
        string temp=q.front();
```

```
        res.push_back(temp);//2
```

```
        q.pop();//2
```

```
        q.push(temp+"6");
```

```
        q.push(temp+"8");
```

```
    }
```

```
    cout<<res.size()<<endl;//in biến đếm
```

```
    for(int i=res.size()-1;i>=0;i--)//3
```

```
        cout<<res[i]<<' ';
```

```
    cout<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    while(t--) initwsolve();
```

```
    return 0;
```

```
}
```

BIẾN ĐỔI S – T (thao tác : nhân , trừ)

Ôn lại

Cho hai số nguyên dương S và T ($S, T < 10000$) và hai thao tác (a), (b) dưới đây:

Thao tác (a): Trừ S đi 1 ($S = S - 1$);

Thao tác (b): Nhân S với 2 ($S = S * 2$);

Hãy dịch chuyển S thành T sao cho số lần thực hiện các thao tác (a), (b) là ít nhất. Ví dụ với S=2, T=5 thì số các bước ít nhất để dịch chuyển S thành T thông qua 4 thao tác sau:

Thao tác (a): $2 * 2 = 4$;

Thao tác (b): $4 - 1 = 3$;

Thao tác (a): $3 * 2 = 6$;

Thao tác (b): $6 - 1 = 5$;

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng Test;
 - T dòng kế tiếp mỗi dòng ghi lại một bộ Test. Mỗi test là một bộ đôi S và T.
- Output: Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
3	4
2 5	4
3 7	3
7 4	

Giải :

//1: chú ý mảng c là dùng để đếm , giá trị của c[i] là số cước tạo thành số i từ số s

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
void initwsolve(){
```

```
    int s,t; cin>>s>>t;
```

```
    if(s>=t){
```

```
        cout<<s-t<<endl; return;
```

```
    }
```

```
    int c[20005];//1
```

```
    for(int i=0;i<20005;i++) c[i]=1e9;
```

```
    c[s]=0;
```

```
    queue<int> q;
```

```
    q.push(s);
```

```
    while( c[t]==1e9){
```

```
        int temp=q.front();q.pop();
```

```
        if(temp-1>0&& c[temp-1]==1e9)
```

```
            q.push(temp-1), c[temp-1]=c[temp]+1;
```

```
        if(temp*2<=20000&& c[temp*2]==1e9)
```

```

        q.push(temp*2), c[temp*2]=c[temp]+1;
    }
    cout<<c[t]<<endl;

}

int main(){
    int test;cin>>test;
    while(test--) initwsolve();
    return 0;
}

```

TÌM SỐ K THỎA MÃN ĐIỀU KIỆN (cs khác nhau và ≤ 5)

Ôn lại

Cho hai số nguyên dương L, R. Hãy đưa ra số các số K trong khoảng [L, R] thỏa mãn điều kiện:

- Tất cả các chữ số của K đều khác nhau.
- Tất cả các chữ số của K đều nhỏ hơn hoặc bằng 5.

Ví dụ với L = 4, R = 13 ta có 5 số thỏa mãn yêu cầu là 4, 5, 10, 12, 13,

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Dòng tiếp theo đưa vào các bộ test. Mỗi bộ test được là một cặp L, R được viết trên một dòng.
- T, L, R thỏa mãn ràng buộc: $1 \leq T \leq 100$; $0 \leq L \leq R \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	5
4 13	100
100 1000	

Giải :

// 1: y tưởng kiểm tra từng số từ 1 đến r

// 2: tại từng số nó ta tách các chữ số ra đưa vào hàng đợi ưu tiên để sắp xếp giảm dần

// 3: sau đó cứ lấy từng chữ số : nếu nó > 5 hoặc giống với chữ số trước thì loại

// 4: cho đến cuối cùng nếu các c số đều thỏa mãn thì mới return 1

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```

int check(int n){
    priority_queue <int>pq;
    while(n){//2
        pq.push(n%10);
        n/=10;
    }
    int temp=0;
    /*3*/ if(pq.top()>5)return 0;
    else {
        temp=pq.top();
        pq.pop();}
    while(pq.size()){
        if(pq.top()>5) return 0;
        if(pq.top()==temp) return 0;
        temp=pq.top();pq.pop();
    }
    return 1;//4
}

void initwsolve(){
    int l,r; cin>>l>>r;
    int res=0;
    for(int i=l;i<=r;i++)//1
        if(check(i)==1) res++;
    cout<<res<<endl;
}

int main(){
    int t;cin>>t;
    while(t--){initwsolve();
}

```

Cách 2:

```

#include<bits/stdc++.h>
using namespace std;
int check(int n){
    vector<int > a;
    while(n){
        a.push_back(n%10);
        n/=10;
    }
}

```

```

    }
    a.push_back(-1);
    sort(a.begin(),a.end(),greater<int>() );
    for(int i=0;i<a.size()-1;i++)
        if(a[i]>5||a[i]==a[i+1]) return 0;
    return 1;
}
void initwsolve(){
    int l,r; cin>>l>>r;
    int res=0;
    for(int i=l;i<=r;i++)
        if(check(i)==1) res++;
    cout<<res<<endl;
}
int main(){
    int t;cin>>t;
    while(t--){initwsolve();
}

```

Đồ thị và đồ thị nâng cao:

CHUYỂN DANH SÁCH CẠNH SANG DANH SÁCH KÊ.

Cho đồ thị vô hướng G được biểu diễn dưới dạng danh sách cạnh. Hãy viết chương trình thực hiện chuyển đổi biểu diễn đồ thị dưới dạng danh sách kề.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E| + 1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh của đồ thị; $|E|$ dòng tiếp theo đưa vào các bộ đôi uV, vV tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

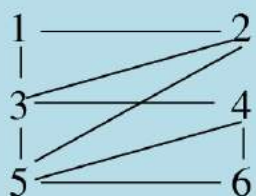
- Đưa ra danh sách kề của các đỉnh tương ứng theo khuôn dạng của ví dụ dưới đây. Các đỉnh trong danh sách in ra theo thứ tự tăng dần.

Ví dụ:

Input:	Output:
1	1: 2 3

6 9	2: 1 3 5
1 2	3: 1 2 4 5
1 3	4: 3 5 6
2 3	5: 2 3 4 6
2 5	6: 4 5
3 4	
3 5	
4 5	
4 6	
5 6	

Giải thích:



Giải :

//note :V(vertex :tập đỉnh trong toán học), E(edge :tập cạnh của hồ sâu/bìa lê sách)

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int a[1001][1001],V,E;
```

```
void initwsolve(){
```

```
    memset(a,0,sizeof(a));
```

```
    cin>>V>>E;
```

```
    for(int i=0;i<E;i++){
```

```
        int x,y;cin>>x>>y;
```

```
        a[x][y]=a[y][x]=1;// đánh dấu cạnh tồn tại được nối
```

```
    }
```

```
    for(int i=1;i<=V;i++){
```

```
        cout<<i<<" : ";
```

```
        for(int j=1;j<=V;j++){
```

```
            if(a[i][j]==1) cout<<j<<' ';
```

```
        cout<<endl;
```

```
    }
```

```
}
```

```
int main(){
```

```

int t;cin>>t;
while(t--) initwsolve();
return 0;
}

```

CHUYỂN TỪ DANH SÁCH KÊ SANG DANH SÁCH CẠNH

Ôn lại

Nội dung bài tập

Cho đơn đồ thị G vô hướng liên thông được mô tả bởi danh sách kê. Hãy in ra danh sách cạnh tương ứng của G.

Input

- Dòng đầu tiên ghi số N là số đỉnh (1
- N dòng tiếp theo mỗi dòng ghi 1 danh sách kê lần lượt theo thứ tự từ đỉnh 1 đến đỉnh N

Output: Ghi ra lần lượt từng cạnh của đồ thị theo thứ tự tăng dần.

Ví dụ

Input	Output
3	
2 3	1 2
1 3	1 3
1 2	2 3

Test danh sách kê sẽ là :

Đỉnh 1 kê với: 2 3

Đỉnh 2 kê vs: 1 3

Đỉnh 3 kê vs: 1 2

=> danh sách cạnh

Giải :

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int main(){
```

```
    int V;cin>>V;
```

```
    cin.ignore();
```

```
    int a[51][51];memset(a,0,sizeof(a));
```

```
    for(int i=1;i<=V;i++){
```

```
        string s;
```

```
        getline(cin,s);
```

```
        s+=' ';
```

```

int j=0;
for(int k=0;k<s.length();k++){
    if(s[k]==' '){
        a[i][j]=a[j][i]=1;
        j=0;
    }
    else j=j*10+s[k]-'0';
}
}
for(int i=1;i<=V;i++){
    for(int j=i+1;j<=V;j++){
        if(a[i][j]==1)cout<<i<<' '<<j<<endl;
    }
}
return 0;
}

```

CHUYỂN DANH SÁCH KÊ SANG MA TRẬN KÊ

Cho đơn đồ thị vô hướng có n đỉnh dưới dạng danh sách kê.

Hãy biểu diễn đồ thị bằng ma trận kê.

Input: Dòng đầu tiên chứa số nguyên n – số đỉnh của đồ thị ($1 \leq n \leq 1000$). n dòng tiếp theo, dòng thứ i chứa các số nguyên là các đỉnh kề với đỉnh i .

Output: Ma trận kê của đồ thị.

Ví dụ:

Input	Output
3	0 1 1
2 3	1 0 1
1 3	1 1 0
1 2	

Giải:

Cách 1:

```

#include<bits/stdc++.h>
using namespace std;
int a[1005][1005];
int main(){
    int V;cin>>V;
    cin.ignore();

```

```

for(int i=1;i<=V;i++){
    string s;
    getline(cin,s);
    s+=' ';
    int j=0;
    for(int k=0;k<s.length();k++){
        if(s[k]==' '){
            a[i][j]=a[j][i]=1;
            j=0;
        }
        else j=j*10+s[k]-'0';
    }
}
for(int i=1;i<=V;i++){
    for(int j=1;j<=V;j++){
        cout<<a[i][j]<<' ';
    }cout<<endl;
}
return 0;
}

```

Cách 2 : hay nhưng sai :

```

#include<bits/stdc++.h>
using namespace std;
int a[1005][1005];
int main(){
    int V;cin>>V;
    for(int i=1;i<=V;i++){
        while(1){
            int temp;cin>>temp;
            a[i][temp]=1;a[temp][i]=1;
            if(getchar()=='\n') break;;
        }
    }
    for(int i=1;i<=V;i++){
        for(int j=1;j<=V;j++)
            cout<<a[i][j]<<' ';
        cout<<endl;
    }
}

```

```

    return 0;
}

```

BIỂU DIỄN ĐỒ THỊ CÓ HƯỚNG.

Cho đồ thị có hướng G được biểu diễn dưới dạng danh sách cạnh. Hãy viết chương trình thực hiện chuyển đổi biểu diễn đồ thị dưới dạng danh sách kề.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E| + 1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh của đồ thị; $|E|$ dòng tiếp theo đưa vào các bộ đôi u, v tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách kề của các đỉnh tương ứng theo khuôn dạng của ví dụ dưới đây. Các đỉnh trong danh sách in ra theo thứ tự tăng dần.

Ví dụ:

Input:	Output:
1	
6 9	
1 2	
2 5	1: 2
3 1	2: 5
3 2	3: 1 2 5
3 5	4: 4
4 3	5: 4 6
5 4	6: 4
5 6	
6 4	

Giải :

```

#include<bits/stdc++.h>
using namespace std;
int a[1001][1001],v,e;
void initwsolve(){
    cin>>v>>e;
    memset(a,0,sizeof(a));
    for(int i=1;i<=e;i++){

```

```

    int x,y;cin>>x>>y;
    a[x][y]=1;
}
for(int i=1;i<=v;i++){
    cout<<i<<": ";
    for(int j=1;j<=v;j++){
        if(a[i][j]==1) cout<<j<<' ';
    }
    cout<<endl;
}
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

CHUYỂN MA TRẬN KÊ SANG DANH SÁCH KÊ

Nội dung bài tập

Ma trận kê A của một đồ thị vô hướng là một ma trận chỉ có các số 0 hoặc 1 trong đó $A[i][j] = 1$ có ý nghĩa là đỉnh i kê với đỉnh j (chỉ số tính từ 1).

Danh sách kê thì liệt kê các đỉnh kê với đỉnh đó theo thứ tự tăng dần.

Hãy chuyển biểu diễn đồ thị từ dạng ma trận kê sang dạng danh sách kê.

Input: Dòng đầu tiên chứa số nguyên n – số đỉnh của đồ thị ($1 < n \leq 1000$). n dòng tiếp theo, mỗi dòng có n số nguyên có giá trị 0 và 1 mô tả ma trận kê của đồ thị.

Output: Gồm n dòng, dòng thứ i chứa các số nguyên là đỉnh có nối với đỉnh i và được sắp xếp tăng dần. Dữ liệu đảm bảo mỗi đỉnh có kết nối với ít nhất 1 đỉnh khác.

Ví dụ:

Input	Output
3	2 3
0 1 1	1 3
1 0 1	1 2
1 1 0	

Giải :

Cách 1:

```
#include<bits/stdc++.h>
```

```

using namespace std;
int a[1001][1001],n;
int main(){
    int n;cin>>n;
    for(int i=1;i<=n;i++)
        for(int j=1;j<=n;j++)
            cin>>a[i][j];

    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            if( a[i][j]==1) cout<<j<<' ';
        }cout<<endl;
    }
    return 0;
}

```

Cách 2: dùng vector

```

#include<bits/stdc++.h>
using namespace std;
int a[1001][1001],n;
vector <vector<int> > res;
int main(){
    int n;cin>>n;
    res.resize(n+1);
    for(int i=1;i<=n;i++){
        for(int j=1;j<=n;j++){
            cin>>a[i][j];
            if( a[i][j]==1) res[i].push_back(j);
        }
    }
    for(int i=1;i<=n;i++){
        for(int j=0;j<res[i].size();j++){
            cout<<res[i][j]<<' ';
        }cout<<endl;
    }
    return 0;
}

```

DFS TRÊN ĐỒ THỊ VÔ HƯỚNG

Ôn lại

Cho đồ thị vô hướng $G =$ được biểu diễn dưới dạng danh sách cạnh. Hãy viết thuật toán duyệt theo chiều sâu bắt đầu tại đỉnh u (DFS(u)=?)

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E| + 1$ dòng: dòng đầu tiên đưa vào ba số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh của đồ thị, và u là đỉnh xuất phát; $|E|$ dòng tiếp theo đưa vào các bộ đôi uV , vV tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh được duyệt theo thuật toán DFS(u) của mỗi test theo khuôn dạng của ví dụ dưới đây.

Ví dụ:

Input:	Output:
1 6 9 5 1 2 1 3 2 3 2 4 3 4 3 5 4 5 4 6 5 6	5 3 1 2 4 6



Giải :

Casch 1 :quay lui

// Depth-first search - DFS

// note : bài này không nên dùng mảng $a[1001][1001]$ sau...cho $a[x][y]=1..$ mà phải dùng vector để push_back từng đỉnh một vì để tối ưu hóa dfs , ta chỉ đưa

những số cần đưa vào , nếu dùng mảng thì có quá nhiều số 0 => cách khai báo vector 2 chiều cũng khác những bài thông thường

// 1: duyệt số cạnh của đồ thị và đánh dấu từng đỉnh xem nó được nối với các đỉnh nào

// 2: dfs(tìm kiếm sau bắt đầu từ đỉnh u)

// 3: duyệt các đỉnh kề được nối với đỉnh u (v) nếu đỉnh kề đó chưa được tính (check[v]==0) thì chấp nhận và tiếp tục tìm đỉnh kề tiếp theo.

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,u,check[1005];
```

```
vector <int> a[1005];
```

```
void dfs(int u){ //2
```

```
    cout<<u<<' ';
```

```
    check[u]=1;
```

```
    /*3*/ for(int i=0;i<a[u].size();i++){
```

```
        int v=a[u][i];
```

```
        if(check[v]==0) dfs(v);
```

```
    }
```

```
}
```

```
void initwsolve(){
```

```
    cin>>V>>E>>u;
```

```
    memset(a,0,sizeof(a));
```

```
    memset(check,0,sizeof(check));
```

```
    /*1*/ for(int i=1;i<=E;i++){
```

```
        int x,y;cin>>x>>y;
```

```
        a[x].push_back(y);
```

```
        a[y].push_back(x);
```

```
    }
```

```
    dfs(u);
```

```
    cout<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    while(t-->0) initwsolve();
```

```
}
```

Cách 2: stack:

```
#include<bits/stdc++.h>
```

```

using namespace std;
int V,E,u,check[1005];
vector <int> a[1005];
void dfs(int u){//2
    stack<int> nx;
    nx.push(u);
    cout<<u<<' ';
    check[u]=1;
    while(nx.size()){
        int s=nx.top();nx.pop();
        for(int i=0;i<a[s].size();i++){
            int t=a[s][i];
            if(check[t]==0){
                cout<<t<<' ';
                check[t]=1;
                nx.push(s);
                nx.push(t);
                break;
            }
        }
    }
}
void initwsolve(){
    cin>>V>>E>>u;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    /*1*/ for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    dfs(u);
    cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

```
}
```

Cách 3 : quay lui dùng ma trận kề vẫn đúng : dùng cho TRR2

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,u,check[1005];
```

```
int a[1005][1005];
```

```
void dfs(int u){
```

```
    cout<<u<<' ';
```

```
    check[u]= 1;
```

```
    for(int v=1;v<=V;v++)
```

```
        if(a[u][v]==1&&check[v]==0) dfs(v);
```

```
}
```

```
void initwsolve(){
```

```
    cin>>V>>E>>u;
```

```
    memset(a,0,sizeof(a));
```

```
    memset(check,0,sizeof(check));
```

```
/*1*/ for(int i=1;i<=E;i++){
```

```
    int x,y;cin>>x>>y;
```

```
    a[x][y]=1;
```

```
    a[y][x]=1;
```

```
}
```

```
dfs(u);
```

```
cout<<endl;
```

```
}
```

```
int main(){
```

```
    int t;cin>>t;
```

```
    while(t--) initwsolve();
```

```
}
```

Cách 4 : stack dùng ma trận kề vẫn đúng dùng cho TRR2

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,u,check[1005];
```

```
int a[1005][1005];
```

```
void dfs(int u){
```

```
    stack<int> nganxep;
```

```
    cout<<u<<' ';
```

```

check[u]=1;
nganxep.push(u);
while(!nganxep.empty()){
    int s=nganxep.top();
    nganxep.pop();
    for(int t=1;t<=V;t++){
        if(a[s][t]==1&&check[t]==0){
            cout<<t<<' ';
            check[t]=1;
            nganxep.push(s);
            nganxep.push(t);
            break;
        }
    }
}
}
void initwsolve(){
    cin>>V>>E>>u;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    /*1*/ for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x][y]=1;
        a[y][x]=1;
    }
    dfs(u);
    cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

BFS TRÊN ĐỒ THỊ VÔ HƯỚNG

Ôn lại

Cho đồ thị vô hướng $G =$ được biểu diễn dưới dạng danh sách cạnh. Hãy viết thuật toán duyệt theo chiều rộng bắt đầu tại đỉnh u (BFS(u)=?)

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào ba số $|V|$, $|E|$, uV tương ứng với số đỉnh, số cạnh và đỉnh bắt đầu duyệt; Dòng tiếp theo đưa vào các bộ đôi uV, vV tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh được duyệt theo thuật toán BFS(u) của mỗi test theo khuôn dạng của ví dụ dưới đây.

Ví dụ:

Input:	Output:
1	
6 9 1	1 2 3 5 4 6
1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	

Giải :

// Breadth First Search : thuật toán duyệt theo chiều rộng

// note : bài này không nên dùng mảng $a[1001][1001]$ sau...cho $a[x][y]=1$.. mà phải dùng vector để push_back từng đỉnh một vì để tối ưu hóa dfs, ta chỉ đưa những số cần đưa vào, nếu dùng mảng thì có quá nhiều số 0 => cách khai báo vector 2 chiều cũng khác những bài thông thường

// 1: duyệt số cạnh của đồ thị và đánh dấu từng đỉnh xem nó được nối với các đỉnh nào

// 2: bfs(tìm kiếm sau bắt đầu từ đỉnh u)

// 3 : giải thích cách tìm:

//lần 1: s=1, t thỏa mãn=2,3, check[2]=1; check[3]=1;

//lần 2: s=2, t thỏa mãn =5, check[5]=1;

//lần 3: s=3, t thỏa mãn =4, check[4]=1;

//lần 4 :s=5, t thỏa mãn =6, check[6]=1;

//lần 5 :s=4 in 4. lần 6 :s=6 in 6

#include<bits/stdc++.h>

using namespace std;

int V,E,u,check[1005];

vector<int> a[1005];

void bfs(int u){//2

queue<int> q;

q.push(u);check[u]=1;

while(q.empty()==0){//3

```

    int s=q.front();q.pop();
    cout<<s<<' ';
    for(int i=0;i<a[s].size();i++){
        int t=a[s][i];
        if(check[t]==0){
            check[t]=1;
            q.push(t);
        }
    }
}
}
void initwsolve(){
    cin>>V>>E>>u;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    /*1*/ for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    bfs(u);
    cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t--) initwsolve();
}

```

TÌM ĐƯỜNG ĐI THEO DFS VỚI ĐỒ THỊ VÔ HƯỚNG

Ôn lại

Cho đồ thị vô hướng $G =$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm đường đi từ đỉnh sIV đến đỉnh tIV trên đồ thị bằng thuật toán DFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào bốn số $|V|$, $|E|$, sIV, tIV tương ứng với số đỉnh, số cạnh, đỉnh u, đỉnh v; Dòng tiếp theo đưa vào các bộ đôi uIV, vIV tương ứng với một cạnh của đồ thị.

- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra đường đi từ đỉnh s đến đỉnh t của mỗi test theo thuật toán DFS của mỗi test theo khuôn dạng của ví dụ dưới đây. Nếu không có đáp án, in ra -1.

Ví dụ:

Input:	Output:
1 6 9 1 6 1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	1 2 3 4 5 6

Ví dụ test khác :

```
1
6 9 1 6
1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 2
=>1 2 3 4 6 (vì 5 không nối với 6)
```

Giải :

// 1: duyệt số cạnh của đồ thị và đánh dấu từng đỉnh xem nó được nối với các đỉnh nào

// 2: dfs (tìm kiếm sau bắt đầu từ đỉnh u)

// 3: duyệt dfs như bài đồ thị vô hướng, khác chỗ đánh dấu giá trị phần tử có chỉ số là đỉnh sau trong mảng chính = đỉnh trước

// 4: in ra mảng res thích hợp theo thứ tự tăng dần (lấy dữ liệu từ mảng b)

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,s,t,check[1005],b[1005]; //b: trước s=> = b[t];
```

```
vector<int> a[1005];
```

```
void dfs(int u){ //2
```

```
    check[u]=1;
```

```
    /*3*/ for(int i=0;i<a[u].size();i++){
```

```
        int v=a[u][i];
```

```
        if(check[v]==0) {
```

```
            b[v]=u;
```

```
            dfs(v);
```

```
        }
```

```
    }
```

```
}
```

```
void initwsolve(){
```

```
    cin>>V>>E>>s>>t;
```

```

    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    /*1*/ for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    dfs(s);
    if(check[t]==0) cout<<-1<<endl;
    /*4*/ else {
        vector<int> res;
        int i=t;cout<<s<<' ';
        while(i!=s){res.push_back(i); i=b[i];}
        for(int i=res.size()-1;i>=0;i--) cout<<res[i]<<' ';
        cout<<endl;
    }
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

Cách 2 : ma trận kề

```

#include<bits/stdc++.h>
using namespace std;
int V,E,s,t,check[1005],b[1005];//b: truooc s=> = b[t];
int a[1005][1005];
void dfs(int u){//2
    check[u]=1;
    /*3*/ for(int v=1;v<=V;v++){
        if(a[u][v]==1&&check[v]==0) {
            b[v]=u;
            dfs(v);
        }
    }
}
void initwsolve(){
    cin>>V>>E>>s>>t;
}

```

```

memset(a,0,sizeof(a));
memset(check,0,sizeof(check));
/*1*/ for(int i=1;i<=E;i++){
    int x,y;cin>>x>>y;
    a[x][y]=1;
    a[y][x]=1;
}
dfs(s);
if(check[t]==0) cout<<-1<<endl;
/*4*/ else {
    vector<int> res;
    int i=t;cout<<s<<' ';
    while(i!=s){res.push_back(i); i=b[i];}
    for(int i=res.size()-1;i>=0;i--) cout<<res[i]<<' ';
    cout<<endl;
}
}
int main(){
    int t;cin>>t;
    while(t--) initwsolve();
}

```

ĐƯỜNG ĐI THEO BFS TRÊN ĐỒ THỊ VÔ HƯỚNG

Ôn tập

Cho đồ thị vô hướng G được biểu diễn dưới dạng danh sách cạnh. Hãy tìm đường đi từ đỉnh s1V đến đỉnh t1V trên đồ thị bằng thuật toán BFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào bốn số $|V|$, $|E|$, s1V, t1V tương ứng với số đỉnh, số cạnh, đỉnh u , đỉnh v ; Dòng tiếp theo đưa vào các bộ đôi u1V, v1V tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra đường đi từ đỉnh s đến đỉnh t của mỗi test theo thuật toán BFS của mỗi test theo khuôn dạng của ví dụ dưới đây. Nếu không có đáp án, in ra -1.

Ví dụ:

Input:	Output:
--------	---------

1 6 9 1 6 1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	1 2 5 6
---	---------

Giải :

// Breadth First Search : thuật toán duyệt theo chiều rộng

// note : bài này không nên dùng mảng a[1001][1001] sau...cho a[x][y]=1.. mà phải dùng vector để push_back từng đỉnh một vì để tối ưu hóa bfs , ta chỉ đưa những số cần đưa vào , nếu dùng mảng thì có quá nhiều số 0 => cách khai báo vector 2 chiều cũng khác những bài thông thường

// 1:duyệt số cạnh của đồ thị và đánh dấu từng đỉnh xem nó được nối với các đỉnh nào

// 2:bfs(tìm kiếm sau bắt đầu từ đỉnh s)

// 3 : giải thích cách tìm:

//lần 1: s=1 , t thỏa mãn=2,3 , check[2]=1;check[3]=1;

//lần 2: s=2 , t thỏa mãn =5 , check[5]=1;

//lần 3: s=3 , t thỏa mãn =4 , check[4]=1;

//lần 4 :s=5 ,t thỏa mãn =6 ,check[6]=1;

//lần 5 :s=4 . lần 6 :s=6 không có ý nghĩa

//4 :in ra mảng res thích hợp theo thứ tự tăng dần (lấy dữ liệu từ mảng b)

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,s,t,check[1005],b[1005];
```

```
vector <int> a[1005];
```

```
void bfs(int u){//2
```

```
    queue <int> q;
```

```
    q.push(u);check[u]=1;
```

```
    while(q.empty()==0){//3
```

```
        int s=q.front();q.pop();
```

```
        for(int i=0;i<a[s].size();i++){
```

```
            int t=a[s][i];
```

```
            if(check[t]==0){
```

```
                check[t]=1;
```

```
                q.push(t);
```

```
                b[t]=s;//b[đỉnh sau]=đỉnh trước
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```

void initwsolve(){
    cin>>V>>E>>s>>t;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    /*1*/ for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    bfs(s);
    if(check[t]==0) cout<<-1<<endl;
    /*4*/ else {
        vector <int> res;
        int i=t;cout<<s<<' ';
        while(i!=s){res.push_back(i); i=b[i];}
        for(int i=res.size()-1;i>=0;i--) cout<<res[i]<<' ';
        cout<<endl;
    }
}

int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

KIỂM TRA ĐƯỜNG ĐI

Ôn tập

Nội dung bài tập

Cho đồ thị vô hướng có N đỉnh và M cạnh. Có Q truy vấn, mỗi truy vấn yêu cầu trả lời câu hỏi giữa 2 đỉnh x và y có tồn tại đường đi tới nhau hay không?

Input:

- Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).
- Mỗi test gồm 2 số nguyên N, M ($1 \leq N, M \leq 1000$).
- M dòng tiếp theo, mỗi dòng gồm 2 số nguyên u, v cho biết có cạnh nối giữa đỉnh u và v .
- Dòng tiếp là số lượng truy vấn Q ($1 \leq Q \leq 1000$).
- Q dòng tiếp theo, mỗi dòng gồm 2 số nguyên x và y .

Output: Với mỗi truy vấn, in ra “YES” nếu có đường đi từ x tới y, in ra “NO” nếu ngược lại.

Ví dụ:

Input:	Output
1	
5 5	
1 2	
2 3	
3 4	NO
1 4	YES
5 6	
2	
1 5	
2 4	

Giải

//1:tại từng bộ q ta tìm đường đi theo dfs với đồ thị vô hướng xuất phát từ đỉnh thứ nhất

//2:xét đường đi đó nếu đi qua đỉnh thứ 2 thì ok Q đúng in YES không thì in NO

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int v,e,u,U,check[1005];
```

```
vector <int> a[1005];
```

```
void dfs(int u){//xuất phát từ đỉnh u
```

```
    check[u]=1;
```

```
    for(int i=0;i<a[u].size();i++){
```

```
        int U=a[u][i];
```

```
        if(check[U]==0) {
```

```
            dfs(U);
```

```
        }
```

```
    }
```

```
}
```

```
void initwsolve(){
```

```
    cin>>v>>e;
```

```
    memset(a,0,sizeof(a));
```

```
/*init*/ for(int i=1;i<=e;i++){
```

```
    int u,v;cin>>u>>v;
```

```

        a[u].push_back(v);
        a[v].push_back(u);
    }
    int q;cin>>q;
    /*1*/ for(int i=1;i<=q;i++){
        int u,v;cin>>u>>v;
        memset(check,0,sizeof(check));
        dfs(u); //1
        /*2*/if(check[v]==0) cout<<"NO"<<endl;
        else cout<<"YES"<<endl;
    }
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

ĐẾM SỐ THÀNH PHẦN LIÊN THÔNG (THEO DFS)

Nội dung bài tập

Cho đồ thị vô hướng $G =$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm số thành phần liên thông của đồ thị..

Input:

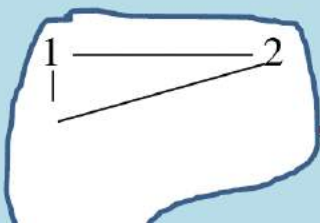
- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh; Dòng tiếp theo đưa vào các bộ đôi u, v tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2;$

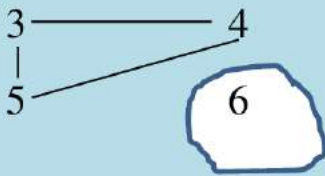
Output:

- Đưa ra số thành phần liên thông của đồ thị bằng thuật toán DFS.

Ví dụ:

Input:	Output:
1 6 6 1 2 1 3 2 3 3 4 3 5 4 5	2





Giải :

// note : Một đồ thị G được gọi là liên thông nếu có dây chuyền giữa mọi cặp đỉnh phân biệt đồ thị G. (tức là chúng nối được với nhau thành một đường)

// 2 đếm số thành phần liên thông bằng cách cho duyệt từng đỉnh , nếu như sau mỗi lần duyệt mà tại đỉnh đó chưa có trong các thành phần trước đó thì res++; tức là phải nằm ở thành phần liên thông khác.

// 3: chỉ là tìm kiếm đường đi dfs

```

#include<bits/stdc++.h>
#include<bits/stdc++.h>
using namespace std;
int V,E,check[1005];
vector<int> a[1005];
void dfs(int u){ //3
    check[u]=1;
    for(int i=0;i<a[u].size();i++){
        int v=a[u][i];
        if(check[v]==0) dfs(v);
    }
}
void initwsolve(){
    /*1init*/int res=0;
    cin>>V>>E;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x].push_back(y);a[y].push_back(x);
    }
    /*2*/ for(int i=1;i<=V;i++){
        if(check[i]==0){
            dfs(i);res++;
        }
    }cout<<res<<endl;
  
```

```

}
int main(){
    int t;cin>>t;
    while(t--) initwsolve();
}

```

Cách 2 : ma trận kề

```

#include<bits/stdc++.h>
#include<bits/stdc++.h>
using namespace std;
int V,E,check[1005];
int a[1005][1005];
void dfs(int u){
    check[u]= 1;
    for(int v=1;v<=V;v++)
        if(a[u][v]==1&&check[v]==0) dfs(v);
}
void initwsolve(){
    /*1init*/int res=0;
    cin>>V>>E;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x][y]=1;
        a[y][x]=1;
    }
    /*2*/ for(int i=1;i<=V;i++){
        if(check[i]==0){
            dfs(i);res++;
        }
    }cout<<res<<endl;
}
int main(){
    int t;cin>>t;
    while(t--) initwsolve();
}

```

TÌM SỐ THÀNH PHẦN LIÊN THÔNG VỚI BFS

Nội dung bài tập

Cho đồ thị vô hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm số thành phần liên thông của đồ thị bằng thuật toán BFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh; Dòng tiếp theo đưa vào các bộ đôi $u\ v$, $v\ u$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

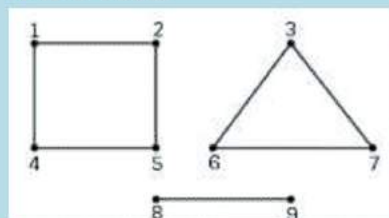
Output:

- Đưa ra số thành phần liên thông của đồ thị bằng thuật toán BFS.

Ví dụ:

Input:	Output:
1 6 6 1 2 1 3 2 3 3 4 3 5 4 5	2

Vd 1 test:



Số thành phần liên thông	Kết quả duyệt BFS	Giá trị trong mảng chuaxet[]
0	Chưa thực hiện	Chuaxet[]={0,0,0,0,0,0,0,0,0}
1	BFS(1): 1,2,4,5	Chuaxet[]={1,1,0,1,1,0,0,0,0}
2	BFS(3): 3,6,7	Chuaxet[]={1,1,1,1,1,1,1,0,0}
3	BFS(8): 8,9	Chuaxet[]={1,1,1,1,1,1,1,1,1}

Giải :

// note :Một đồ thị G được gọi là liên thông nếu có dây chuyền giữa mọi cặp đỉnh phân biệt đồ thị G.(tức là chúng nối được với nhau thành một đường)

//2 đếm số thành phần liên thông bằng cách cho duyệt từng đỉnh , nếu như sau mỗi lần duyệt mà tại đỉnh đó chưa có trong các thành phần trước đó thì res++; tức là phải nằm ở thành phần liên thông khác.

//3: chỉ là tìm kiếm đường đi bfs

```
#include<bits/stdc++.h>
#include<bits/stdc++.h>
using namespace std;
int V,E,check[1005];
vector<int> a[1005];
void bfs(int u){//3
    queue <int> q;
    q.push(u);check[u]=1;
    while(q.empty()==0){
        int s=q.front();q.pop();
        for(int i=0;i<a[s].size();i++){
            int t=a[s][i];
            if(check[t]==0){
                check[t]=1;
                q.push(t);
            }
        }
    }
}
void initwsolve(){
/*1init*/int res=0;
    cin>>V>>E;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x].push_back(y);a[y].push_back(x);
    }
/*2*/ for(int i=1;i<=V;i++){
    if(check[i]==0){//2
        bfs(i);res++;
    }
}cout<<res<<endl;
}
```

```

int main(){
    int t;cin>>t;
    while(t--) initwsolve();
}

```

Cách 2: ma trận kề

```

#include<bits/stdc++.h>
#include<bits/stdc++.h>
using namespace std;
int V,E,check[1005];
int a[1005][1005];
void bfs(int u){
    queue<int> q;
    check[u]=1;
    q.push(u);
    while(q.size()){
        int s=q.front();
        q.pop();
        for(int t=1;t<=V;t++){
            if(a[s][t]==1&&check[t]==0){
                check[t]=1;
                q.push(t);
            }
        }
    }
}

```

```

void initwsolve(){
/*1init*/int res=0;
    cin>>V>>E;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x][y]=1;
        a[y][x]=1;
    }
}

```

```

/*2*/ for(int i=1;i<=V;i++){
        if(check[i]==0){
            bfs(i);res++;
        }
    }cout<<res<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

KIỂM TRA TÍNH LIÊN THÔNG MẠNH

Nội dung bài tập

Cho đồ thị có hướng $G = (V, E)$ được biểu diễn dưới dạng danh sách cạnh. Hãy kiểm tra xem đồ thị có liên thông mạnh hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh; Dòng tiếp theo đưa vào các bộ đôi u, v tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra “YES”, hoặc “NO” theo từng dòng tương ứng với test là liên thông mạnh hoặc không liên thông mạnh.

Ví dụ:

Input:	Output:
1 6 9 1 2 2 4 3 1 3 2 3 5 4 3 5 4 5 6 6 3	YES

Giải : 1 2 , 2 4 , 3 1 , 3 2 , 3 5 , 4 3 , 5 4 , 5 6 , 6 3.



Cách 1 : dama trận kẻ tr

```
#include<bits/stdc++.h>
using namespace std;
int V,E,check[1002];
int a[1002][1002];
void dfs(int u){
    check[u]= 1;
    for(int v=1;v<=V;v++)
        if(a[u][v]==1&&check[v]==0) dfs(v);
}
void initwsolve(){
    /*init*/cin>>V>>E;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x][y]=1;
    }
    /*2*/ for(int i=1;i<=V;i++){
        dfs(i);
        for(int j=1;j<=V;j++){
            if(check[j]==0){
                cout<<"NO"<<endl; return;
            }
        }
        memset(check,0,sizeof(check));
    } cout<<"YES"<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}
```

Cách 2: ds kẻ

// note :Liên thông mạnh (strongly connected): Đồ thị có hướng gọi là liên thông mạnh nếu có đường đi từ a tới b và từ b tới a với mọi cặp đỉnh a và b của đồ thị. Xem thêm thành phần liên thông mạnh.

// 2: duyệt bắt đầu từng đỉnh một ,xem là sau khi tìm kiếm dfs thì tất các các đỉnh có thuộc cùng 1 liên thông hay không , chỉ cần 1 đỉnh thuộc liên thông khác thì => liên thông yếu hoặc liên thông 1 phần

```

//3: chỉ là tìm kiếm đường đi dfs
#include<bits/stdc++.h>
using namespace std;
int v,e,u,res,check[1005];
vector <int> a[1005];
void dfs(int u){ //3
    check[u]=1;
    for(int i=0;i<a[u].size();i++){
        int U=a[u][i];
        if(check[U]==0) dfs(U);
    }
}
void initwsolve(){
    /*init*/cin>>v>>e;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=e;i++){
        int u,v;cin>>u>>v;
        a[u].push_back(v);
    }
    /*2*/ for(int i=1;i<=v;i++){
        dfs(i);
        for(int j=1;j<=v;j++){
            if(check[j]==0){
                cout<<"NO"<<endl; return;
            }
        }
        memset(check,0,sizeof(check));
    } cout<<"YES"<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

KIỂM TRA TÍNH LIÊN THÔNG MẠNH VỚI BFS b buộc ds kề

Nội dung bài tập

Cho đồ thị có hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy kiểm tra xem đồ thị có liên thông mạnh hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh; Dòng tiếp theo đưa vào các bộ đôi uV , vV tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra “YES”, hoặc “NO” theo từng dòng tương ứng với test là liên thông mạnh hoặc không liên thông mạnh.

Ví dụ:

Input:	Output:
1 6 9 1 2 2 4 3 1 3 2 3 5 4 3 5 4 5 6 6 3	YES

Chú ý : nộp bài dfs vẫn đúng.

Giải :

// note :Liên thông mạnh (strongly connected): Đồ thị có hướng gọi là liên thông mạnh nếu có đường đi từ a tới b và từ b tới a với mọi cặp đỉnh a và b của đồ thị.

Xem thêm thành phần liên thông mạnh.

// 2: duyệt bắt đầu từ từng đỉnh một ,xem là sau khi tìm kiếm bfs thì tất các các đỉnh có thuộc cùng 1 liên thông hay không , chỉ cần 1 đỉnh thuộc liên thông khác thì => liên thông yếu hoặc liên thông 1 phần

//3: chỉ là tìm kiếm đường đi bfs

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int v,e,u,res,check[1005];
```

```
vector <int> a[1005];
```

```
void bfs(int u){ //3
```

```
    queue <int> q;
```

```
    q.push(u); check[u]=1;
```

```
    while(q.empty()==0){
```

```
        int x=q.front();q.pop();
```

```
        for(int i=0;i<a[x].size();i++){
```

```
            int U=a[x][i];
```

```
            if(check[U]==0){
```

```
                check[U]=1;
```

```
                q.push(U);
```

```

    }
    }
}

void initwsolve(){
/*init*/cin>>v>>e;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=e;i++){
        int u,v;cin>>u>>v;
        a[u].push_back(v);
    }
/*2*/ for(int i=1;i<=v;i++){
    bfs(i);
    for(int j=1;j<=v;j++){
        if(check[j]==0){
            cout<<"NO"<<endl; return;
        }
    }
    memset(check,0,sizeof(check));
} cout<<"YES"<<endl;
}

int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

LIỆT KÊ ĐỈNH TRỤ (với dfs)

Nội dung bài tập

Cho đồ thị vô hướng liên thông $G = (V, E)$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS hoặc BFS, hãy đưa ra tất cả các đỉnh trụ của đồ thị?

Input:

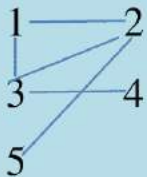
- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh; Dòng tiếp theo đưa vào các bộ đôi u, v tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh trụ của mỗi test theo từng dòng.

Ví dụ:

Input:	Output:
1 5 5 1 2 1 3 2 3 2 5 3 4	2 3



Giải :

// note: (Đỉnh trụ của đồ thị là đỉnh mà khi xóa nó đi thì số thành phần liên thông của đồ thị tăng lên)

//2: Trước hết ta duyệt dfs 1 lần đồ thị để tìm ra số thành phần liên thông ban đầu .

//3: Mỗi lần duyệt ta xóa 1 đỉnh của đồ thị. Dùng dfs duyệt tìm số thành phần liên thông của đồ thị. Nếu số thành phần liên thông của đồ thị tăng lên thì đỉnh bị xóa là đỉnh trụ.

//4: chỉ là tìm kiếm đường đi dfs

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int v,e,u,check[1005];
```

```
vector <int> a[1005];
```

```
void dfs(int u){ //4
```

```
    check[u]=1;
```

```
    for(int i=0;i<a[u].size();i++){
```

```
        int U=a[u][i];
```

```
        if(check[U]==0) dfs(U);
```

```
    }
```

```
}
```

```
void initwsolve(){
```

```
/*init*/int inva=0,res=0;;//invariable :cố định, bất biến
```

```
    cin>>v>>e;
```

```
    memset(a,0,sizeof(a));
```

```
    memset(check,0,sizeof(check));
```

```
    for(int i=1;i<=e;i++){
```

```
        int u,v;cin>>u>>v;
```

```
        a[u].push_back(v);
```

```

        a[v].push_back(u);
    }
    /*2*/ for(int i=1;i<=v;i++){
        if(check[i]==0){
            dfs(i);inva++;}
    }
    /*3*/ for(int i=1;i<=v;i++){
        memset(check,0,sizeof(check));
        check[i]=1;res=0;
        for(int j=1;j<=v;j++){
            if(check[j]==0){ dfs(j);res++;}
        }
        if(res>inva) cout<<i<<' ';
    }
}
int main(){
    int t;cin>>t;
    while(t--) initwsolve();
}

```

Cách 2 : ma trận kề trr:

```

#include<bits/stdc++.h>
using namespace std;
int V,E,check[1005];//b: truooc s=> = b[t];
int a[1005][1005];
void dfs(int u){
    check[u]= 1;
    for(int v=1;v<=V;v++)
        if(a[u][v]==1&&check[v]==0) dfs(v);
}

```

```

void initwsolve(){
    cin>>V>>E;
    int inva=0,res=0; //invariable :cố định, bất biến
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    /*1*/ for(int i=1;i<=E;i++){

```

```

    int x,y;cin>>x>>y;
    a[x][y]=1;
    a[y][x]=1;
}
for(int i=1;i<=V;i++){
    if(check[i]==0){
        dfs(i);inva++;}
}
for(int i=1;i<=V;i++){
    memset(check,0,sizeof(check));
    check[i]=1;res=0;
    for(int j=1;j<=V;j++){
        if(check[j]==0){dfs(j);res++;}
    }
    if(res>inva) cout<<i<<' ';
}
cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

Cách 3 : nhưng chú ý bài toán coi như đồ thị này đã liên thông => chỉ cần so sánh >1 hay chưa vẫn đúng:

```

#include<bits/stdc++.h>
using namespace std;
int V,E,check[1005];//b: trước s=> = b[t];
int a[1005][1005];
void dfs(int u){
    check[u]= 1;
    for(int v=1;v<=V;v++)
        if(a[u][v]==1&&check[v]==0) dfs(v);
}

```

```

void initwsolve(){
    cin>>V>>E;
    int inva=0,res=0; //invariable :cố định, bất biến
}

```

```

memset(a,0,sizeof(a));
memset(check,0,sizeof(check));
/*1*/ for(int i=1;i<=E;i++){
    int x,y;cin>>x>>y;
    a[x][y]=1;
    a[y][x]=1;
}
for(int i=1;i<=V;i++){
    memset(check,0,sizeof(check));
    check[i]=1;res=0;
    for(int j=1;j<=V;j++){
        if(check[j]==0){dfs(j);res++;}
    }
    if(res>1) cout<<i<<' ';
}
cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t--){initwsolve();
}

```

LIỆT KÊ ĐỈNH TRỤ VỚI BFS

Nội dung bài tập

Cho đồ thị vô hướng liên thông $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy đưa ra tất cả các đỉnh trụ của đồ thị?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh; Dòng tiếp theo đưa vào các bộ đôi uV, vV tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh trụ của mỗi test theo từng dòng.

Ví dụ:

Input:	Output:
1 5 5	2 3

1 2 1 3 2 3 2 5 3 4	
---------------------	--

Giải :

Cách 1 : ma trận kề:

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,check[1005];//b: truoc s=> = b[t];
```

```
int a[1005][1005];
```

```
void bfs(int u){
```

```
    queue<int> q;
```

```
    check[u]=1;
```

```
    q.push(u);
```

```
    while(q.size()){
```

```
        int s=q.front();
```

```
        q.pop();
```

```
        for(int t=1;t<=V;t++){
```

```
            if(a[s][t]==1&&check[t]==0){
```

```
                check[t]=1;
```

```
                q.push(t);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
void initwsolve(){
```

```
    cin>>V>>E;
```

```
    int res=0;
```

```
    memset(a,0,sizeof(a));
```

```
    memset(check,0,sizeof(check));
```

```
/*1*/ for(int i=1;i<=E;i++){
```

```
    int x,y;cin>>x>>y;
```

```
    a[x][y]=1;
```

```
    a[y][x]=1;
```

```
}
```

```
for(int i=1;i<=V;i++){
```

```
    memset(check,0,sizeof(check));
```

```
    check[i]=1;res=0;
```

```
    for(int j=1;j<=V;j++){
```

```

        if(check[j]==0){bfs(j);res++;}
    }
    if(res>1) cout<<i<<' ';
}
cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

Cách 2 : danh sách cạnh

// note: (Đỉnh trụ của đồ thị là đỉnh mà khi xóa nó đi thì số thành phần liên thông của đồ thị tăng lên)

//2: Trước hết ta duyệt dfs 1 lần đồ thị để tìm ra số thành phần liên thông ban đầu .

//3: Mỗi lần duyệt ta xóa 1 đỉnh của đồ thị. Dùng bfs duyệt tìm số thành phần liên thông của đồ thị. Nếu số thành phần liên thông của đồ thị tăng lên thì đỉnh bị xóa là đỉnh trụ.

//4: chỉ là đánh dấu tìm kiếm đường đi bfs

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int v,e,u,check[1005];
```

```
vector <int> a[1005];
```

```
void bfs(int u){//3
```

```
    queue <int> q;
```

```
    q.push(u); check[u]=1;
```

```
    while(q.empty()==0){
```

```
        int x=q.front();q.pop();
```

```
        for(int i=0;i<a[x].size();i++){
```

```
            int U=a[x][i];
```

```
            if(check[U]==0){
```

```
                check[U]=1;
```

```
                q.push(U);
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
void initwsolve(){
```

```
/*init*/int inva=0,res=0; //invariable :cố định, bất biến
```

```

cin>>v>>e;
memset(a,0,sizeof(a));
memset(check,0,sizeof(check));
for(int i=1;i<=e;i++){
    int u,v;cin>>u>>v;
    a[u].push_back(v);
    a[v].push_back(u);
}
/*2*/ for(int i=1;i<=v;i++){
    if(check[i]==0){
        bfs(i);inva++;}
}
memset(check,0,sizeof(check));
/*3*/ for(int i=1;i<=v;i++){
    check[i]=1;res=0;
    for(int j=1;j<=v;j++){
        if(check[j]==0){bfs(j);res++;}
    }
    if(res>inva) cout<<i<<' ';
    memset(check,0,sizeof(check));
}
}
int main(){
    int t;cin>>t;
    while(t-- ) initwsolve();
}

```

LIỆT KÊ CẠNH CẦU

Ôn lại

Cho đồ thị vô hướng liên thông $G =$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS hoặc BFS, hãy đưa ra tất cả các cạnh cầu của đồ thị?

Input:

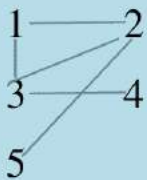
- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh; Dòng tiếp theo đưa vào các bộ đôi uV , vV tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các cách cầu của mỗi test theo từng dòng. In ra đáp án theo thứ tự từ điển, theo dạng “a b ...” với $a < b$.

Ví dụ:

Input:	Output:
1 5 5 1 2 1 3 2 3 2 5 3 4	2 5 3 4



Giải :

Cách 1 : ma trận kề:TLE

```

#include<bits/stdc++.h>
#include<bits/stdc++.h>
#define ii pair<int,int>
using namespace std;
int V,E,check[1001];
int a[1001][1001];
vector<pair<int,int> >canh;
void dfs(int u){
    check[u]= 1;
    for(int v=1;v<=V;v++)
        if(a[u][v]==1&&check[v]==0) dfs(v);
}
void initwsolve(){
    cin>>V>>E;
    canh.clear();
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x][y]=1;
        a[y][x]=1;
    }
}

```

```

        canh.push_back(ii(x,y) );
    }
    for(int i=0;i<E;i++){
        a[canh[i].first][canh[i].second]=0;
        int res=0;
        memset(check,0,sizeof(check));
        for(int k=1;k<=V;k++){
            if(check[k]==0)res++;dfs(k);
        }
        if(res>1)
            cout<<canh[i].first<<' '<<canh[i].second<<' ';
        a[canh[i].first][canh[i].second]=1;
    }
    cout<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

Cách 2 :danh sách kề:

// (cạnh cầu của đồ thị là khi xóa cạnh đó đi thì số thành phần liên thông của đồ thị tăng lên)

// 1: Trước hết ta duyệt DFS 1 lần đồ thị để tìm ra số thành phần liên thông ban đầu.

// 2:Ta xóa lần lượt các cạnh của đồ thị, sau đó dùng DFS duyệt tìm số thành phần liên thông của đồ thị. Nếu số thành phần liên thông tăng chứng tỏ cạnh bị xóa là cạnh cầu.

// 3: chỉ là duyệt đường đi theo dfs.

// (cạnh cầu của đồ thị là khi xóa cạnh đó đi thì số thành phần liên thông của đồ thị tăng lên)

```
#include<bits/stdc++.h>
```

```
#include<bits/stdc++.h>
```

```
#define ii pair<int,int>
```

```
using namespace std;
```

```
int V,E,check[1001];
```

```
vector<int > a[1005];
```

```
vector<pair<int,int> >canh;
```

```

void init(int x){
    memset(check,0,sizeof(check));
    for(int i=1;i<=V;i++)a[i].clear();
    for(int i=0;i<canh.size();i++){
        if(i!=x){
            a[canh[i].first].push_back(canh[i].second);
            a[canh[i].second].push_back(canh[i].first);
        }
    }
}

void dfs(int u){
    check[u]= 1;
    for(int i=0;i<a[u].size();i++){
        int v=a[u][i];
        if(check[v]==0) dfs(v);
    }
}

void initwsolve(){
    cin>>V>>E;
    canh.clear();
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        canh.push_back(ii(x,y) );
    }
    for(int i=0;i<E;i++){
        init(i);
        int res=0;
        memset(check,0,sizeof(check));
        for(int k=1;k<=V;k++){
            if(check[k]==0)res++;dfs(k);
        }
        if(res>1)
            cout<<canh[i].first<<' '<<canh[i].second<<' ';
    }
    cout<<endl;
}

```

```

}
int main(){
    int t;cin>>t;
    while(t--) initwsolve();
}

```

KIỂM TRA CHU TRÌNH

Ôn lại

Cho đồ thị vô hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; Dòng tiếp theo đưa vào các bộ đôi $u \square V, v \square V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc "NO" kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Ví dụ:

Input:	Output:
1 6 9 1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	YES

Giải:

// theo như hình vẽ thì chu trình có 1 2 3 4 5 6.

// khi đó par là 5 nên nếu tìm được 1 đỉnh v nào đó khác 5 thì tức là được 1 chu trình

// => chu trình chính là 1 2 3 4 5 6 4.

// chuyển sang chu trình tìm từ đỉnh v nhưng par lúc này là đỉnh u trước đó

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,u,check[1005],cycle=0;
```

```
int a[1005][1005];
```

```
void dfs(int u,int par){
```

```
    check[u]= 1;
```

```
    for(int v=1;v<=V;v++){
```

```

        if(a[u][v]==1){
            if(check[v]==0)(dfs(v,u));//
            else if(check[v]==1&&v!=par)cycle=1;
        }
    }
}

void initwsolve(){
    cin>>V>>E;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    cycle=0;
    /*1*/ for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x][y]=1;
        a[y][x]=1;
    }
    for(int i=1;i<=V;i++){
        if(check[i]==0) dfs(i,0);
    }
    if(cycle==1)cout<<"YES"<<endl;
    else cout<<"NO"<<endl;
}

int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

KIỂM TRA CHU TRÌNH VỚI BFS

Cho đồ thị vô hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

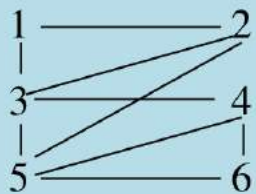
- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; Dòng tiếp theo đưa vào các bộ đôi uV, vV tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc “NO” kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Ví dụ:

Input:	Output:
1 6 9 1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	YES



Giải:

// theo như hình vẽ thì chu trình có 1 2 3 4 5 6.

// khi đó par là 5 nên nếu tìm được 1 đỉnh v nào đó khác 5 thì tức là được 1 chu trình

// => chu trình chính là 1 2 3 4 5 6 4.

// chuyển sang chu trình tìm từ đỉnh v nhưng par lúc này là đỉnh u trước đó

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,u,check[1005],cycle=0;
```

```
int a[1005][1005];
```

```
void dfs(int u,int par){
```

```
    check[u]= 1;
```

```
    for(int v=1;v<=V;v++){
```

```
        if(a[u][v]==1){
```

```
            if(check[v]==0)(dfs(v,u));//
```

```
            else if(check[v]==1&&v!=par)cycle=1;
```

```
        }
```

```
    }
```

```
}
```

```
void initwsolve(){
```

```
    cin>>V>>E;
```

```
    memset(a,0,sizeof(a));
```

```
    memset(check,0,sizeof(check));
```

```
    cycle=0;
```

```
    for(int i=1;i<=E;i++){
```

```

    int x,y;cin>>x>>y;
    a[x][y]=1;
    a[y][x]=1;
}
for(int i=1;i<=V;i++){
    if(check[i]==0) dfs(i,0);
}
if(cycle==1)cout<<"YES"<<endl;
else cout<<"NO"<<endl;
}

int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

KIỂM TRA CHU TRÌNH SỬ DỤNG DISJOIN SET

Cho đồ thị vô hướng $G=(V, E)$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng Disjoin Set, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; Dòng tiếp theo đưa vào các bộ đôi u, v tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc "NO" kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Ví dụ:

Input:	Output:
1 6 9 1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	YES

Giải : up code kiểm tra chu trình dfs cũng đúng:

```

#include<bits/stdc++.h>
using namespace std;
int V,E,u,check[1005],cycle=0;

```

```

int a[1005][1005];
void dfs(int u,int par){
    check[u]= 1;
    for(int v=1;v<=V;v++){
        if(a[u][v]==1){
            if(check[v]==0)(dfs(v,u));//
            else if(check[v]==1&&v!=par)cycle=1;
        }
    }
}
void initwsolve(){
    cin>>V>>E;
    memset(a,0,sizeof(a));
    memset(check,0,sizeof(check));
    cycle=0;
    /*1*/ for(int i=1;i<=E;i++){
        int x,y;cin>>x>>y;
        a[x][y]=1;
        a[y][x]=1;
    }
    for(int i=1;i<=V;i++){
        if(check[i]==0) dfs(i,0);
    }
    if(cycle==1)cout<<"YES"<<endl;
    else cout<<"NO"<<endl;
}

int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();
}

```

KIỂM TRA CHU TRÌNH TRÊN ĐỒ THỊ CÓ HƯỚNG VỚI DFS

Ôn lại

Cho đồ thị có hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh, số cạnh của đồ thị; Dòng tiếp theo đưa vào các bộ đôi uV , vV tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc “NO” kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Ví dụ:

Input:	Output:
1 6 9 1 2 2 4 3 1 3 2 3 5 4 3 5 4 5 6 6 4	YES

Giải :

//1: xác nhận đỉnh u đã được thăm trong quá trình duyệt cây gốc u

//2: tại các đỉnh v kề với đỉnh u thì nếu v chưa xét thì tiếp tục tìm đường bắt đầu từ v

//3: nếu gặp lại 1 đỉnh v trong quá trình duyệt cây dfs gốc u rồi, thì đồ thị này có chu trình => cycle=1;

//4 : kết thúc quá trình duyệt cây , tất cả các đỉnh đều cập nhật check[i]=2

```
#include<bits/stdc++.h>
```

```
using namespace std;
```

```
int V,E,u,check[1005],cycle=0;
```

```
int a[1005][1005];
```

```
void dfs(int u){
```

```
    check[u]= 1;//1
```

```
    for(int v=1;v<=V;v++){
```

```
        if(a[u][v]==1){
```

```
            if(check[v]==0) dfs(v);//2
```

```
            else if(check[v]==1) cycle=1;//3
```

```
        }
```

```
    }
```

```
    check[u]=2;
```

```
}
```

```
void initwsolve(){
```

```
    cin>>V>>E;
```

```
    memset(a,0,sizeof(a));
```

```
    memset(check,0,sizeof(check));
```

```
    cycle=0;
```

```

/* */ for(int i=1;i<=E;i++){
    int x,y;cin>>x>>y;
    a[x][y]=1;
}
for(int i=1;i<=V;i++){
    if(check[i]==0)dfs(i);
}
if(cycle==1)cout<<"YES"<<endl;
else cout<<"NO"<<endl;
}

int main(){
    int t;cin>>t;
    while(t--){
        initwsolve();
    }
}

```

ĐƯỜNG ĐI VÀ CHU TRÌNH EULER VỚI ĐỒ THỊ VÔ HƯỚNG

Ôn lại

Cho đồ thị vô hướng liên thông $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy kiểm tra xem đồ thị có đường đi Euler hay chu trình Euler hay không?

Đường đi Euler bắt đầu tại một đỉnh, và kết thúc tại một đỉnh khác.

Chu trình Euler bắt đầu tại một đỉnh, và kết thúc chu trình tại chính đỉnh đó.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; Dòng tiếp theo đưa vào các bộ đôi $u\hat{V}, v\hat{V}$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra 1, 2, 0 kết quả mỗi test theo từng dòng tương ứng với đồ thị có đường đi Euler, chu trình Euler và trường hợp không tồn tại.

Ví dụ:

Input:	Output:
2	
6 10	2
1 2 1 3 2 3 2 4 2 5 3 4 3 5 4 5 4 6 5 6	1
6 9	

1 2 1 3 2 3 2 4 2 5 3 4 3 5 4 5 4 6	
-------------------------------------	--

Giải :

//chú ý : chu trình euler khi tất cả các bậc của các đỉnh là bậc chẵn (ko có bậc lẻ nào). Đường đi euler khi có 2 đỉnh bậc lẻ . còn lại thì ko có ctrình hoặc đường đi euler

```
#include<bits/stdc++.h>
using namespace std;
vector< vector<int> > a;
int V,E,x,y;
void initwsolve(){
    a.clear();
    cin>>V>>E;
    a.resize(1005);
    for(int i=1;i<=E;i++){
        cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    int odd=0;
    for(int i=1;i<=V;i++){
        if(a[i].size()%2!=0) odd++;
        if(odd==0) cout<<"2\n";
        else if(odd==2) cout<<"1\n";
        else cout<<"0\n";
    }
}

int main(){
    int t; cin>>t;
    while(t--){initwsolve();}
    return 0;
}
```

CHU TRÌNH EULER TRONG ĐỒ THỊ CÓ HƯỚNG

Cho đồ thị có hướng liên thông yếu $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy kiểm tra xem đồ thị có chu trình Euler hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh, số cạnh của đồ thị; Dòng tiếp theo đưa vào các bộ đôi uV , vV tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra 1, 0 kết quả mỗi test theo từng dòng tương ứng với đồ thị có chu trình Euler và trường hợp không tồn tại đáp án.

Ví dụ:

Input:	Output:
2	
6 10	
1 2 2 4 2 5 3 1 3 2 4 3 4 5 5 3 5 6 6 4	1
3 3	0
1 2 2 3 1 3	

Giai:

```
#include<bits/stdc++.h>
using namespace std;
vector< vector<int> > a,b;
int V,E,x,y;
void initwsolve(){
    a.clear(),b.clear();
    cin>>V>>E;
    a.resize(1005);b.resize(1005);
    for(int i=1;i<=E;i++){
        cin>>x>>y;
        a[x].push_back(y);//số bán bậc ra đỉnh x
        b[y].push_back(x);//số bán bậc vào đỉnh y
    }
    for(int i=1;i<=V;i++){
        if(a[i].size()!=b[i].size()){
            cout<<"0\n";return;
        }
    }
    cout<<1<<endl;
}

int main(){
    int t; cin>>t;
    while(t--){initwsolve();
```

```

    return 0;
}

```

KIỂM TRA ĐỒ THỊ CÓ PHẢI LÀ CÂY HAY KHÔNG

Ôn lại

Một đồ thị N đỉnh là một cây, nếu như nó có đúng $N-1$ cạnh và giữa 2 đỉnh bất kì, chỉ tồn tại duy nhất 1 đường đi giữa chúng.

Cho một đồ thị N đỉnh và $N-1$ cạnh, hãy kiểm tra đồ thị đã cho có phải là một cây hay không?

Input:

- Dòng đầu tiên là số lượng bộ test T ($T \leq 20$).
- Mỗi test bắt đầu bởi số nguyên N ($1 \leq N \leq 1000$).
- $N-1$ dòng tiếp theo, mỗi dòng gồm 2 số nguyên u, v cho biết có cạnh nối giữa đỉnh u và v .

Output:

- Với mỗi test, in ra “YES” nếu đồ thị đã cho là một cây, in ra “NO” trong trường hợp ngược lại.

Ví dụ:

Input	Output
2	
4	
1 2	
1 3	YES
2 4	NO
4	
1 2	
1 3	
2 3	

Giải :

Cách 1:

```

#include<bits/stdc++.h>
using namespace std;
int V,E,u,check[1005],cycle=0;
int a[1005][1005];
void dfs(int u,int par){
    check[u]= 1;
    for(int v=1;v<=V;v++){
        if(a[u][v]==1){
            if(check[v]==0)(dfs(v,u));//
            else if(check[v]==1&&v!=par)cycle=1;
        }
    }
}
void initwsolve(){
    cin>>V;

```

```

int E=V-1;
memset(a,0,sizeof(a));
memset(check,0,sizeof(check));
cycle=0;
/*1*/ for(int i=1;i<=E;i++){
    int x,y;cin>>x>>y;
    a[x][y]=1;
    a[y][x]=1;
}
dfs(1,0);
for(int i=1;i<=V;i++)
    if(check[i]==0) cycle=1;
if(cycle==1)cout<<"NO"<<endl;
else cout<<"YES"<<endl;
}

int main(){
    int t;cin>>t;
    while(t-->0) initwsolve();

}

```

Cách 2:

```

#include<bits/stdc++.h>
using namespace std;
bool check[1005];
vector< vector<int> > a;
int V,E, x,y;
bool BFS(int u){
    vector<int> parent(1005, -1);
    queue<int> q;
    q.push(u);
    check[u]=true;
    while(!q.empty()){
        int s=q.front();
        q.pop();
        for(int i=0;i<a[s].size();i++){
            int t=a[s][i];

```

```

        if(check[t]==0){
            check[t]=1;
            q.push(t);
            parent[t]=s;
        }
        else if(parent[s]!=t) return 1;
    }
}
return 0;
}
bool initwsolve(){
    a.clear();
    cin>>V;
    a.resize(1005);
    memset(check,0,sizeof(check));
    for(int i=1;i<=V-1;i++){
        cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    if(BFS(1)) return 0;
    for(int i=1;i<=V;i++)
        if(!check[i]) return 0;
    return 1;
}
int main(){
    int t; cin>>t;
    while(t--){
        if(initwsolve()) cout<<"YES"<<endl;
        else cout<<"NO"<<endl;
    }
    return 0;
}

```

CÂY KHUNG CỦA ĐỒ THỊ THEO THUẬT TOÁN DFS

Ôn lại

Cho đồ thị vô hướng $G=(V, E)$. Hãy xây dựng một cây khung của đồ thị G với đỉnh $u \in V$ là gốc của cây bằng thuật toán DFS.

Input

Dòng đầu tiên gồm một số nguyên T ($1 \leq T \leq 20$) là số lượng bộ test.

Tiếp theo là T bộ test, mỗi bộ test có dạng sau:

- Dòng đầu tiên gồm 3 số nguyên $N=|V|$, $M=|E|$, u ($1 \leq N \leq 10^3$, $1 \leq M \leq 10^5$, $1 \leq u \leq N$).
- M dòng tiếp theo, mỗi dòng gồm 2 số nguyên a, b ($1 \leq a, b \leq N$, $a \neq b$) tương ứng cạnh nối hai chiều từ a tới b .
- Dữ liệu đảm bảo giữa hai đỉnh chỉ tồn tại nhiều nhất một cạnh nối.

Output

Với mỗi bộ test, nếu tồn tại cây khung thì in ra $N - 1$ cạnh của cây khung với gốc là đỉnh u trên $N - 1$ dòng theo thứ tự duyệt của thuật toán DFS. Ngược lại nếu không tồn tại cây khung thì in ra -1.

Ví dụ

Input	Output
2	
4 3 2	
1 2	2 1
1 3	1 3
2 4	3 4
3 4	-1
4 2 2	
1 2	
3 4	

Giải :

```
#include <bits/stdc++.h>
#define p pair<int,int>
using namespace std;
vector<int> a[1005];
int v,e,s;
bool check[1005];
vector<p> st;

void DFS(int u){
    check[u] = 1;
    for(int i=0 ; i<a[u].size();i++){
        int v = a[u][i];
        if(!check[v]){
            st.push_back(p(u,v));
            DFS(v);
        }
    }
}

void initwsolve(){
    memset(check,0,sizeof(check));
    st.clear();
    cin>>v>>e>>s;
```

```

    for(int i=0 ; i<1000 ; i++) a[i].clear();
    for(int i=0 ; i<e ; i++){
        int x,y; cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    DFS(s);
    if(st.size() != v-1) cout<<"-1"<<endl;
    else
        for(int i=0;i<st.size() ; i++)
            cout<<st[i].first<<" "<<st[i].second<<endl;
}
int main(){
    int t;cin>>t;
    while(t--){initwsolve();
}
}

```

CÂY KHUNG CỦA ĐỒ THỊ THEO THUẬT TOÁN BFS

Ôn lại

Cho đồ thị vô hướng $G=(V, E)$. Hãy xây dựng một cây khung của đồ thị G với đỉnh $u \in V$ là gốc của cây bằng thuật toán BFS.

Input

Dòng đầu tiên gồm một số nguyên T ($1 \leq T \leq 20$) là số lượng bộ test.

Tiếp theo là T bộ test, mỗi bộ test có dạng sau:

- Dòng đầu tiên gồm 3 số nguyên $N=|V|$, $M=|E|$, u ($1 \leq N \leq 10^3$, $1 \leq M \leq 10^5$, $1 \leq u \leq N$).
- M dòng tiếp theo, mỗi dòng gồm 2 số nguyên a, b ($1 \leq a, b \leq N$, $a \neq b$) tương ứng cạnh nối hai chiều từ a tới b .
- Dữ liệu đảm bảo giữa hai đỉnh chỉ tồn tại nhiều nhất một cạnh nối.

Output

Với mỗi bộ test, nếu tồn tại cây khung thì in ra $N - 1$ cạnh của cây khung với gốc là đỉnh u trên $N - 1$ dòng theo thứ tự duyệt của thuật toán BFS. Ngược lại nếu không tồn tại cây khung thì in ra -1.

Ví dụ

Input	Output
2	
4 4 2	
1 2	2 1
1 3	2 4
2 4	1 3
3 4	-1
4 2 2	
1 2	

Giải :

```
#include <bits/stdc++.h>
#define p pair<int,int>
using namespace std;
vector<int> a[1005];
int v,e,s;
bool check[1005];
vector<p> st;

void BFS(int u){
    queue<int> q;
    q.push(u);
    check[u] = 1;
    while(!q.empty()){
        int s = q.front(); q.pop();
        for(int i=0 ; i<a[s].size();i++){
            int t = a[s][i];
            if(!check[t]){
                check[t] = 1;
                q.push(t);
                st.push_back(p(s,t));
            }
        }
    }
}

void initwsolve(){
    memset(check,0,sizeof(check));
    st.clear();
    cin>>v>>e>>s;
    for(int i=0 ; i<1000 ; i++) a[i].clear();
    for(int i=0 ; i<e ; i++){
        int x,y; cin>>x>>y;
        a[x].push_back(y);
        a[y].push_back(x);
    }
    BFS(s);
    if(st.size() != v-1) cout<<"-1"<<endl;
```

```

else
    for(int i=0;i<st.size() ; i++)
        cout<<st[i].first<<" "<<st[i].second<<endl;
}
int main(){
    int t;cin>>t;
    while(t-->0)initwsolve();

}

```

ĐƯỜNG ĐI HAMILTON

Ôn lại

Đường đi đơn trên đồ thị có hướng hoặc vô hướng đi qua tất cả các đỉnh của đồ thị mỗi đỉnh đúng một lần được gọi là đường đi Hamilton. Cho đồ thị vô hướng $G = \langle V, E \rangle$, hãy kiểm tra xem đồ thị có đường đi Hamilton hay không?

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số V, E tương ứng với số đỉnh, số cạnh của đồ thị; phần thứ hai đưa vào các cạnh của đồ thị.
- T, V, E thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq V \leq 10$; $1 \leq E \leq 15$.

Output:

- Đưa ra 1 hoặc 0 tương ứng với test có hoặc không có đường đi Hamilton theo từng dòng.

Ví dụ:

Input	Output
2	1
4 4	0
1 2 2 3 3 4 2 4	
4 3	
1 2 2 3 2 4	

Giải :

```

#include<bits/stdc++.h>
using namespace std;
int n, m;
vector<int> a[1000];
bool check[100];
bool DFS(int i, int dem){
    if(dem == n)return 1;
    check[i] = 1;
    for(int j = 0; j < a[i].size(); j++){
        if(!check[a[i][j]] && DFS(a[i][j], dem + 1)) return 1;
    }
    check[i] = 0;
}

```

```

        return 0;
    }
    void initwsolve(){
        cin >> n >> m;
        for(int i = 1; i <= n; i++) a[i].clear();
        for(int i = 0; i < m; i++){
            int x, y;cin >> x >> y;
            a[x].push_back(y);a[y].push_back(x);
        }
        memset(check,false,sizeof(check));
        int stop = 0;
        for(int i = 1; i <= n; i++){
            if(DFS(i,1)){
                stop = 1;break;
            }
        }
        if(stop == 1)cout << "1\n";
        else cout << "0\n";
    }
    int main(){
        int t;
        cin >> t;
        while(t--)initwsolve();
    }
}

```

DIJKSTRA

Ôn lại

Cho đồ thị có trọng số không âm $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh trọng số. Hãy viết chương trình tìm đường đi ngắn nhất từ đỉnh $u \in V$ đến tất cả các đỉnh còn lại trên đồ thị.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai ba số $|V|$, $|E|$ tương ứng với số đỉnh và $u \in V$ là đỉnh bắt đầu; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ ba $u \in V$, $v \in V$, w tương ứng với một cạnh cùng với trọng số cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra kết quả của mỗi test theo từng dòng. Kết quả mỗi test là trọng số đường đi ngắn nhất từ đỉnh u đến các đỉnh còn lại của đồ thị theo thứ tự tăng dần các đỉnh.

Ví dụ:

Input:	Output:
1 9 12 1 1 2 4 1 8 8 2 3 8 2 8 11 3 4 7 3 6 4 3 9 2 4 5 9 4 6 14 5 6 10 6 7 2 6 9 6	0 4 12 19 21 11 9 8 14

Giải :

```
#include<bits/stdc++.h>
using namespace std;
#define p pair<int,int>
#define INF 1e9
vector< vector<p> > a;
int V,E,x;
void initwsolve(){
    a.clear();
    cin>>V>>E>>x;
    a.resize(1005);

    for(int i=1;i<=E;i++){
        int x,y,k;
        cin>>x>>y>>k;
        a[x].push_back(p(y,k));
        a[y].push_back(p(x,k));
    }
    priority_queue< p,vector<p>,greater<p> > pq;
    vector<int> dist(1005,INF);
    pq.push(p(0,x));
    dist[x]=0;
    while(!pq.empty()){
        int u=pq.top().second;
        pq.pop();
```

```

        for(int i=0;i<a[u].size();i++){
            int v=a[u][i].first;
            int doDai=a[u][i].second;

            if(dist[v]>dist[u]+doDai){
                dist[v]=dist[u]+doDai;
                pq.push(p(dist[v],v));
            }
        }
    }
    for(int i=1;i<=V;i++) cout<<dist[i]<<" ";
    cout<<endl;
}
int main(){
    int t;
    cin>>t;
    while(t-->0) initwsolve();
    return 0;
}

```

KRUSKAL

Ôn lại

Cho đồ thị vô hướng có trọng số $G=\langle V, E, W \rangle$. Nhiệm vụ của bạn là hãy xây dựng một cây khung nhỏ nhất của đồ thị bằng thuật toán Kruskal.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số V, E tương ứng với số đỉnh và số cạnh của đồ thị; phần thứ 2 đưa vào E cạnh của đồ thị, mỗi cạnh là một bộ 3: đỉnh đầu, đỉnh cuối và trọng số của cạnh.
- T, S, D thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq V \leq 100$; $1 \leq E, W \leq 10000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	4
3 3	5
1 2 5	
2 3 3	
1 3 1	

2 1	
1 2 5	

Giải:

```
#include <bits/stdc++.h>
using namespace std;
struct edge {
    int fir, en, var;
};
int v,e;
vector <edge> vv;
int p[1005];
int find(int x){
    if(p[x] == -1) return x;
    return find(p[x]);
}
void union1(int x, int y){
    p[x] = y;
}
bool cycle(vector<edge> ve){
    memset(p,-1,sizeof(p));
    for(int i=0 ; i<ve.size() ;i++){
        int x = find(ve[i].fir);
        int y = find(ve[i].en);
        if(x == y){
            return 1;
        }
        union1(x, y);
    }
    return 0;
}
bool cmp(edge e, edge ee){
    return e.var < ee.var;
}
void initwsolve(){
    cin>>v>>e;
    vv.clear();
    for (int i=0; i<e; i++){
```

```

        edge tt;
        cin>>tt.fir>>tt.en>>tt.var;
        vv.push_back(tt);
    }
    vector <edge> vvv;
    sort(vv.begin() , vv.end() ,cmp);
    vvv.push_back(vv[0]);
    int sum=vv[0].var;

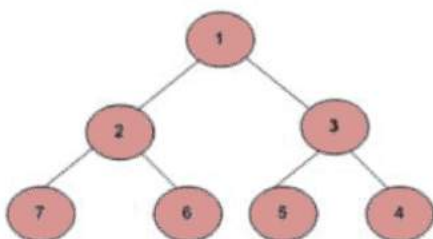
    for(int i=1 ; i<vv.size() ; i++){
        vvv.push_back(vv[i]);
        if(cycle(vvv)){
            vvv.pop_back();
        }
        else {
            sum += vv[i].var;
        }
    }
    cout<<sum<<endl;
}
int main (){
    int t;cin >> t;
    while(t--){initwsolve();
}

```

Cây nhị phân

KIỂM TRA NODE LÁ

Cho cây nhị phân, nhiệm vụ của bạn là kiểm tra xem tất cả các node lá của cây có cùng một mức hay không? Ví dụ với cây dưới đây sẽ cho ta kết quả là Yes.



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	
2	
1 2 R 1 3 L	1
4	0
10 20 L 10 30 R 20 40 L 20 60 R	

Giải:

```
#include<bits/stdc++.h>
using namespace std;
int n;
struct Tree{
    int value;
    Tree *left,*right;
};
Tree *newNode(int val){
    Tree *temp=new Tree;
    temp->left=temp->right=NULL;
    temp->value=val;
    return temp;
}
Tree* buildTree(){
    map<int,Tree*> mp;
    Tree *root = NULL;
    while(n--){
        int a,b;
        char x;
        cin>>a>>b>>x;
        Tree *cha = newNode(a);
        if(mp.find(a)==mp.end()){
```

```

        mp[a]=cha;
        if(root==NULL) root=cha;
    }
    else cha=mp[a];
    Tree *con=newNode(b);
    if(x=='L') cha->left=con;
    else if(x=='R')cha->right=con;
    mp[b]=con;
}
return root;
}
bool checkLevelLeaf(Tree* root){
    if (root==NULL)
        return true;
    queue<Tree*> q;
    q.push(root);
    int result = INT_MAX;
    int level = 0;
    while (!q.empty()) {
        int size = q.size();
        level += 1;
        while(size--){
            Tree* temp = q.front();
            q.pop();
            if (temp->left!=NULL) {
                q.push(temp->left);
                if(temp->left->right==NULL && temp->left->left==NULL){

                    if (result == INT_MAX)
                        result = level;

                    else if (result != level)
                        return false;
                }
            }
            if (temp->right!=NULL){
                q.push(temp->right);
                if (temp->right->left==NULL && temp->right->right==NULL) {

```

```

        if (result == INT_MAX)
            result = level;
        else if(result != level)
            return false;
    }
}
}
}
return true;
}
void initwsolve(){
    cin>>n;
    Tree *root=buildTree();
    if(checkLevelLeaf(root)) cout<<1<<endl;
    else cout<<0<<endl;
}
int main(){
    int t;
    cin>>t;
    while(t-->0)initwsolve();
    return 0;
}

```

CÂY BIỂU THỨC 1

Cây biểu thức là một cây nhị phân trong đó mỗi node trung gian là một phép toán, mỗi node lá là một toán hạng. Ví dụ với biểu thức $P = 3 + ((5+9)*2)$ sẽ được biểu diễn như cây dưới đây. Đối với cây biểu thức, duyệt theo thứ tự trước ta sẽ được biểu thức trung tố, duyệt theo thứ tự sau ta sẽ được biểu thức hậu tố, duyệt theo thứ tự giữa ta được biểu thức tiền tố. Chú ý, cây biểu thức luôn là cây nhị phân đầy (mỗi node trung gian đều có hai node con).

Cho biểu thức hậu tố P, hãy sử dụng cây biểu thức để đưa ra biểu thức trung tố tương ứng với biểu thức P.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test là một biểu thức hậu tố P.
- T, P thỏa mãn ràng buộc : $1 \leq T \leq 100$; $1 \leq \text{lengh}(P) \leq 100$.

Output:

- Đưa ra biểu thức trung tố tương ứng với P.

Ví dụ:

Input	Output
2 ab+ef*g* - wlrb+-*	a + b - e * f * g w * l - r + b

Giải:

```
#include<bits/stdc++.h>
using namespace std;
struct Tree{
    char value;
    Tree *left,*right;
};
bool isOp(char c){
    if(c=='+' || c=='-' || c=='*' || c=='/') return true;
    return false;
}
Tree *newNode(char val){
    Tree *temp=new Tree;
    temp->left=temp->right=NULL;
    temp->value=val;
    return temp;
}
void LNR(Tree *t){
    if(t!=NULL){
        LNR(t->left);
        cout<<t->value;
        LNR(t->right);
    }
}
void initwsolve(){
    string s; cin>>s;
    stack<Tree*> st;
    Tree *t,*t1,*t2;
    for(int i=0;i<s.length();i++){
        if(!isOp(s[i])){
```

```

        t=newNode(s[i]);
        st.push(t);
    }
    else{
        t=newNode(s[i]);
        if(!st.empty()){
            t1=st.top();
            st.pop();
        }
        if(!st.empty()){
            t2=st.top();
            st.pop();
        }
        t->left=t2;
        t->right=t1;
        st.push(t);
    }
}
if(!st.empty()) t=st.top();
LNR(t);
cout<<endl;
}

int main(){
    int t;
    cin>>t;
    while(t-->0) initwsolve();
    return 0;
}

```

CÂY BIỂU THỨC 2

Cho một cây biểu thức là một cây nhị phân đầy đủ bao gồm các phép toán $+$, $-$, $*$, $/$ và một số toán hạng có giá trị nguyên. Nhiệm vụ của bạn là hãy tính toán giá trị biểu thức được biểu diễn trên cây nhị phân đầy đủ. Ví dụ với cây dưới đây là biểu diễn của biểu thức $P = ((5*4) + (100-20))$ sẽ cho ta giá trị là 100.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test là gồm hai dòng: dòng thứ nhất đưa vào N là số lượng node của cây; dòng thứ hai đưa vào nội dung các node của cây; các node được viết cách nhau một vài khoảng trống. Các số có giá trị nguyên không vượt quá 1000.
- T, N, P thỏa mãn ràng buộc : $1 \leq T \leq 100$; $1 \leq N$, $\text{lenght}(P) \leq 100$.

Output:

- Đưa ra giá trị của cây biểu thức.

Ví dụ:

Input	Output
2 7 + * - 5 4 100 20 3 - 4 7	100 -3

Giải :

```
#include<bits/stdc++.h>
using namespace std;
int n;
string s[1005];
struct Tree{
    string value;
    Tree *left,*right;
};
int stringToNum(string a){
    int num=0;
    for(int i=0;i<a.length();i++) num=num*10+(a[i]-'0');
    return num;
}
Tree *newNode(string val){
    Tree *temp=new Tree;
    temp->left=temp->right=NULL;
    temp->value=val;
    return temp;
}
Tree *insertLevel(Tree* root,int i){
    if (i < n){
        Tree *temp = newNode(s[i]);
        root = temp;
    }
}
```

```

        root->left = insertLevel(root->left,2*i+1);
        root->right = insertLevel(root->right,2*i+2);
    }
    return root;
}
int cal(Tree *root){
    if(!root) return 0;
    if(!root->left && !root->right)
        return stringToNum(root->value);
    int l_val=cal(root->left);
    int r_val=cal(root->right);
    if(root->value=="+")
        return l_val+r_val;
    if (root->value=="-")
        return l_val-r_val;
    if (root->value=="*")
        return l_val*r_val;
    return l_val/r_val;
}
void initwsolve(){
    cin>>n;
    for(int i=0;i<n;i++) cin>>s[i];
    Tree *root=new Tree;
    root=insertLevel(root,0);
    cout<<cal(root)<<endl;
}
int main(){
    int t;
    cin>>t;
    while(t--){initwsolve();
    return 0;
}

```

DUYỆT CÂY 1

Cho phép duyệt cây nhị phân Inorder và Preorder, hãy đưa ra kết quả phép duyệt Postorder của cây nhị phân. Ví dụ với cây nhị phân có các phép duyệt cây nhị phân của cây dưới đây:

Inorder : 4 2 5 1 3 6

Preorder: : 1 2 4 5 3 6

Postorder : 4 5 2 6 3 1

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng node; dòng tiếp theo đưa vào N số theo phép duyệt Inorder; dòng cuối cùng đưa vào N số là kết quả của phép duyệt Preorder; các số được viết cách nhau một vài khoảng trống.
- T, N, node thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 1000$; $1 \leq \text{giá trị node} \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt Postorder theo từng dòng.

Ví dụ:

Input	Output
1 6 4 2 5 1 3 6 1 2 4 5 3 6	4 5 2 6 3 1

Giải:

```
#include<bits/stdc++.h>
using namespace std;
int innor[1005], preor[1005], n;
struct Tree {
    int value;
    Tree* left, * right;
};
Tree* newNode(int val) {
    Tree* temp = new Tree;
    temp->left = temp->right = NULL;
    temp->value = val;
    return temp;
}
Tree* buildTree(int start, int end, int &preIndex, map<int, int> & mp) {

    if (start > end) return NULL;

    Tree* t = newNode(preor[preIndex++]);
    if (start == end) return t;
```

```

        int inIndex = mp[t->value];
        t->left = buildTree(start, inIndex - 1, preIndex, mp);
        t->right = buildTree(inIndex + 1, end, preIndex, mp);
        return t;
    }
    Tree* buildTreeWrap() {
        map<int, int> mp;
        for (int i = 0; i < n; i++)
            mp[innor[i]] = i;

        int preIndex=0;
        return buildTree(0, n - 1, preIndex, mp);
    }
    void printPos(Tree* t) {
        if (t != NULL) {
            printPos(t->left);
            printPos(t->right);
            cout << t->value << " ";
        }
    }
    void initwsolve() {
        cin >> n;
        for (int i = 0; i < n; i++) cin >> innor[i];
        for (int i = 0; i < n; i++) cin >> preor[i];
        Tree* root = buildTreeWrap();
        printPos(root);
        cout << endl;
    }
    int main() {
        int t;
        cin >> t;
        while (t--) initwsolve();
        return 0;
    }

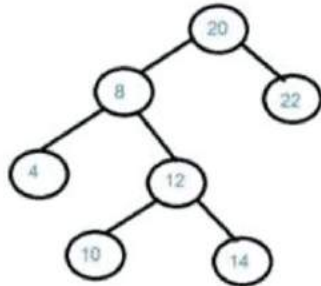
```

DUYỆT CÂY 2

Cho hai mảng là phép duyệt Inorder và Level-order, nhiệm vụ của bạn là xây dựng cây nhị phân và đưa ra kết quả phép duyệt Postorder. Level-order là phép duyệt theo từng mức của cây. Ví dụ như cây dưới đây ta có phép Inorder và Level-order như dưới đây:

Inorder : 4 8 10 12 14 20 22

Level order: 20 8 22 4 12 10 14



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng node; dòng tiếp theo đưa vào N số là phép duyệt Inorder; dòng cuối cùng đưa vào N số là phép duyệt Level-order; các số được viết cách nhau một vài khoảng trống.
- T, N, node thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq A[i] \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt Postorder theo từng dòng.

Ví dụ:

Input	Output
2	
3	
1 0 2	1 2 0
0 1 2	3 4 1 5 6 2 0
7	
3 1 4 0 5 2 6	
0 1 2 3 4 5 6	

Giải:

```

#include<bits/stdc++.h>
using namespace std;
int innor[1005], levelor[1005], n;
struct Tree {
    int value;

```

```

    Tree* left, * right;
};
Tree* newNode(int val) {
    Tree* temp = new Tree;
    temp->left = temp->right = NULL;
    temp->value = val;
    return temp;
}
Tree* buildTree(int start, int end, map<int, int> mp) {
    if (start > end) return NULL;
    int index=start;
    for(int j=start+1;j<=end;j++){
        if(mp[innor[j]]<mp[innor[index]]) index=j;
    }
    Tree* t = newNode(innor[index]);
    t->left = buildTree(start, index - 1, mp);
    t->right = buildTree(index + 1, end, mp);
    return t;
}
Tree* buildTreeWrap() {
    map<int, int> mp;
    for (int i = 0; i < n; i++)
        mp[levelor[i]] = i;
    return buildTree(0, n - 1, mp);
}
void printPos(Tree* t) {
    if (t != NULL) {
        printPos(t->left);
        printPos(t->right);
        cout << t->value << " ";
    }
}
void initwsolve() {
    cin >> n;
    for (int i = 0; i < n; i++) cin >> innor[i];
    for (int i = 0; i < n; i++) cin >> levelor[i];
    Tree* root = buildTreeWrap();
    printPos(root);
}

```

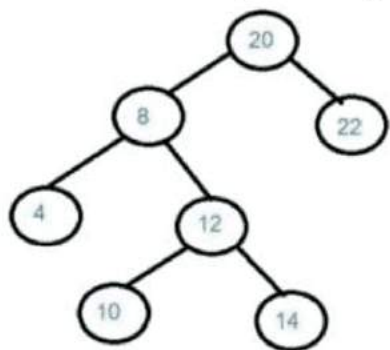
```

        cout << endl;
    }
    int main() {
        int t;
        cin >> t;
        while (t--) initwsolve();
        return 0;
    }

```

DUYỆT CÂY THEO MỨC

Cho cây nhị phân, nhiệm vụ của bạn là duyệt cây theo Level-order. Phép duyệt level-order trên cây là phép thăm node theo từng mức của cây. Ví dụ với cây dưới đây sẽ cho ta kết quả của phép duyệt level-order: 20 8 22 4 12 10 14.



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt level-order theo từng dòng.

Ví dụ:

Input	Output
2	
2	1 3 2
1 2 R 1 3 L	10 0 30 40 60
4	

10 20 L 10 30 R 20 40 L 20 60 R	
---------------------------------	--

Giải :

```
#include<bits/stdc++.h>
using namespace std;
int n;
struct Tree{
    int value;
    Tree *left,*right;
};
Tree *newNode(int val){
    Tree *temp=new Tree;
    temp->left=temp->right=NULL;
    temp->value=val;
    return temp;
}
Tree* buildTree(){
    map<int,Tree*> mp;
    Tree *root = NULL;
    while(n--){
        int a,b;
        char x;
        cin>>a>>b>>x;
        Tree *cha = newNode(a);
        if(mp.find(a)==mp.end()){
            mp[a]=cha;
            if(root==NULL) root=cha;
        }
        else cha=mp[a];

        Tree *con=newNode(b);
        if(x=='L') cha->left=con;
        else if(x=='R')cha->right=con;
        mp[b]=con;
    }
    return root;
}
void levelOrder(Tree *root){
    if(root==NULL) return;
```

```

queue<Tree*> q;
q.push(root);
while(!q.empty()){
    Tree* t=q.front();
    cout<<t->value<<" ";
    q.pop();
    if(t->left!=NULL) q.push(t->left);
    if(t->right!=NULL) q.push(t->right);
}
}
void initwsolve(){
    cin>>n;
    Tree *root=buildTree();
    levelOrder(root);
    cout<<endl;
}

int main(){
    int t; cin>>t;
    while(t-->0) initwsolve();
    return 0;
}

```

DUYỆT CÂY NHỊ PHÂN TÌM KIẾM 1

Cho mảng $A[]$ gồm N node là biểu diễn phép duyệt theo thứ tự giữa (Preorder) của cây nhị phân tìm kiếm. Nhiệm vụ của bạn là đưa ra phép duyệt theo thứ tự sau của cây nhị phân tìm kiếm.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng node; dòng tiếp theo đưa vào N số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- T, N , node thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq A[i] \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt Postorder theo từng dòng.

Ví dụ:

Input	Output
-------	--------

2	
5	35 30 100 80 40
40 30 35 80 100	35 32 30 120 100 90 80 40
8	
40 30 32 35 80 90 100 120	

Giải:

```
#include<bits/stdc++.h>
using namespace std;
int n,preor[1005];
struct Tree {
    int value;
    Tree* left, * right;
};
Tree* newNode(int val) {
    Tree* temp = new Tree;
    temp->left = temp->right = NULL;
    temp->value = val;
    return temp;
}
Tree* buildTree(int min, int max, int &preIndex) {
    if (preIndex>=n) return NULL;
    int val=preor[preIndex];
    if(val<min || val>max) return NULL;
    Tree* t = newNode(val);
    preIndex++;
    t->left = buildTree(min, val - 1,preIndex);
    t->right = buildTree(val + 1, max,preIndex);
    return t;
}
Tree* buildTreeWrap() {
    int preIndex=0;
    return buildTree(INT_MIN,INT_MAX, preIndex);
}
void printPos(Tree* t) {
    if(t==NULL) return;
    if (t != NULL) {
        printPos(t->left);
```

```

        printPos(t->right);
        cout << t->value << " ";
    }
}

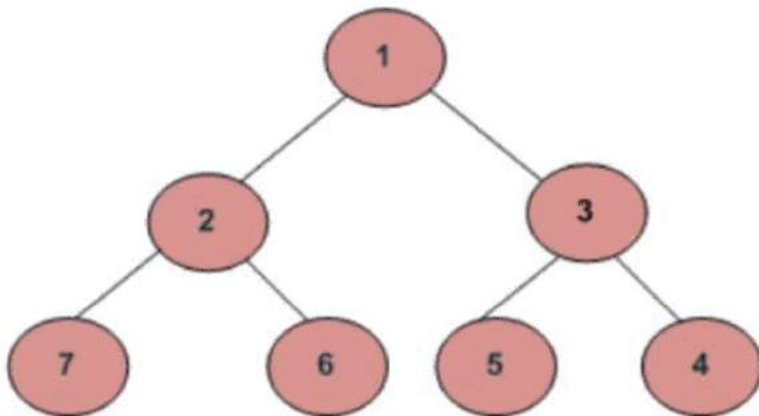
void initwsolve() {
    cin >> n;
    for (int i = 0; i < n; i++) cin >> preor[i];
    Tree* root = buildTreeWrap();
    printPos(root);
    cout << endl;
}

int main() {
    int t;
    cin >> t;
    while (t--) initwsolve();
}

```

DUYỆT CÂY THEO MỨC ĐẢO NGƯỢC

Cho cây nhị phân, nhiệm vụ của bạn là duyệt cây theo mức đảo ngược (reverse-level-order). Với cây dưới đây sẽ cho ta kết quả của phép duyệt theo mức đảo ngược là : 7 6 5 4 3 2 1.



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.

- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt reverse-level-order theo từng dòng.

Ví dụ:

Input	Output
2 2 1 2 R 1 3 L 4 10 20 L 10 30 R 20 40 L 20 60 R	3 2 1 40 20 30 10

Giải :

```
#include<bits/stdc++.h>
using namespace std;
int n;
struct Tree{
    int value;
    Tree *left,*right;
};
Tree *newNode(int val){
    Tree *temp=new Tree;
    temp->left=temp->right=NULL;
    temp->value=val;
    return temp;
}
Tree* buildTree(){
    map<int,Tree*> mp;
    Tree *root = NULL;
    while(n--){
        int a,b;
        char x;
        cin>>a>>b>>x;
        Tree *cha = newNode(a);
        if(mp.find(a)==mp.end()){
            mp[a]=cha;
            if(root==NULL) root=cha;
        }
        else cha=mp[a];
```

```

        Tree *con=newNode(b);
        if(x=='L') cha->left=con;
        else if(x=='R')cha->right=con;
        mp[b]=con;
    }
    return root;
}

void levelOrder(Tree *root){
    if(root==NULL) return;
    queue<Tree*> q;
    stack<int> st;
    q.push(root);
    while(!q.empty()){
        Tree* t=q.front();
        st.push(t->value);
        q.pop();
        if(t->right!=NULL) q.push(t->right);
        if(t->left!=NULL) q.push(t->left);
    }
    while(!st.empty()){
        cout<<st.top()<<" ";
        st.pop();
    }
}

void initwsolve(){
    cin>>n;
    Tree *root=buildTree();
    levelOrder(root);
    cout<<endl;
}

int main(){
    int t;cin>>t;
    while(t-->0)initwsolve();
}

```

DUYỆT CÂY KIỂU XOẢN ỐC (duyệt cây 7)

Cho cây nhị phân, nhiệm vụ của bạn là duyệt cây theo xoắn ốc (spiral-order).
 Phép. Ví dụ với cây dưới đây sẽ cho ta kết quả của phép duyệt spiral-order: 1 2
 3 4 5 6 7.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt level-order theo từng dòng.

Ví dụ:

Input	Output
2	
2	
1 2 R 1 3 L	1 3 2
4	10 0 30 60 40
10 20 L 10 30 R 20 40 L 20 60 R	

Giải:

```
#include<bits/stdc++.h>
using namespace std;
int n;
struct Tree{
    int value;
    Tree *left,*right;
};
Tree *newNode(int val){
    Tree *temp=new Tree;
    temp->left=temp->right=NULL;
    temp->value=val;
    return temp;
}
Tree* buildTree(){
    map<int,Tree*> mp;
    Tree *root = NULL;
    while(n--){
        int a,b;
```

```

    char x;
    cin>>a>>b>>x;
    Tree *cha = newNode(a);
    if(mp.find(a)==mp.end()){
        mp[a]=cha;
        if(root==NULL) root=cha;
    }
    else cha=mp[a];

    Tree *con=newNode(b);
    if(x=='L') cha->left=con;
    else if(x=='R')cha->right=con;
    mp[b]=con;
}
return root;
}

void levelOrder(Tree *root){
    if(root==NULL) return;
    deque<Tree*> dq;
    dq.push_front(root);
    bool flag=false;
    while(!dq.empty()){
        int dsize=dq.size();
        if(flag){
            while(dsize--){
                Tree* t=dq.front();
                dq.pop_front();
                cout<<t->value<<" ";
                if(t->left!=NULL) dq.push_back(t->left);
                if(t->right!=NULL) dq.push_back(t->right);
            }
        }
        else{
            while(dsize--){
                Tree* t=dq.back();
                dq.pop_back();
                cout<<t->value<<" ";
                if(t->right!=NULL) dq.push_front(t->right);
            }
        }
    }
}

```

```

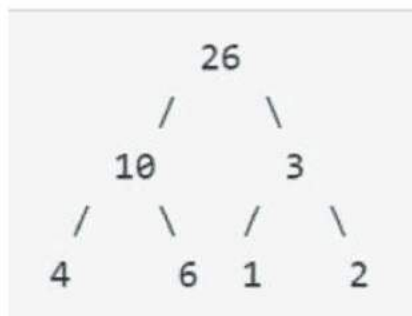
        if(t->left!=NULL) dq.push_front(t->left);
    }

    }
    flag=!flag;
}
}
void initwsolve(){
    cin>>n;
    Tree *root=buildTree();
    levelOrder(root);
    cout<<endl;
}
int main(){
    int t; cin>>t;
    while(t-->0) initwsolve();
    return 0;
}

```

CÂY NHỊ PHÂN TỔNG

Cho cây nhị phân, nhiệm vụ của bạn là kiểm tra xem cây nhị phân có phải là một cây tổng hay không? Một cây nhị phân được gọi là cây tổng nếu tổng các node con của node trung gian bằng giá trị node cha. Node không có node con trái hoặc phải được hiểu là có giá trị 0. Ví dụ dưới đây là một cây tổng



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.

- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ví dụ:

Input	Output
2	
2	
3 1 L 3 2 R	1
4	0
10 20 L 10 30 R 20 40 L 20 60 R	

Giải :

```
#include<bits/stdc++.h>
using namespace std;
int n;
struct Tree{
    int value;
    Tree *left,*right;
};
Tree *newNode(int val){
    Tree *temp=new Tree;
    temp->left=temp->right=NULL;
    temp->value=val;
    return temp;
}
Tree* buildTree(){
    map<int,Tree*> mp;
    Tree *root = NULL;
    while(n--){
        int a,b;
        char x;
        cin>>a>>b>>x;
        Tree *cha = newNode(a);
        if(mp.find(a)==mp.end()){
            mp[a]=cha;
            if(root==NULL) root=cha;
        }
        else cha=mp[a];
```

```

        Tree *con=newNode(b);
        if(x=='L') cha->left=con;
        else if(x=='R')cha->right=con;
        mp[b]=con;
    }
    return root;
}
int sum(Tree* root){
    if(root==NULL) return 0;
    return sum(root->left) + root->value + sum(root->right);
}
bool checkSumTree(Tree* root){
    int leftData=0,rightData=0;
    if(root==NULL || (root->left==NULL && root->right==NULL)) return true;
    leftData=sum(root->left);
    rightData=sum(root->right);
    if((root->value==leftData+rightData) && checkSumTree(root->left)
    && checkSumTree(root->right)) return true;
    return false;
}

void initwsolve(){
    cin>>n;
    Tree *root=buildTree();
    if(checkSumTree(root)) cout<<1<<endl;
    else cout<<0<<endl;
}

int main(){
    int t; cin>>t;
    while(t--){initwsolve();
    return 0;
}

```